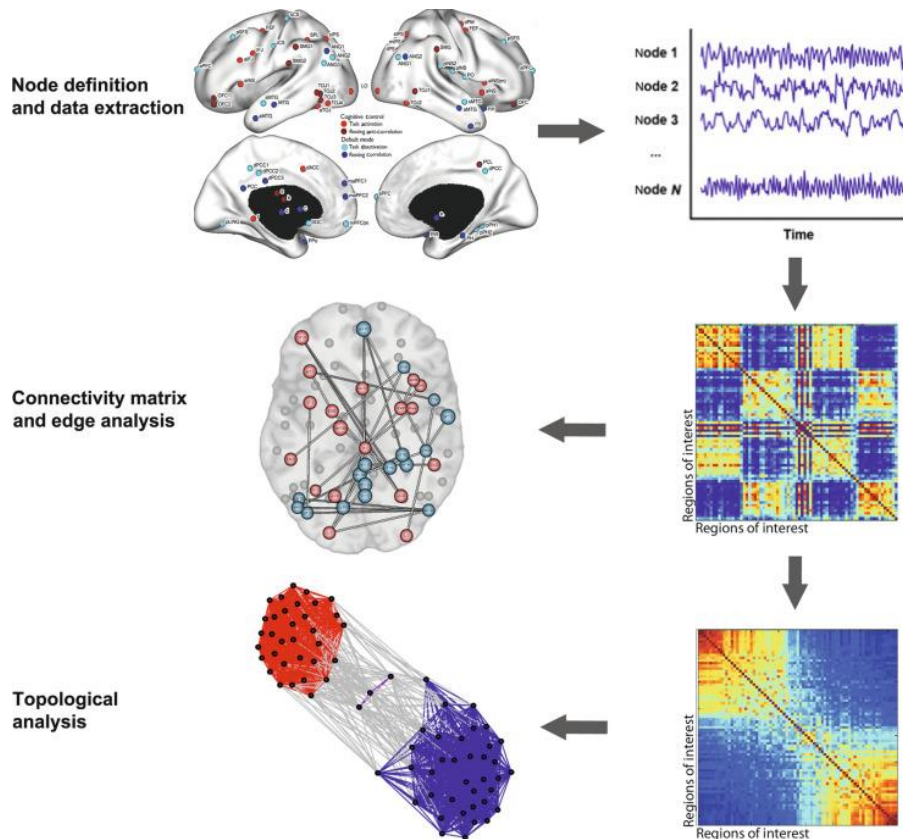




GRAPHS & NETWORKS

Ali Moltajaei Farid

Motivation

Graph theory (GT) applications in computer science

Algorithm execution modelling

GSM network modelling and analysis

Services connectivity analysis

Web documents clustering

WSN modelling

Scheduling problems modelling and analysis

Optimization problems solving

IoT modelling and problem analysis

Computer networks topology generation

Web pages ranking/link analysis

Operating system function modelling

Encryption process phases analysis

P2p network modelling and uses in blockchain

Application in image analysis

GRAPH TERMINOLOGY

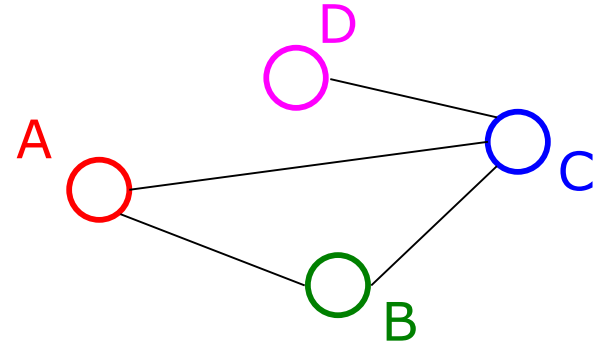
Undirected Graphs

In **undirected graphs**, edges have no specific direction

- Edges are always "two-way"

Thus, $(u, v) \in E$ implies $(v, u) \in E$.

- Only one of these edges needs to be in the set
- The other is implicit, so normalize how you check for it

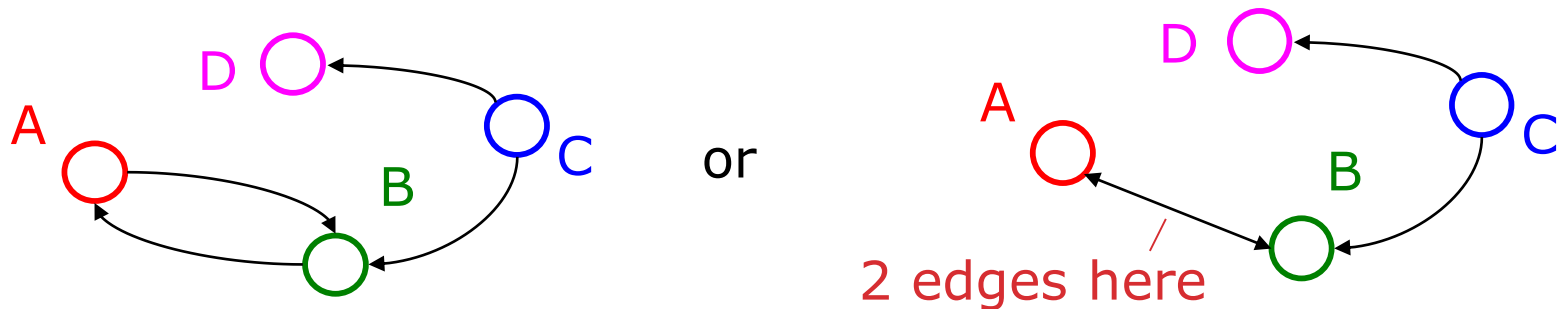


Degree of a vertex: number of edges containing that vertex

- Put another way: the number of adjacent vertices

Directed Graphs

In **directed graphs** (or **digraphs**), edges have direction



Thus, $(u, v) \in E$ does not imply $(v, u) \in E$.

Let $(u, v) \in E$ mean $u \rightarrow v$

- Call u the **source** and v the **destination**
- **In-Degree** of a vertex: number of in-bound edges (edges where the vertex is the destination)
- **Out-Degree** of a vertex: number of out-bound edges (edges where the vertex is the source)

Self-Edges, Connectedness

A **self-edge** a.k.a. a **loop** edge is of the form (u, u)

- The use/algorithm usually dictates if a graph has:
 - No self edges
 - Some self edges
 - All self edges

A node can have a(n) degree / in-degree / out-degree of **zero**

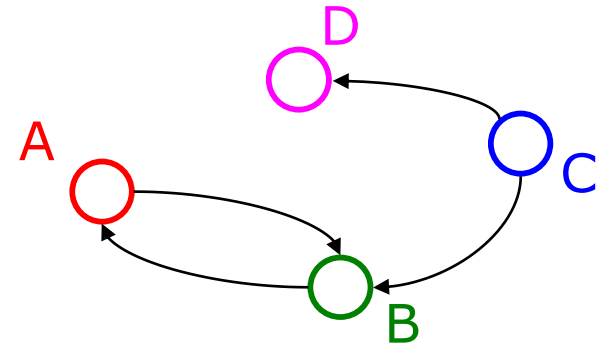
A graph does not have to be **connected**

- Even if every node has non-zero degree
- More discussion of this to come

More Notation

For a graph $G = (V, E)$:

- $|V|$ is the number of vertices
- $|E|$ is the number of edges
 - Minimum?
 - Maximum for undirected?
 - Maximum for directed?



$V = \{A, B, C, D\}$
 $E = \{(C, B), (A, B), (B, A), (C, D)\}$

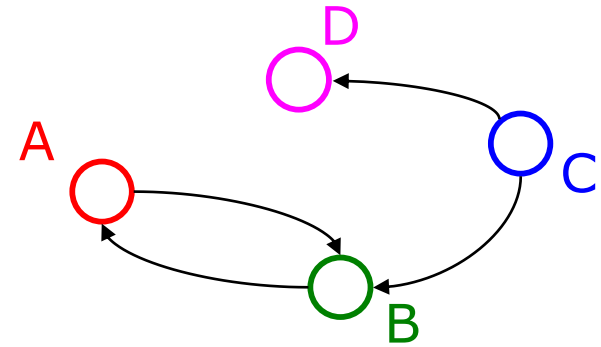
If $(u, v) \in E$, then v is a **neighbor** of u (i.e., v is **adjacent** to u)

- Order matters for directed edges:
 u is not adjacent to v unless $(v, u) \in E$

More Notation

For a graph $G = (V, E)$:

- $|V|$ is the number of vertices
- $|E|$ is the number of edges
 - Minimum? 0
 - Maximum for undirected? $|V||V+1|/2 \in O(|V|^2)$
 - Maximum for directed? $|V|^2 \in O(|V|^2)$



If $(u, v) \in E$, then v is a **neighbor** of u (i.e., v is **adjacent** to u)

- Order matters for directed edges:
 u is not adjacent to v unless $(v, u) \in E$

Examples Again

Which would use **directed edges**?

Which would have **self-edges**?

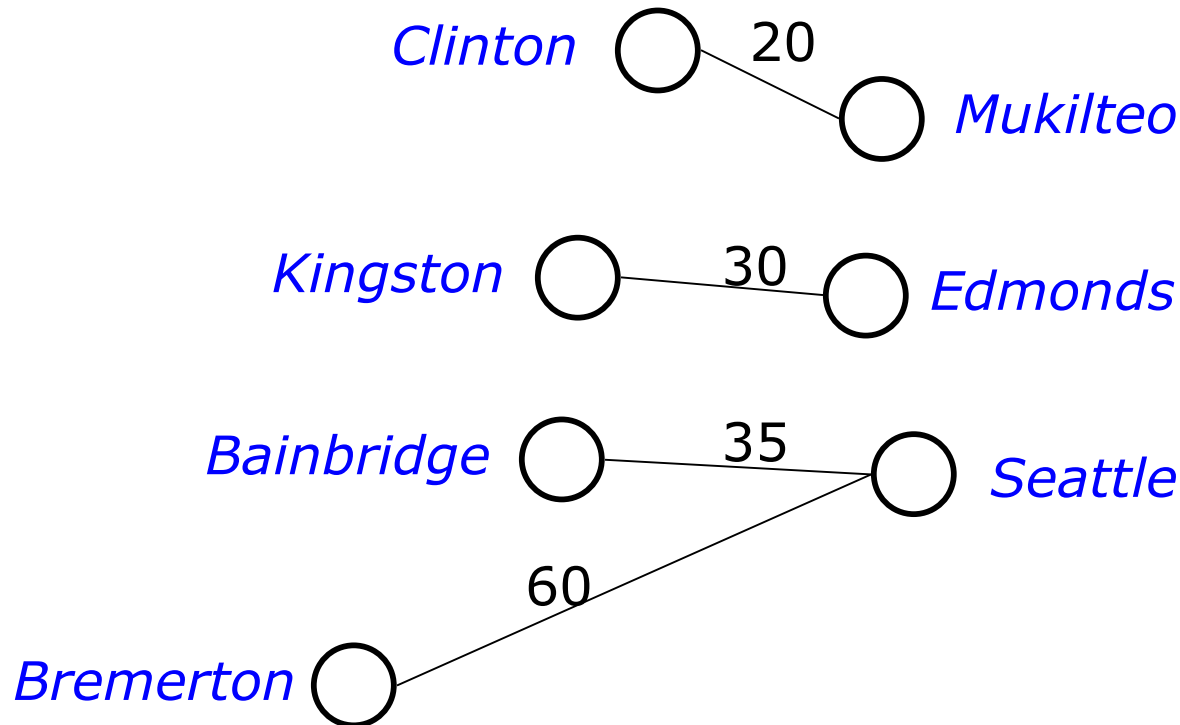
Which could have **0-degree nodes**?

- Web pages with links
- Facebook friends
- "Input data" for the Kevin Bacon game
- Methods in a program that call each other
- Road maps
- Airline routes
- Family trees
- Course pre-requisites

Weighted Graphs

In a weighted graph, each edge has a **weight** or **cost**

- Typically numeric (ints, decimals, doubles, etc.)
- Orthogonal to whether graph is directed
- Some graphs allow negative weights; many do not



Examples Again

What, if anything, might **weights** represent for each of these?

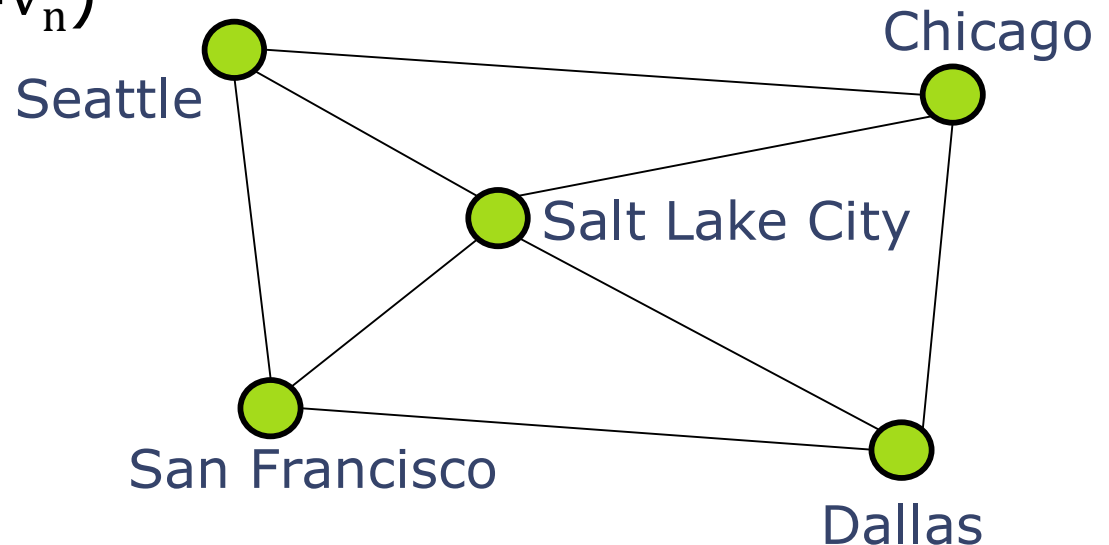
Do **negative weights** make sense?

- Web pages with links
- Facebook friends
- "Input data" for the Kevin Bacon game
- Methods in a program that call each other
- Road maps
- Airline routes
- Family trees
- Course pre-requisites

Paths and Cycles

We say "a **path** exists from v_0 to v_n " if there is a list of vertices $[v_0, v_1, \dots, v_n]$ such that $(v_i, v_{i+1}) \in E$ for all $0 \leq i < n$.

A **cycle** is a path that begins and ends at the same node ($v_0 = v_n$)



Example path (that also happens to be a cycle):

[Seattle, Salt Lake City, Chicago, Dallas, San Francisco, Seattle]

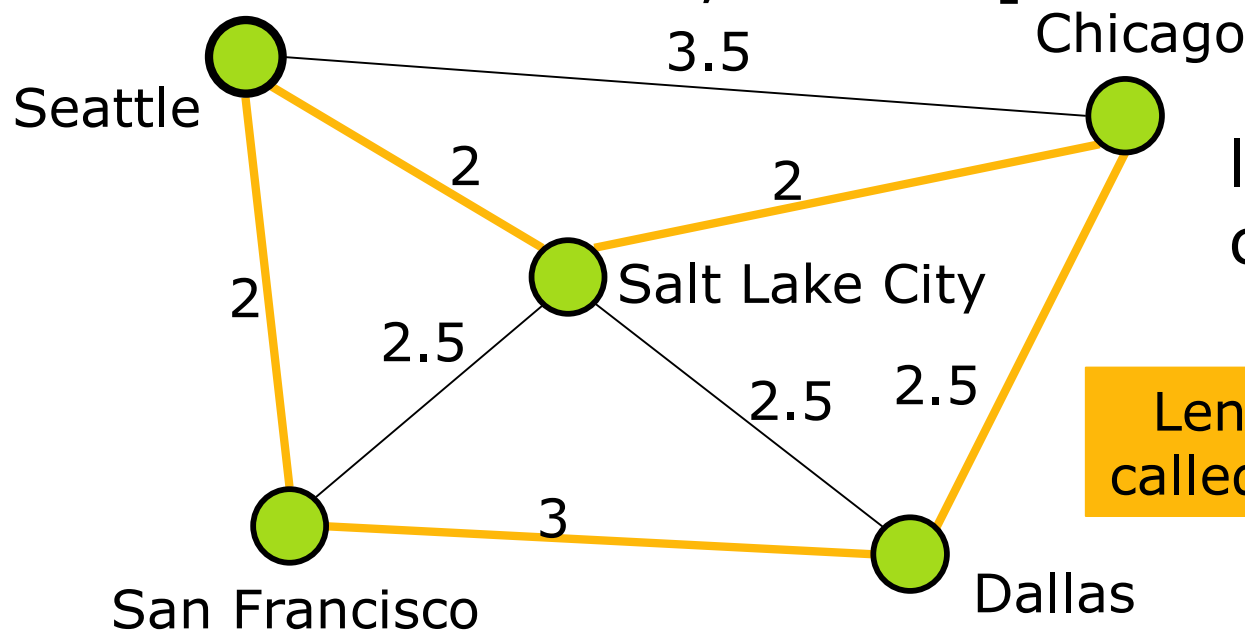
Path Length and Cost

Path length: Number of edges in a path

Path cost: Sum of the weights of each edge

Example where

$P = [\text{Seattle, Salt Lake City, Chicago, Dallas, San Francisco, Seattle}]$



$\text{length}(\mathbf{P}) = 5$
 $\text{cost}(\mathbf{P}) = 11.5$

Length is sometimes called "unweighted cost"

Simple Paths and Cycles

A **simple path** repeats no vertices (except the first might be the last):

[Seattle, Salt Lake City, San Francisco, Dallas]

[Seattle, Salt Lake City, San Francisco, Dallas, Seattle]

A **cycle** is a path that ends where it begins:

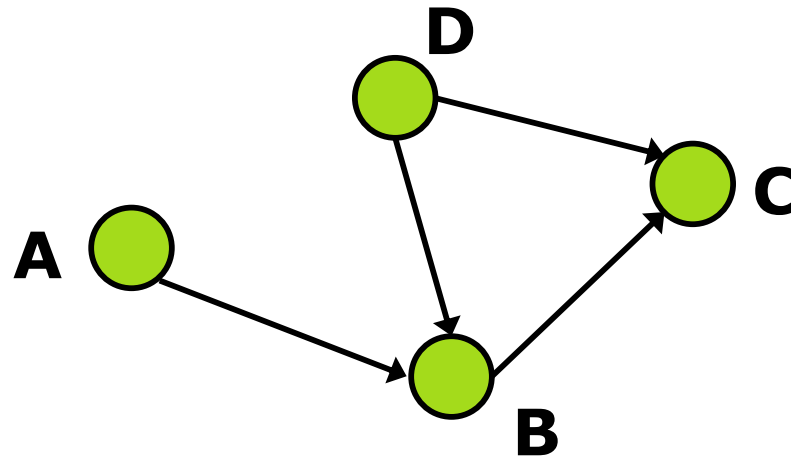
[Seattle, Salt Lake City, Seattle, Dallas, Seattle]

A **simple cycle** is a cycle and a simple path:

[Seattle, Salt Lake City, San Francisco, Dallas, Seattle]

Paths and Cycles in Directed Graphs

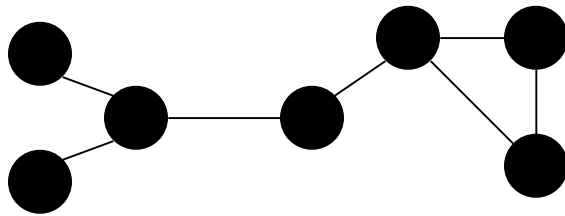
Example:



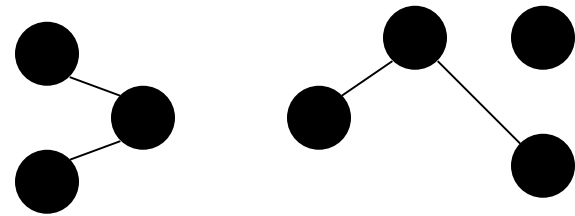
- Is there a path from A to D? **No**
- Does the graph contain any cycles? **No**

Undirected Graph Connectivity

An undirected graph is **connected** if for all pairs of vertices $u \neq v$, there exists a *path* from u to v

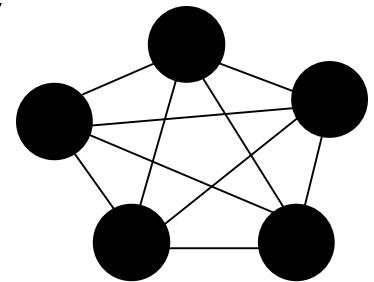


Connected graph



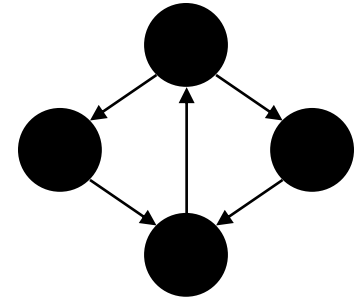
Disconnected graph

An undirected graph is **complete**, or **fully connected**, if for all pairs of vertices $u \neq v$ there exists an *edge* from u to v

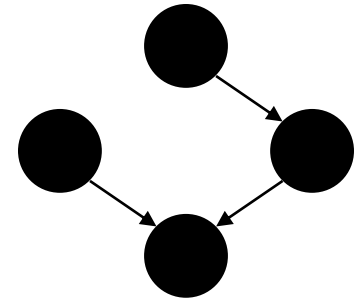


Directed Graph Connectivity

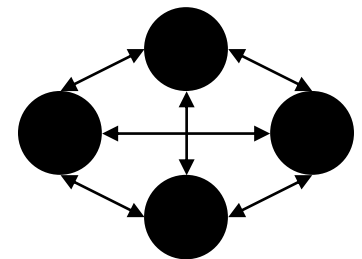
A directed graph is **strongly connected** if there is a path from every vertex to every other vertex



A directed graph is **weakly connected** if there is a path from every vertex to every other vertex *ignoring direction of edges*



A direct graph is **complete** or **fully connected**, if for all pairs of vertices $u \neq v$, there exists an *edge* from u to v



Examples Again

For undirected graphs: connected?

For directed graphs: strongly connected?
weakly connected?

- Web pages with links
- Facebook friends
- "Input data" for the Kevin Bacon game
- Methods in a program that call each other
- Road maps
- Airline routes
- Family trees
- Course pre-requisites

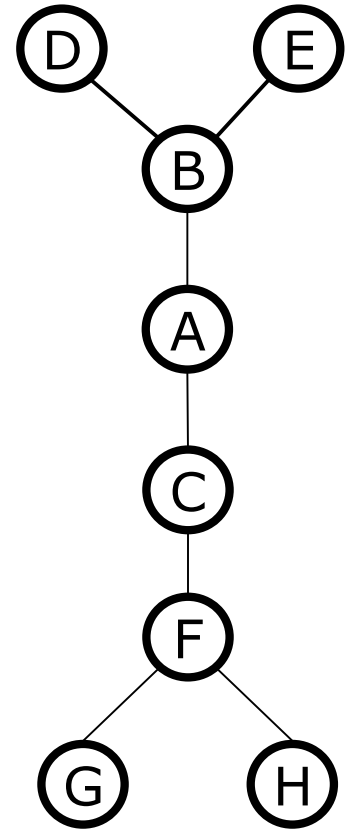
Trees as Graphs

When talking about graphs, we say a **tree** is a graph that is:

- undirected
- acyclic
- connected

All trees are graphs, but NOT all graphs are trees

How does this relate to the trees we know and "love"?



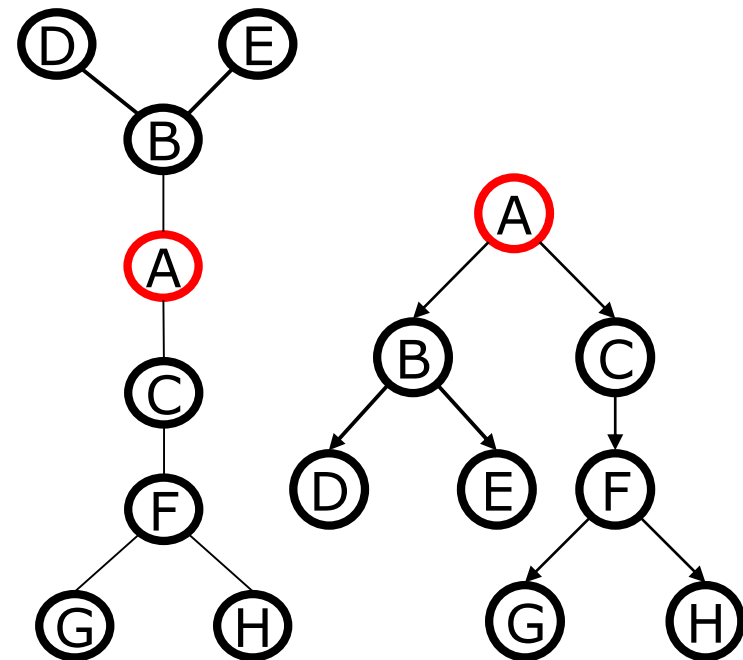
Rooted Trees

We are more accustomed to **rooted trees** where:

- We identify a unique **root**
- We think of edges as directed: parent to children

Picking a root gives a unique rooted tree

- The tree is simply drawn differently and with undirected edges



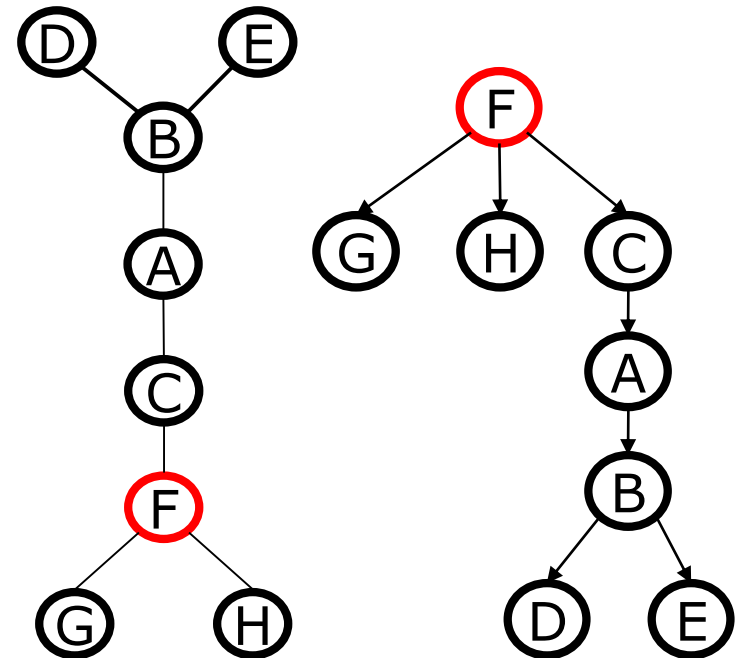
Rooted Trees

We are more accustomed to **rooted trees** where:

- We identify a unique **root**
- We think of edges as directed: parent to children

Picking a root gives a unique rooted tree

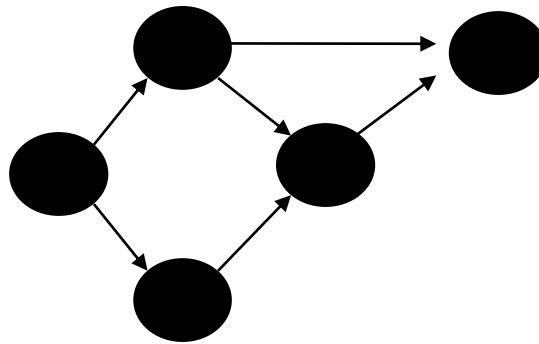
- The tree is simply drawn differently and with undirected edges



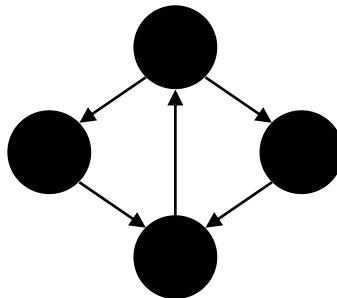
Directed Acyclic Graphs (DAGs)

A **DAG** is a directed graph with no directed cycles

- Every rooted directed tree is a DAG
- But not every DAG is a rooted directed tree



- Every DAG is a directed graph
- But not every directed graph is a DAG



Examples Again

Which of our **directed-graph** examples do you expect to be a **DAG**?

- Web pages with links
- Facebook friends
- "Input data" for the Kevin Bacon game
- Methods in a program that call each other
- Road maps
- Airline routes
- Family trees
- Course pre-requisites

Density / Sparsity

Recall:

In an undirected graph, $0 \leq |E| < |V|^2$

Recall:

In a directed graph, $0 \leq |E| \leq |V|^2$

So for any graph, $|E|$ is $O(|V|^2)$

Another fact:

If an undirected graph is *connected*,
then $|E| \geq |V| - 1$ (pigeonhole principle)

Density / Sparsity

$|E|$ is often much smaller than its maximum size

We do not always approximate as $|E|$ as $O(|V|^2)$

- This is a correct bound, but often not tight

If $|E|$ is $\Theta(|V|^2)$ (the bound is tight), we say the graph is **dense**

- More sloppily, dense means "lots of edges"

If $|E|$ is $O(|V|)$ we say the graph is **sparse**

- More sloppily, sparse means "most possible edges missing"

Insert humorous statement here

GRAPH DATA STRUCTURES

What's the Data Structure?

Graphs are often useful for lots of data and questions

- Example: "What's the lowest-cost path from x to y"

But we need a data structure that represents graphs

Which data structure is "best" can depend on:

- properties of the graph (e.g., dense versus sparse)
- the common queries about the graph ("is (u,v) an edge?" vs "what are the neighbors of node u ?")

We will discuss two standard graph representations

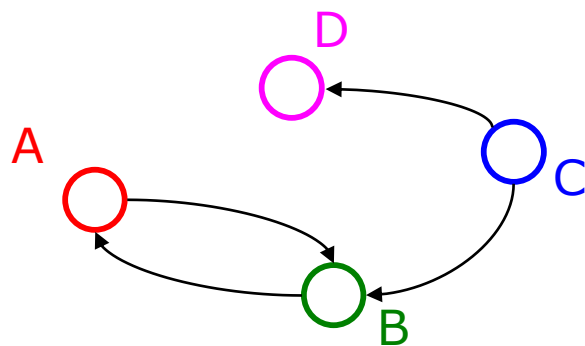
- **Adjacency Matrix** and **Adjacency List**
- Different trade-offs, particularly time versus space

Adjacency Matrix

Assign each node a number from 0 to $|V|-1$

A $|V| \times |V|$ matrix of Booleans (or 0 vs. 1)

- Then $M[u][v] == \text{true}$ means there is an edge from u to v



	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

Adjacency Matrix Properties

Running time to:

- Get a vertex's out-edges:
- Get a vertex's in-edges:
- Decide if some edge exists:
- Insert an edge:
- Delete an edge:

Space requirements:

	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

Best for sparse or dense graphs?

Adjacency Matrix Properties

Running time to:

- Get a vertex's out-edges: $O(|V|)$
- Get a vertex's in-edges: $O(|V|)$
- Decide if some edge exists: $O(1)$
- Insert an edge: $O(1)$
- Delete an edge: $O(1)$

Space requirements:

$O(|V|^2)$

	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

Best for sparse or dense graphs? *dense*

Adjacency Matrix Properties

How will the adjacency matrix vary for an **undirected graph**?

- Will be symmetric about diagonal axis
- Matrix: Could we save space by using only about half the array?

	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	T	F

- But how would you "get all neighbors"?

Adjacency Matrix Properties

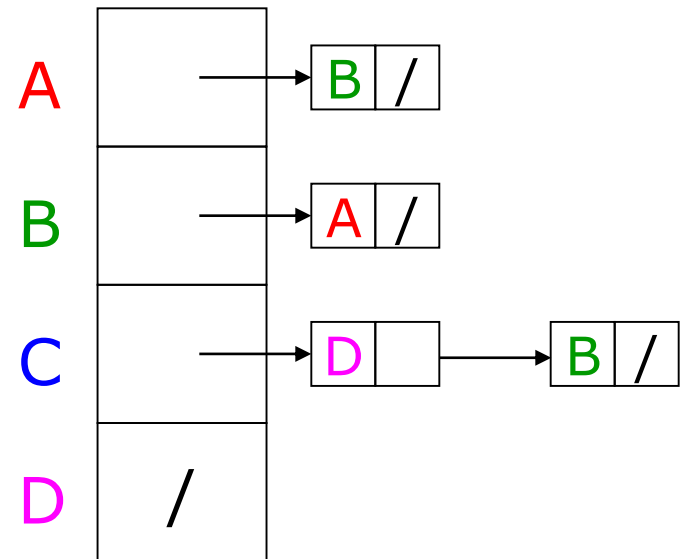
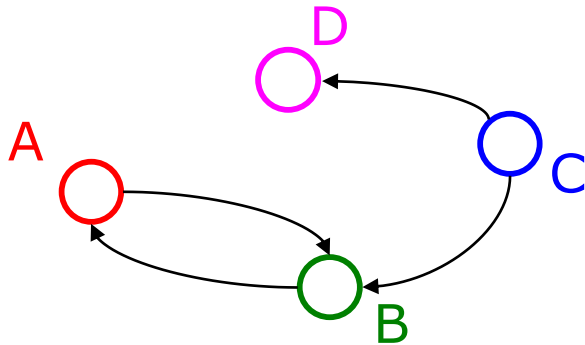
How can we adapt the representation for **weighted graphs**?

- Instead of Boolean, store a number in each cell
- Need some value to represent 'not an edge'
 - 0, -1, or some other value based on how you are using the graph
 - Might need to be a separate field if no restrictions on weights

Adjacency List

Assign each node a number from 0 to $|V|-1$

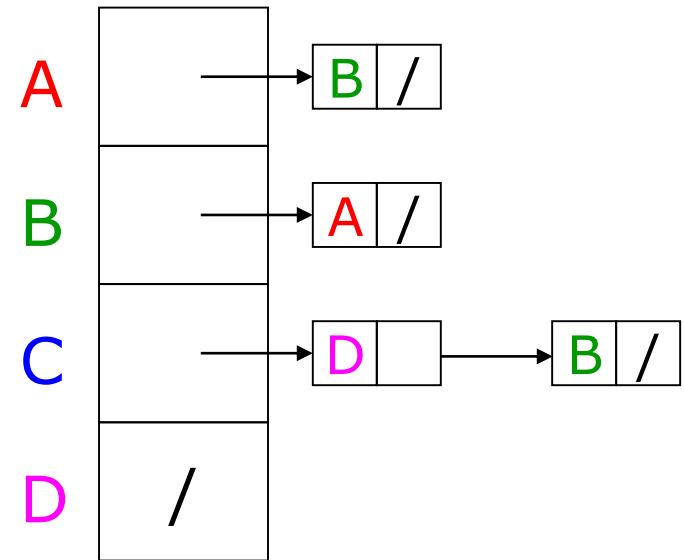
- An array of length $|V|$ in which each entry stores a list of all adjacent vertices (e.g., linked list)



Adjacency List Properties

Running time to:

- Get a vertex's out-edges:
- Get a vertex's in-edges:
- Decide if some edge exists:
- Insert an edge:
- Delete an edge:



Space requirements:

Best for sparse or dense graphs?

Adjacency List Properties

Running time to:

- Get a vertex's out-edges:
 $O(d)$ where d is out-degree of vertex
- Get a vertex's in-edges:
 $O(|E|)$ (could keep a second adjacency list for this!)
- Decide if some edge exists:
 $O(d)$ where d is out-degree of source
- Insert an edge:
 $O(1)$ (unless you need to check if it's already there)
- Delete an edge:
 $O(d)$ where d is out-degree of source

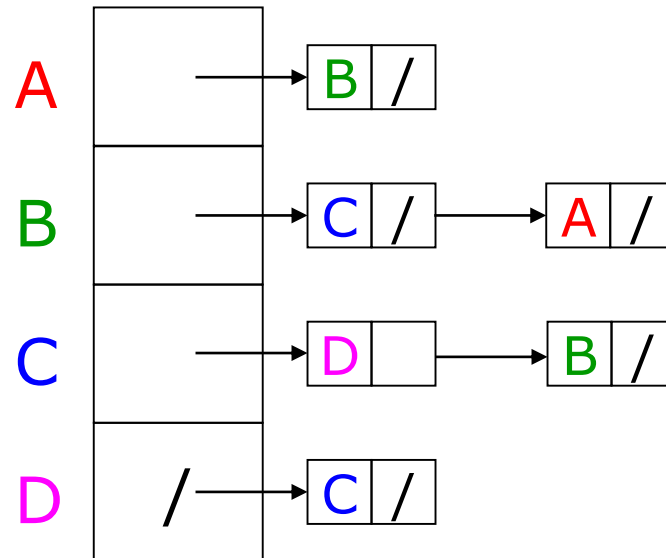
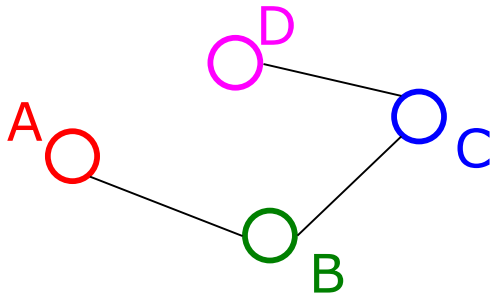
Space requirements: $O(|V| + |E|)$

Best for sparse or dense graphs? *sparse*

Undirected Graphs

Adjacency lists also work well for undirected graphs with one caveat

- Put each edge in two lists to support efficient "get all neighbors"



Which is better?

Graphs are often sparse

- Streets form grids
- Airlines rarely fly to all cities

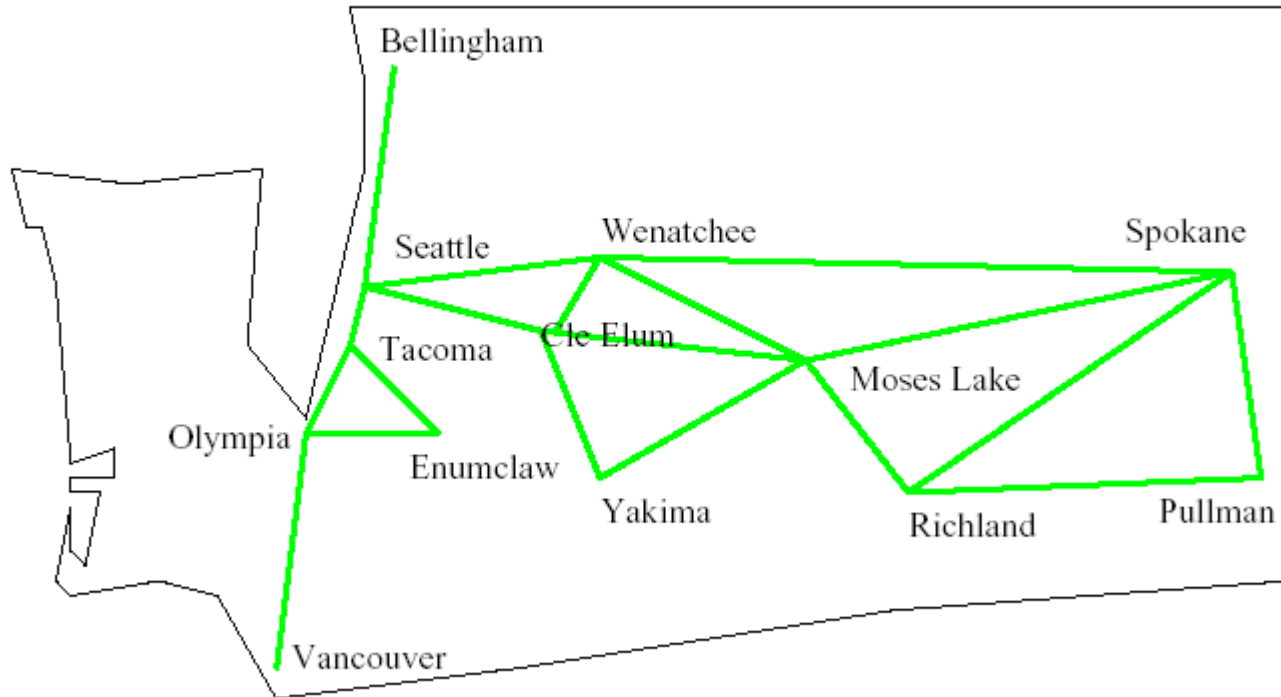
Adjacency lists should generally be your default choice

- Slower performance compensated by greater space savings

Might be easier to list what isn't a graph application...

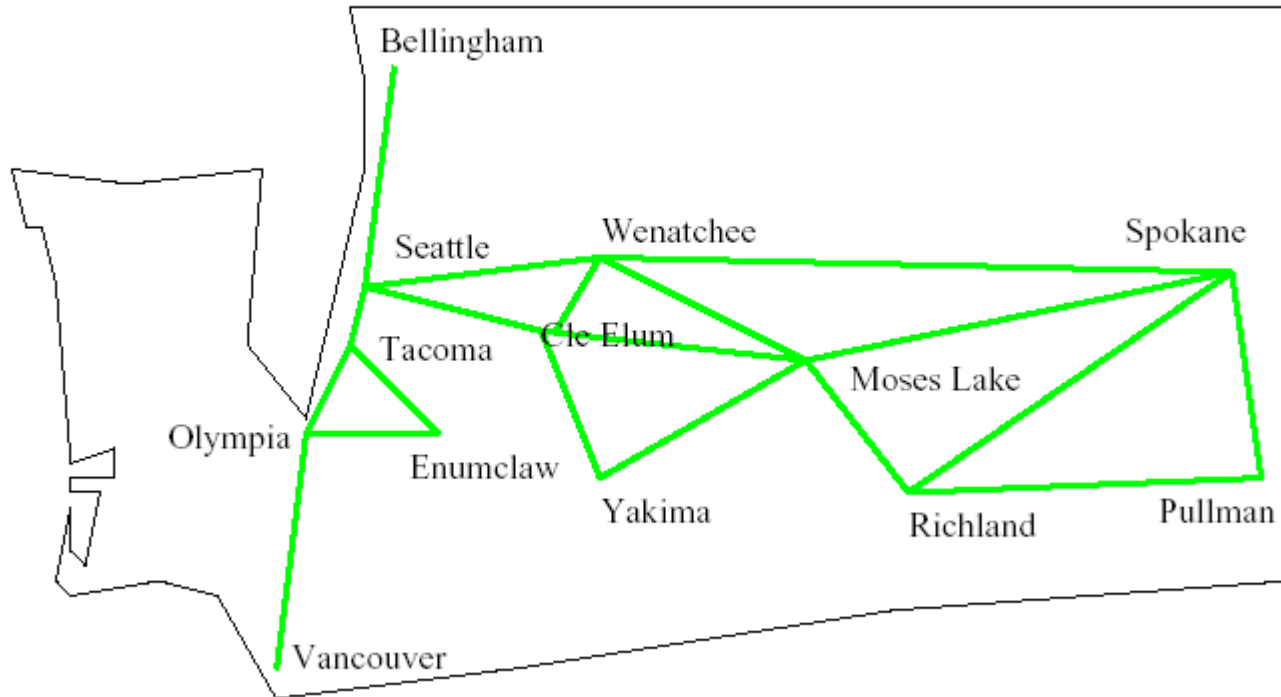
APPLICATIONS OF GRAPHS: TRAVERSALS

Application: Moving Around WA State



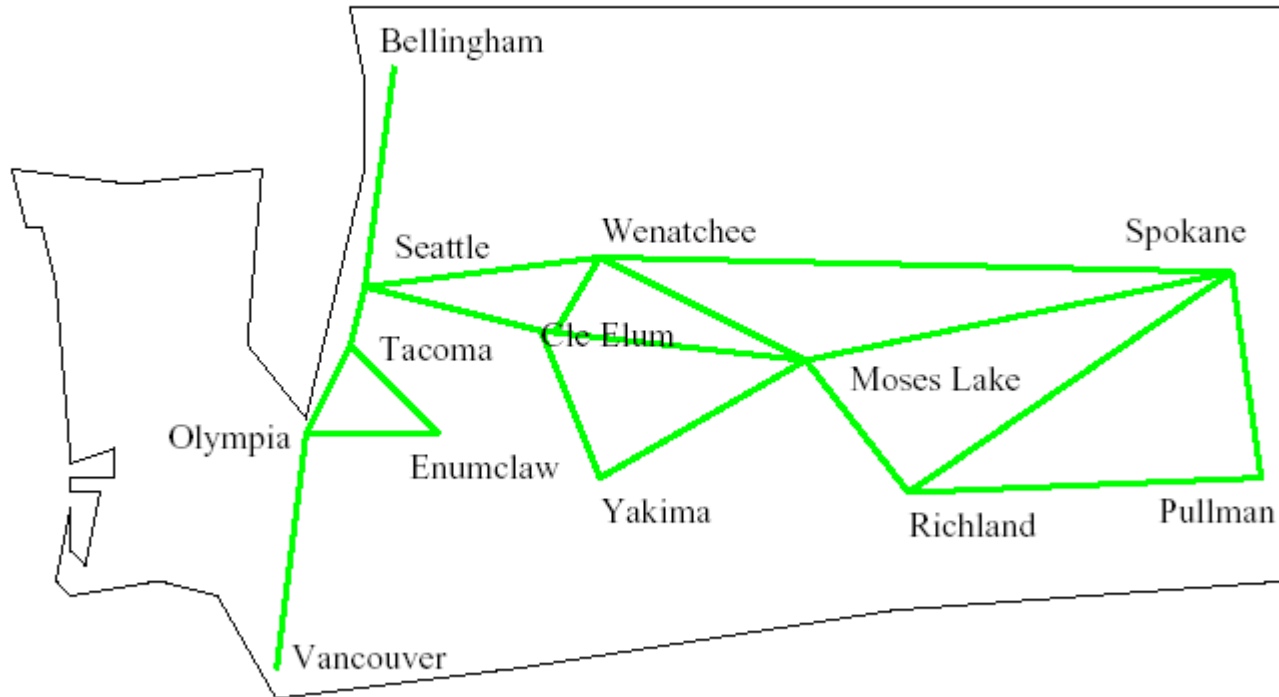
What's the *shortest way* to get from
Seattle to Pullman?

Application: Moving Around WA State



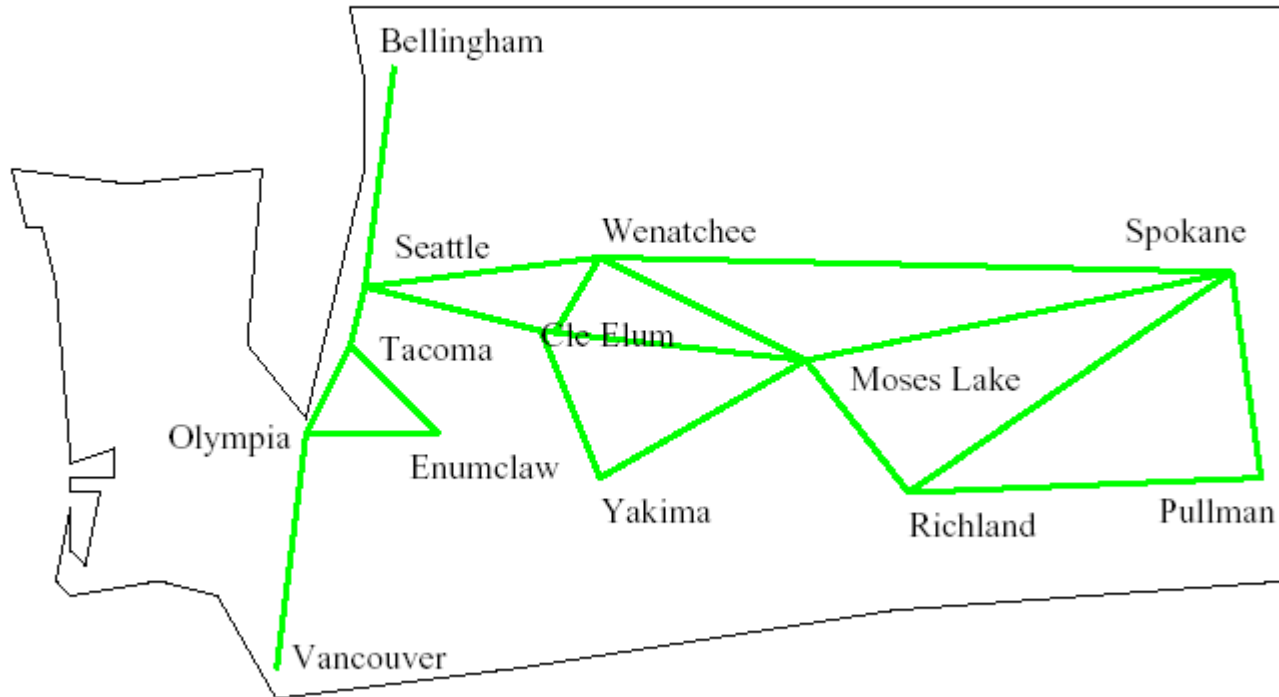
What's the *fastest way* to get from Seattle to Pullman?

Application: Reliability of Communication



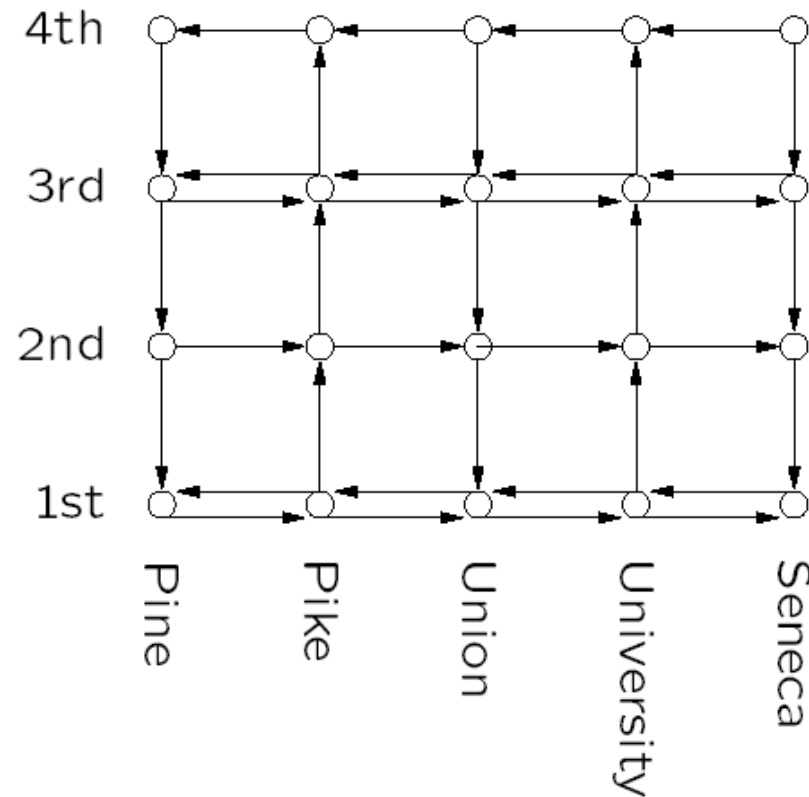
If Wenatchee's phone exchange *goes down*,
can Seattle still talk to Pullman?

Application: Reliability of Communication



If Tacoma's phone exchange *goes down*,
can Olympia still talk to Spokane?

Applications: Bus Routes Downtown



If we're at 3rd and Pine, how can we get to
1st and University using Metro?
How about 4th and Seneca?

Graph Traversals

For an arbitrary graph and a starting node v , find all nodes reachable from v (i.e., there exists a path)

- Possibly "do something" for each node (print to output, set some field, return from iterator, etc.)

Related Problems:

- Is an undirected graph connected?
- Is a digraph weakly/strongly connected?
 - For strongly, need a cycle back to starting node

Graph Traversals

Basic Algorithm for Traversals:

- Select a starting node
- Make a set of nodes adjacent to current node
- Visit each node in the set but "mark" each nodes after visiting them so you don't revisit them (and eventually stop)
- Repeat above but skip "marked nodes"

In Rough Code Form

```
traverseGraph(Node start) {  
    Set pending = emptySet();  
    pending.add(start)  
    mark start as visited  
    while(pending is not empty) {  
        next = pending.remove()  
        for each node u adjacent to next  
            if(u is not marked) {  
                mark u  
                pending.add(u)  
            }  
        }  
    }  
}
```

Running Time and Options

Assuming add and remove are $O(1)$, entire traversal is $O(|E|)$ if using an adjacency list

The order we traverse depends entirely on how add and remove work/are implemented

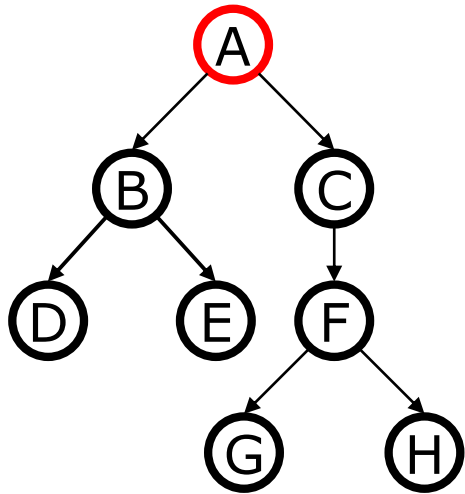
- DFS: a stack "depth-first graph search"
- BFS: a queue "breadth-first graph search"

DFS and BFS are "big ideas" in computer science

- Depth: recursively explore one part before going back to the other parts not yet explored
- Breadth: Explore areas closer to start node first

Recursive DFS, Example with Tree

A tree is a graph and DFS and BFS are particularly easy to "see" in one

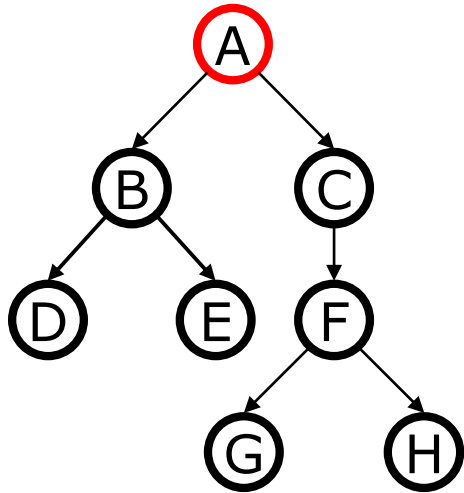


```
DFS(Node start) {  
    mark and process start  
    for each node u adjacent to start  
        if u is not marked  
            DFS(u)  
}
```

Order processed: A, B, D, E, C, F, G, H

- This is a "pre-order traversal" for trees
- The marking is unneeded here but because we support arbitrary graphs, we need a means to process each node exactly once

DFS with Stack, Example with Tree

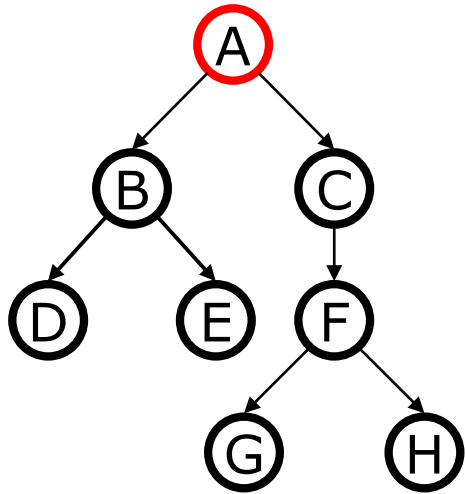


```
DFS2(Node start) {  
    initialize stack s to hold start  
    mark start as visited  
    while(s is not empty) {  
        next = s.pop() // and "process"  
        for each node u adjacent to next  
            if(u is not marked)  
                mark u and push onto s  
    }  
}
```

Order processed: A, C, F, H, G, B, E, D

- A different order but still a perfectly fine traversal of the graph

BFS with Queue, Example with Tree



```
BFS(Node start) {  
  initialize queue q to hold start  
  mark start as visited  
  while(q is not empty) {  
    next = q.dequeue() // and "process"  
    for each node u adjacent to next  
      if(u is not marked)  
        mark u and enqueue onto q  
  }  
}
```

Order processed: A, B, C, D, E, F, G, H

- A "level-order" traversal

DFS/BFS Comparison

BFS always finds the shortest path (or "optimal solution") from the starting node to a target node

- Storage for BFS can be extremely large
- A k -nary tree of height h could result in a queue size of k^h

DFS can use less space in finding a path

- If longest path in the graph is p and highest out-degree is d then DFS stack never has more than $d \cdot p$ elements

Implications

For large graphs, DFS is hugely more memory efficient, *if we can limit the maximum path length to some fixed d .*

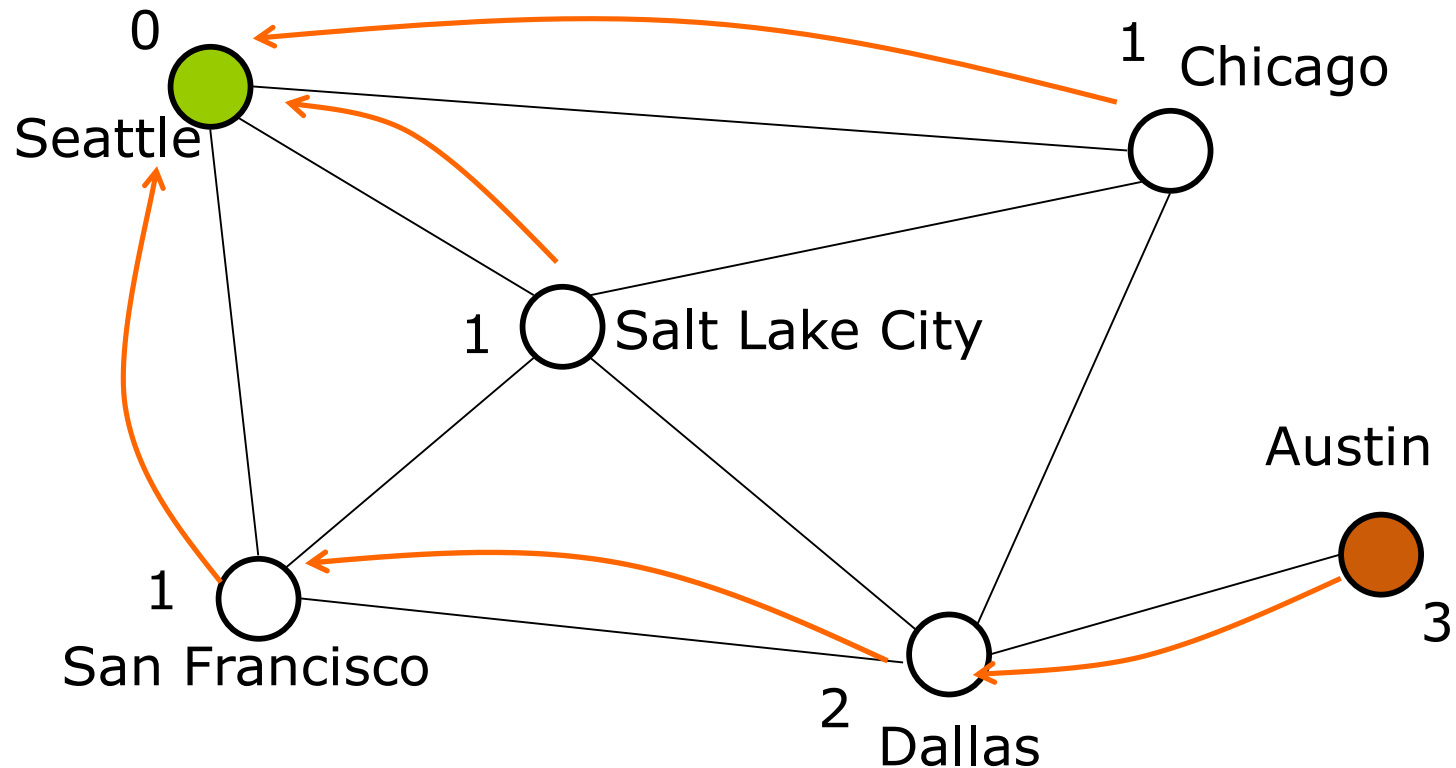
If we *knew* the distance from the start to the goal in advance, we could simply *not add any children to stack after level d*

But what if we don't know d in advance?

Example using BFS

What is a path from Seattle to Austin?

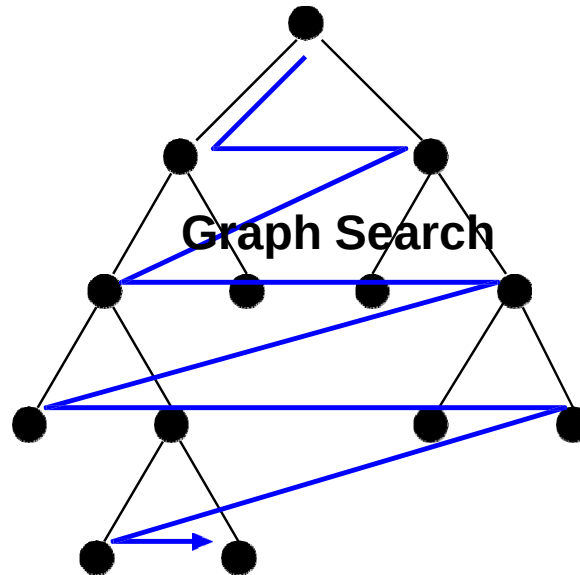
- Remember marked nodes are not re-enqueued
- Note shortest paths may not be unique



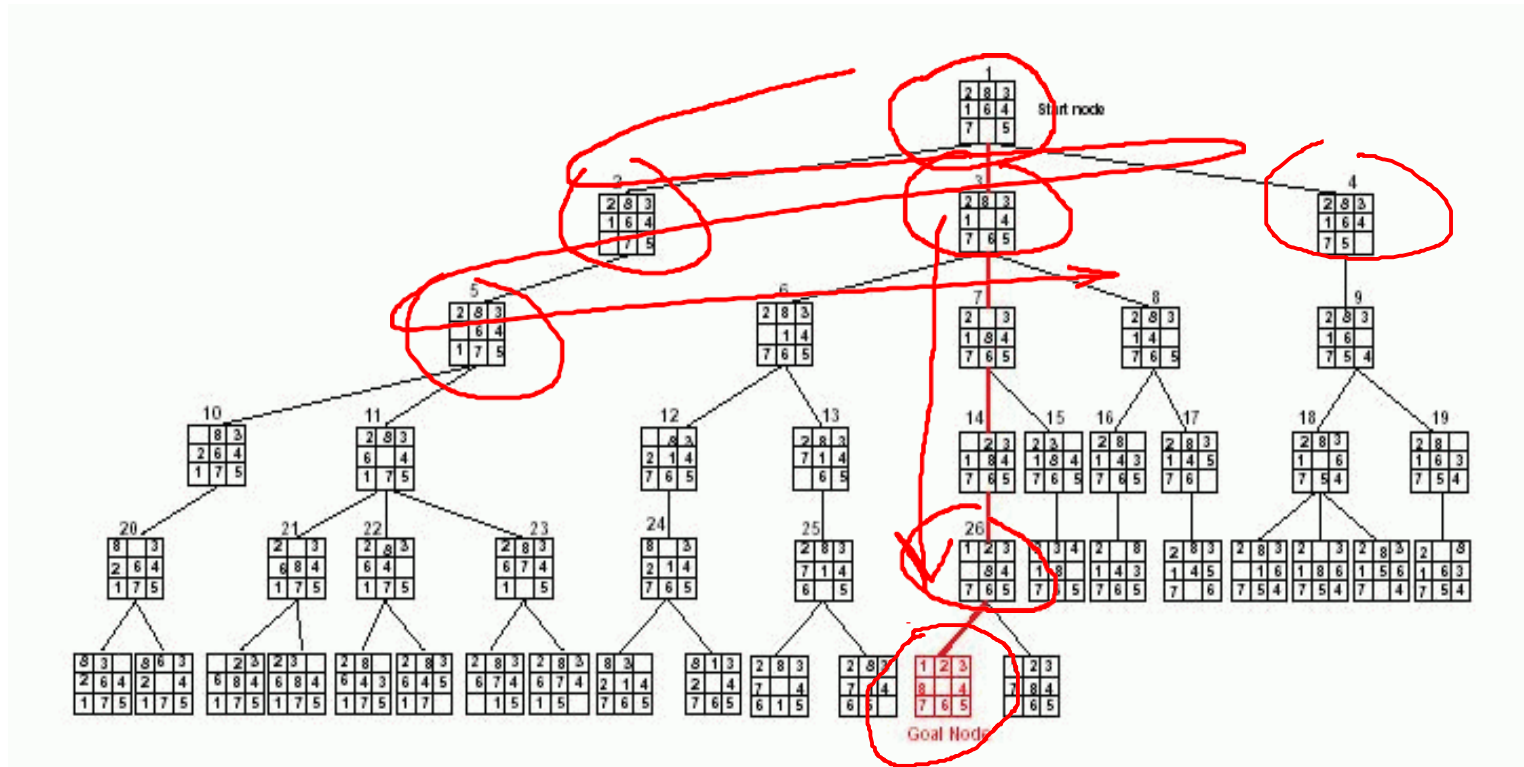
Graph Search

- Uninformed Search
 - Breadth-First Search
 - Depth-First Search
 - Iterative Deepening
- Informed Search: Best-First (Heuristic) Search
- 2-player Search
 - Min-Max algorithm
 - Alpha-Beta algorithm
- And/Or graphs

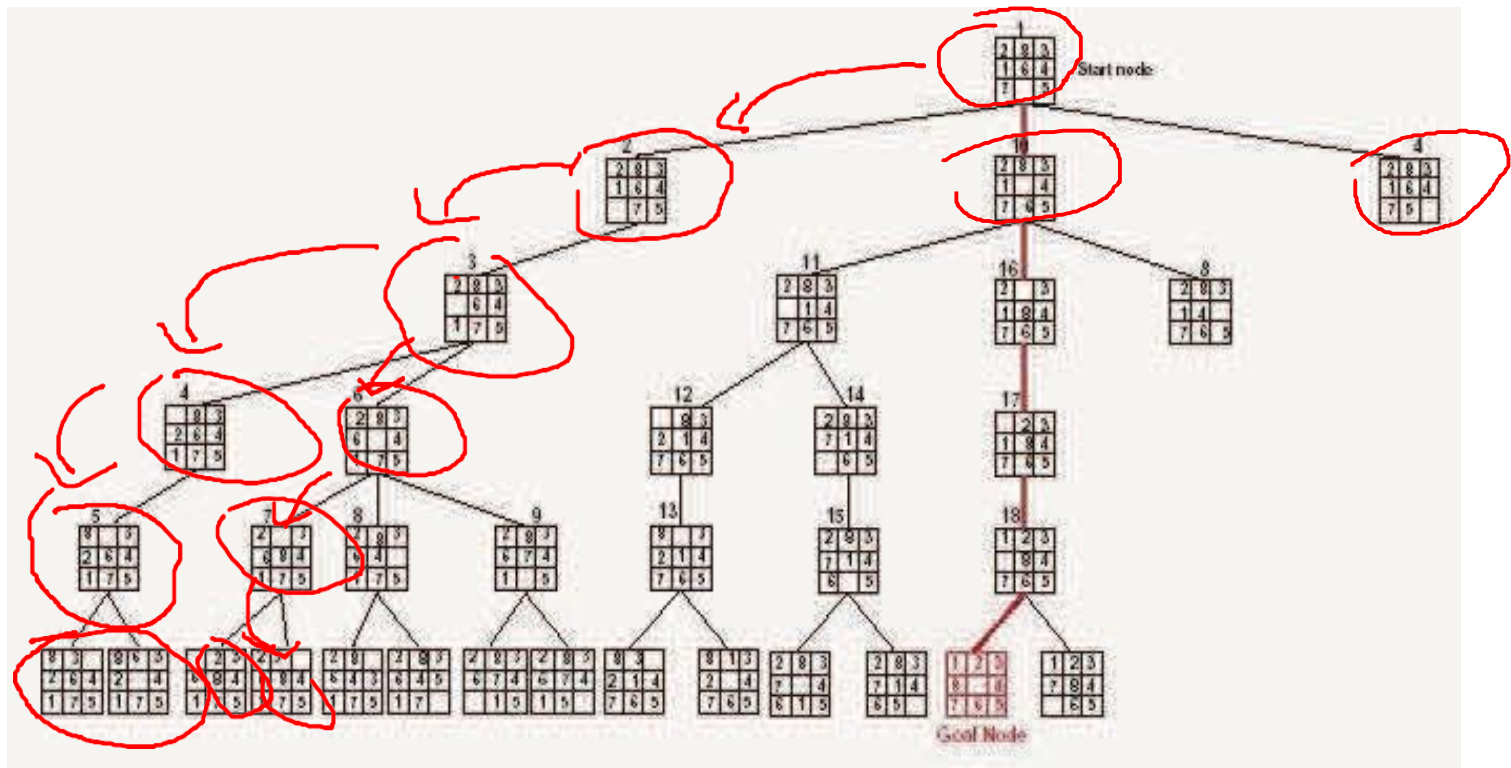
Breadth-First Search



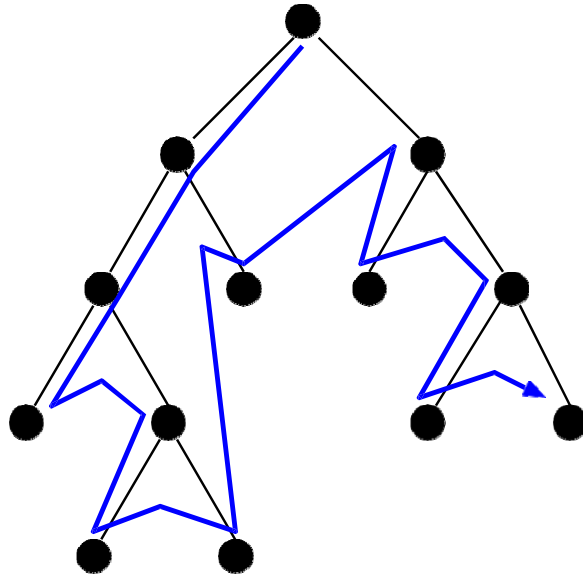
Breadth-First Search



Depth-First Search



Depth-First Search

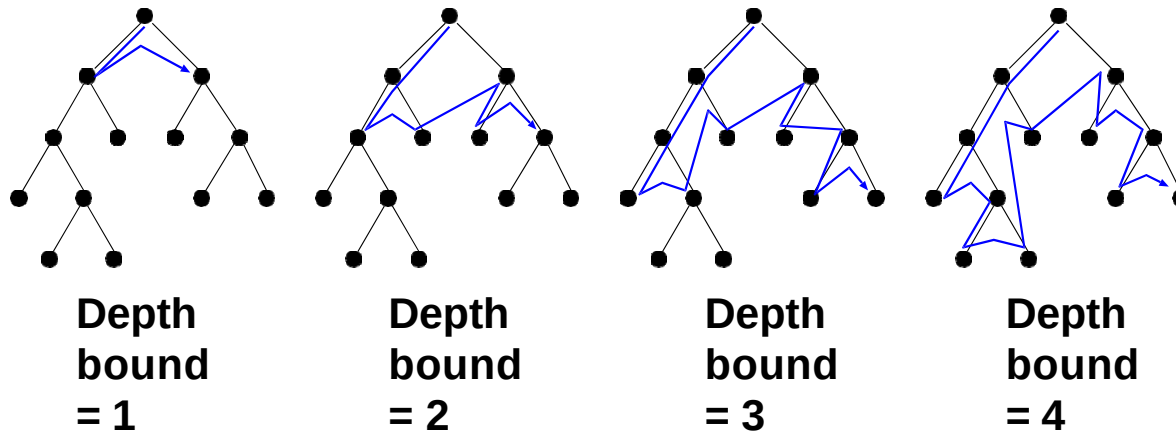


Iterative Deepening

- Successive depth-first searches are conducted, each with depth bounds increasing by 1, until a goal node is found.
- Enjoys the linear memory requirements of depth-first search while guaranteeing that a goal node of minimal depth will be found.
- The number of nodes expanded by iterative-deepening search is not many more than would be expanded by breadth-first search.

Iterative Deepening

Stages in Iterative-Deepening Search



Heuristic Search

Heuristic evaluation functions

$$f(n) = g(n) + h(n)$$

Example of the 8-puzzle :

1. Depth in the search space + number of tiles out of place.
2. Depth in the search space + minimum number of moves to reach the goal state.

$$f(s) = 0 + 1 + 2 + 1 + 1 = 5$$

2	8	3
1	6	4
7	5	

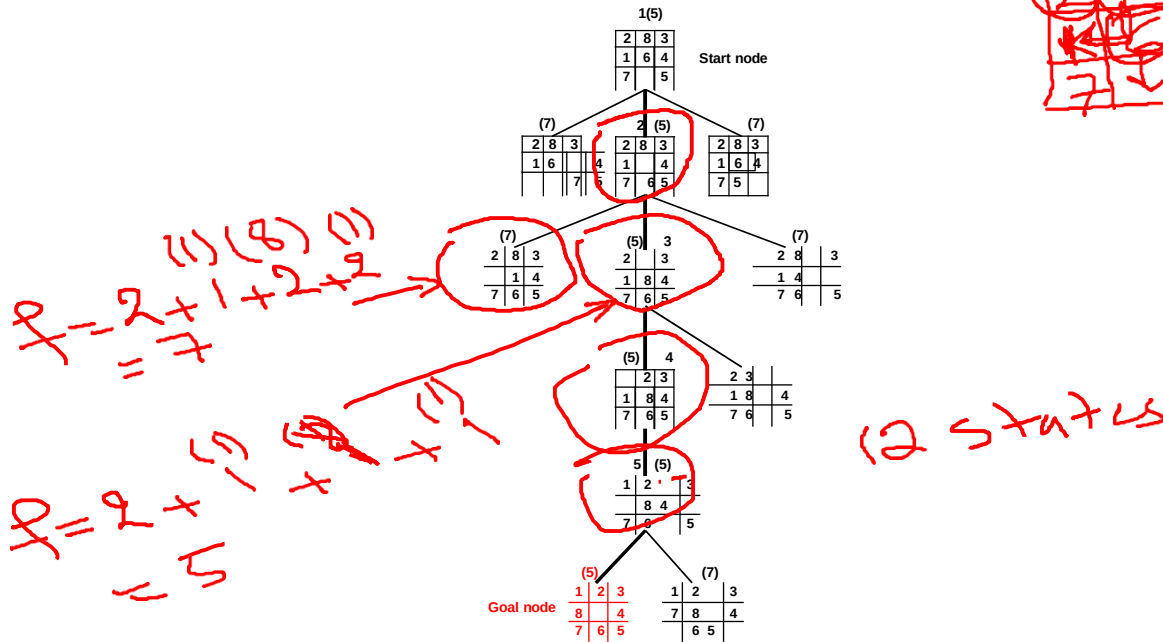


Figure 9: Depth in the search space + minimum number of moves to reach the goal state.

$$h_1(n) \leq h_2(n) \leq h^*(n)$$

A Algorithm*

Algorithm A: Best-first search using $f(n)$.

Algorithm A^* : Best-first search using $f(n)$ and where :

- $g^*(n) \leq g(n)$
- $h^*(n) \geq h(n)$

MinMax Algorithm

Two-person games :

- Difficulty to develop search algorithms (unpredictable opponent).
- Searching the space of possible moves and counter moves by the opponent.
- Guiding the play along a path to a winning state.

MinMax algorithm :

Max : the player trying to win (maximize his advantage).

Min : the opponent trying to minimize Max's score.

MinMax Algorithm

MinMax algorithm :

1. Descend to the tip of leaf nodes of the tree.
2. Propagate values back up the tree.
 - (a) If the parent state is Max, give it the maximum value among its children.
 - (b) If the parent is a Min node, give it the minimum value of its children.

MinMax Algorithm

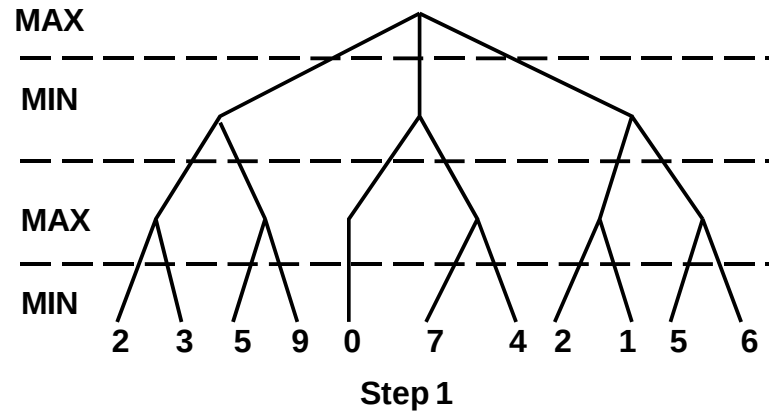


Figure 11: MinMax to a hypothetical state space.

MinMax Algorithm

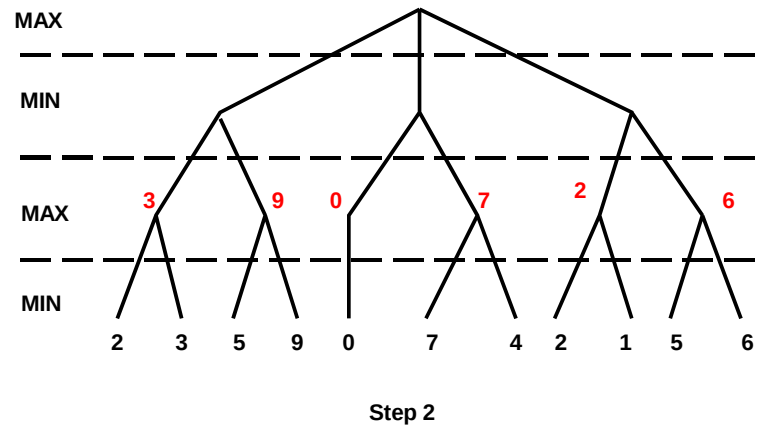


Figure 12: MinMax to a hypothetical state space.

MinMax Algorithm

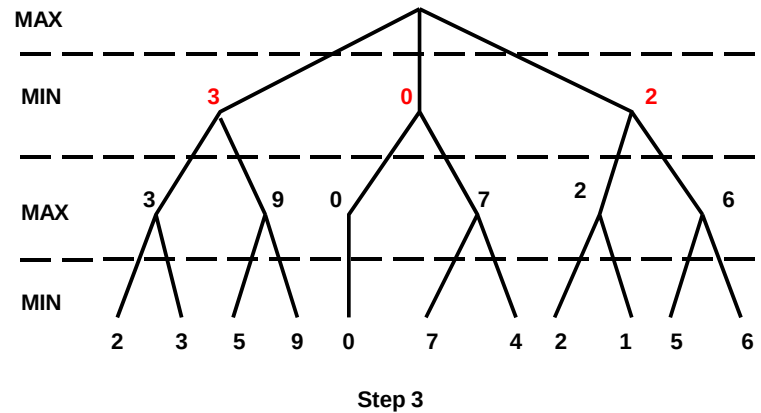


Figure 13: MinMax to a hypothetical state space.

MinMax Algorithm

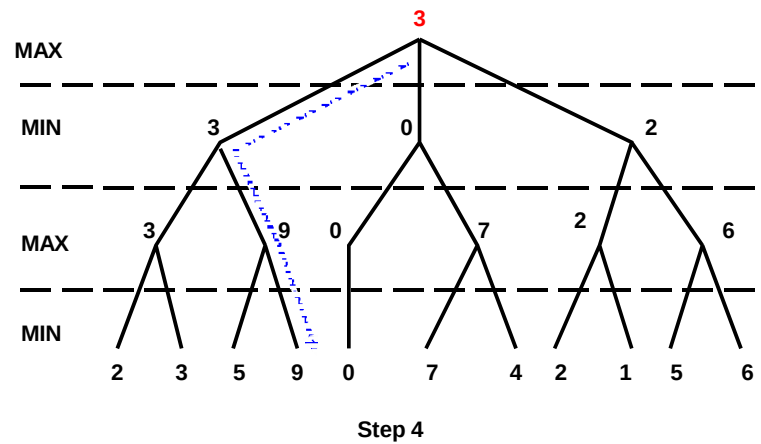


Figure 14: MinMax to a hypothetical state space.

Alpha-Beta Algorithm

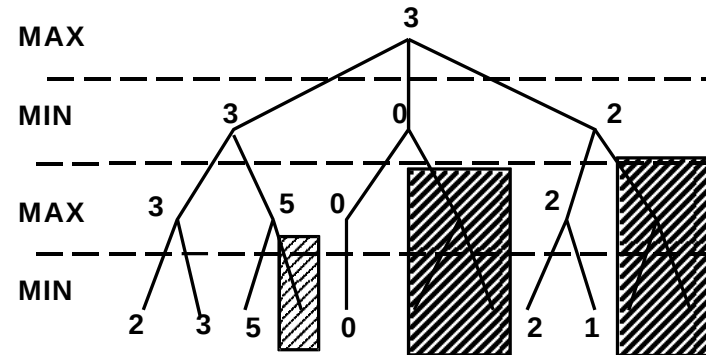


Figure 15: Alpha-Beta pruning applied to a hypothetical state space.

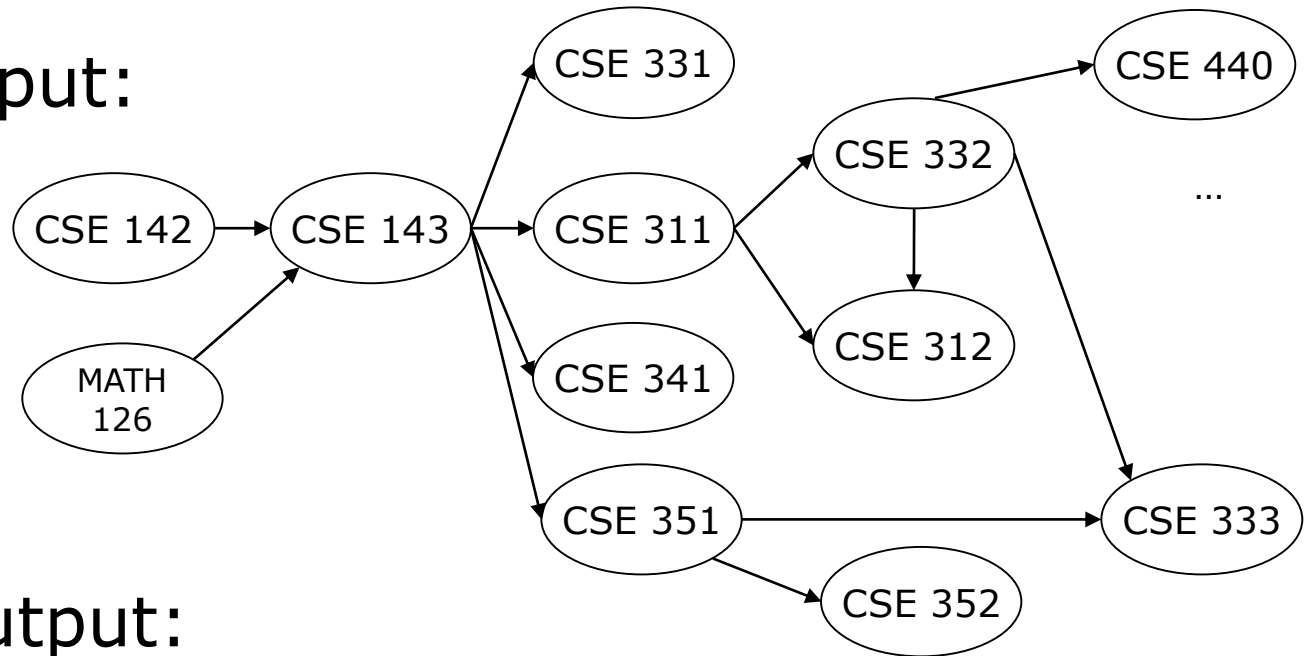
And-Or Graphs

- Technique for solving problems that can be decomposed into sub problems
- Links between nodes indicates relations between problems
- Or node: one of its successor node has to be solved
- And node: all of its successor node has to be solved
- Problem can be specified by two things: start node, goal nodes.
- Goal nodes: trivial (or primitive) problems
- Cost can be attached to arcs or nodes

Topological Sort

Problem: Given a DAG $G=(V, E)$, output all the vertices in order such that if no vertex appears before any other vertex that has an edge to it

Example input:



Example output:

- 142, 126, 143, 311, 331, 332, 312, 341, 351, 333, 440, 352

Questions and Comments

Terminology:

A DAG represents a **partial order** and a topological sort produces a **total order** that is consistent with it

Why do we perform topological sorts only on DAGs?

- Because a cycle means there is no correct answer

Is there always a unique answer?

- No, there can be one or more answers depending on the provided graph

What DAGs have exactly 1 answer?

- Lists

Uses Topological Sort

Figuring out how to finish your degree

Computing the order in which to recalculate cells in a spreadsheet

Determining the order to compile files with dependencies

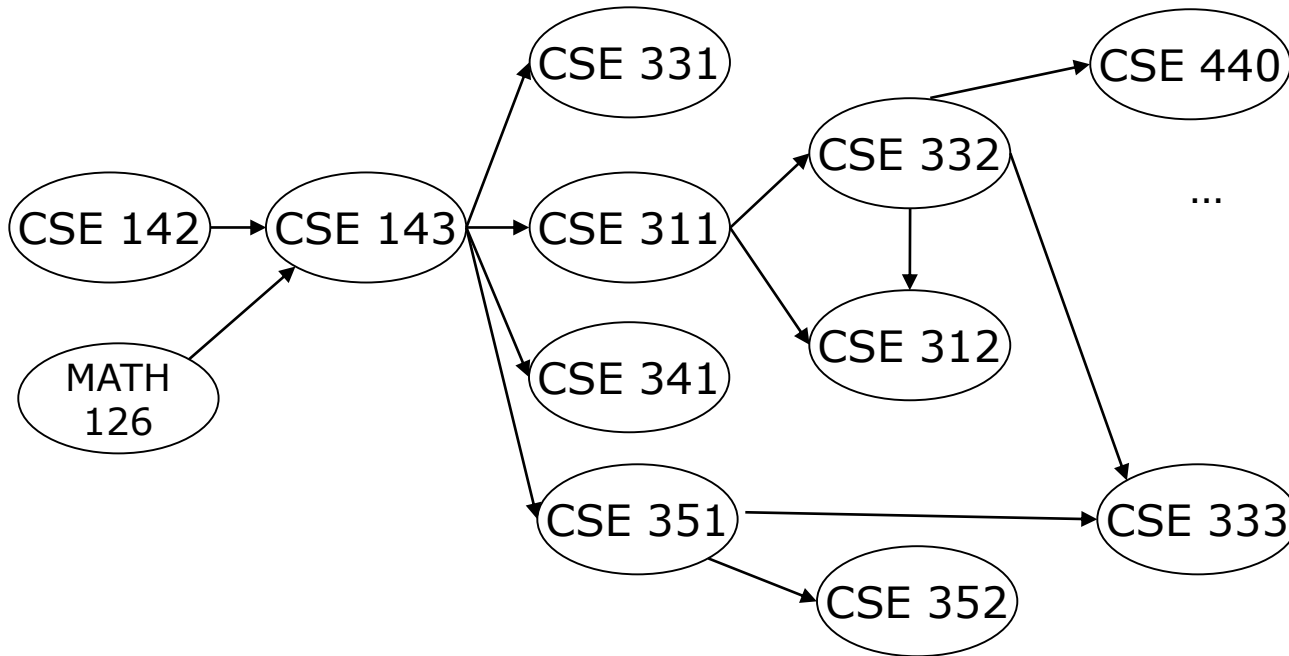
In general, use a dependency graph to find an allowed order of execution

Topological Sort: First Approach

1. Label each vertex with its in-degree
 - Think "write in a field in the vertex"
 - You could also do this with a data structure on the side
2. While there are vertices not yet outputted:
 - a) Choose a vertex **v** labeled with in-degree of 0
 - b) Output **v** and "remove it" from the graph
 - c) For each vertex **u** adjacent to **v**, **decrement in-degree** of **u**
 - (i.e., **u** such that $(\mathbf{v}, \mathbf{u})$ is in E)

Example

Output:



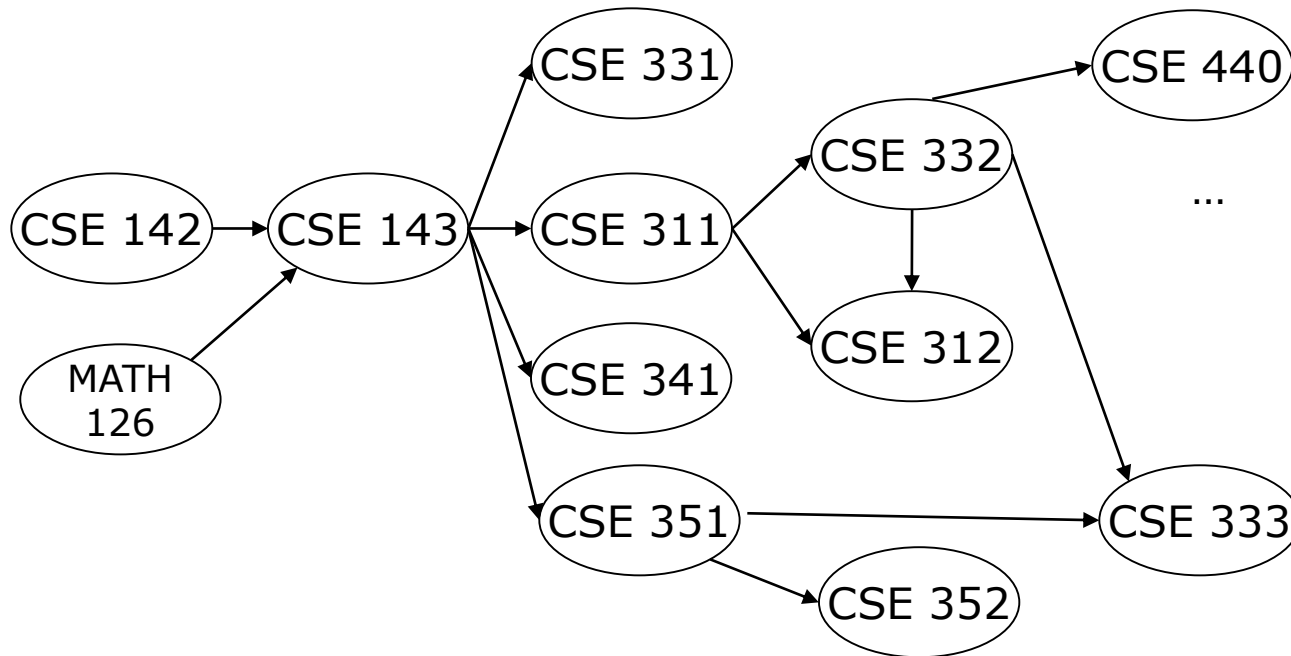
Node: 126 142 143 311 312 331 332 333 341 351 352 440

Removed?

In-deg:

Example

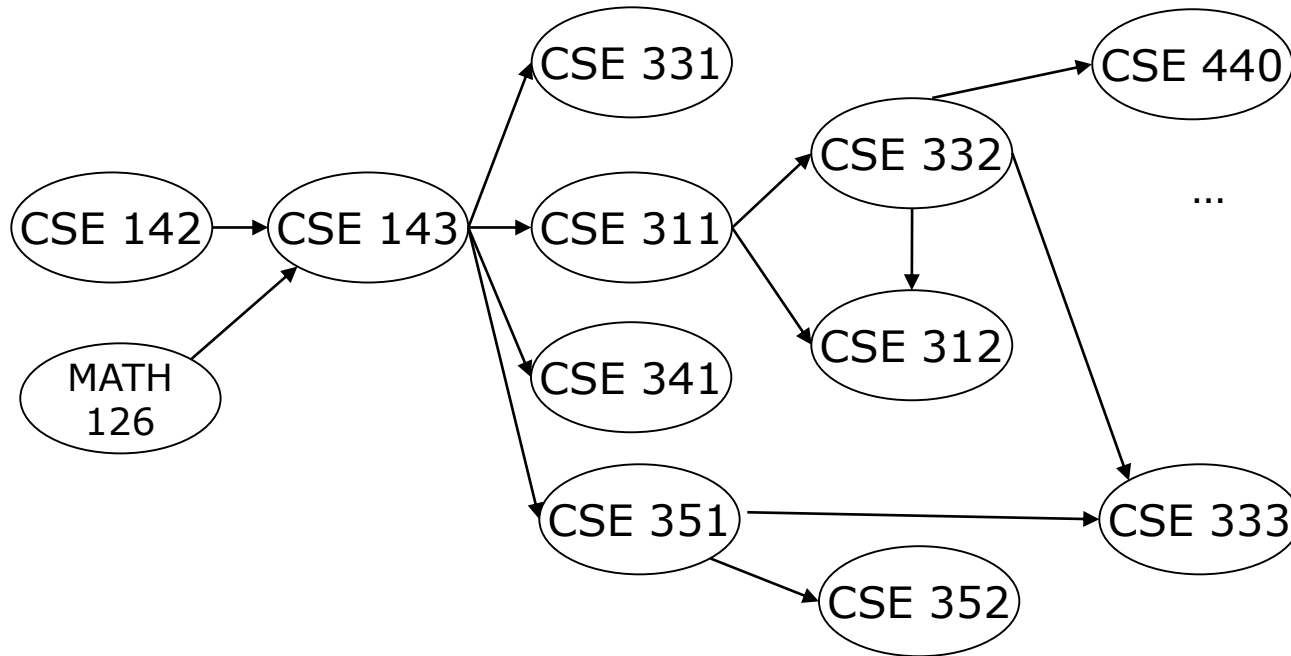
Output:



Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?												
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1

Example

Output:
126



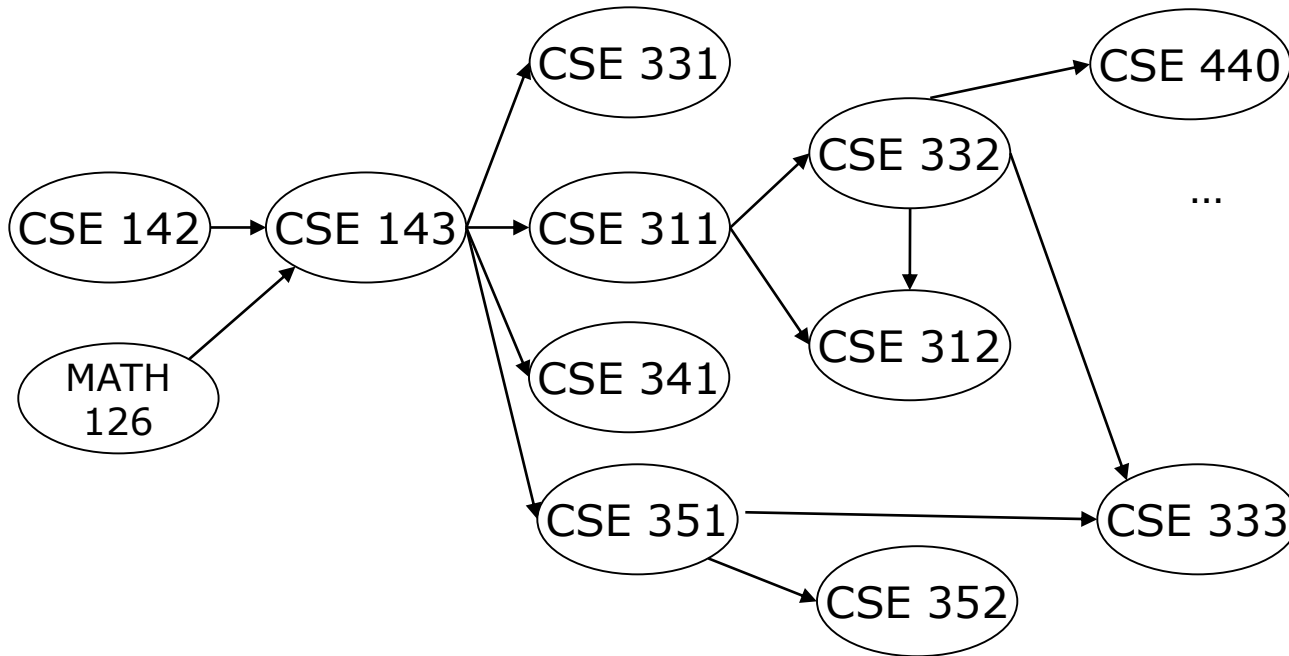
Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x											
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1									

Example

Output:

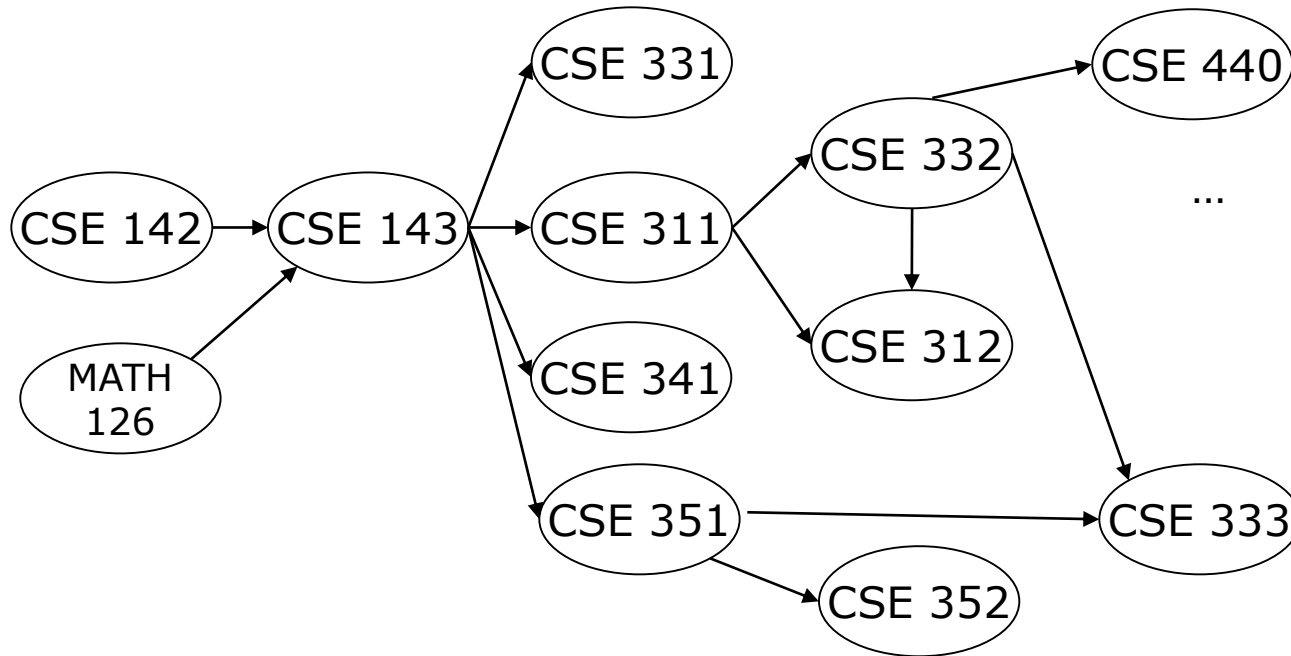
126

142



Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x										
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1									
			0									

Example



Output:

126

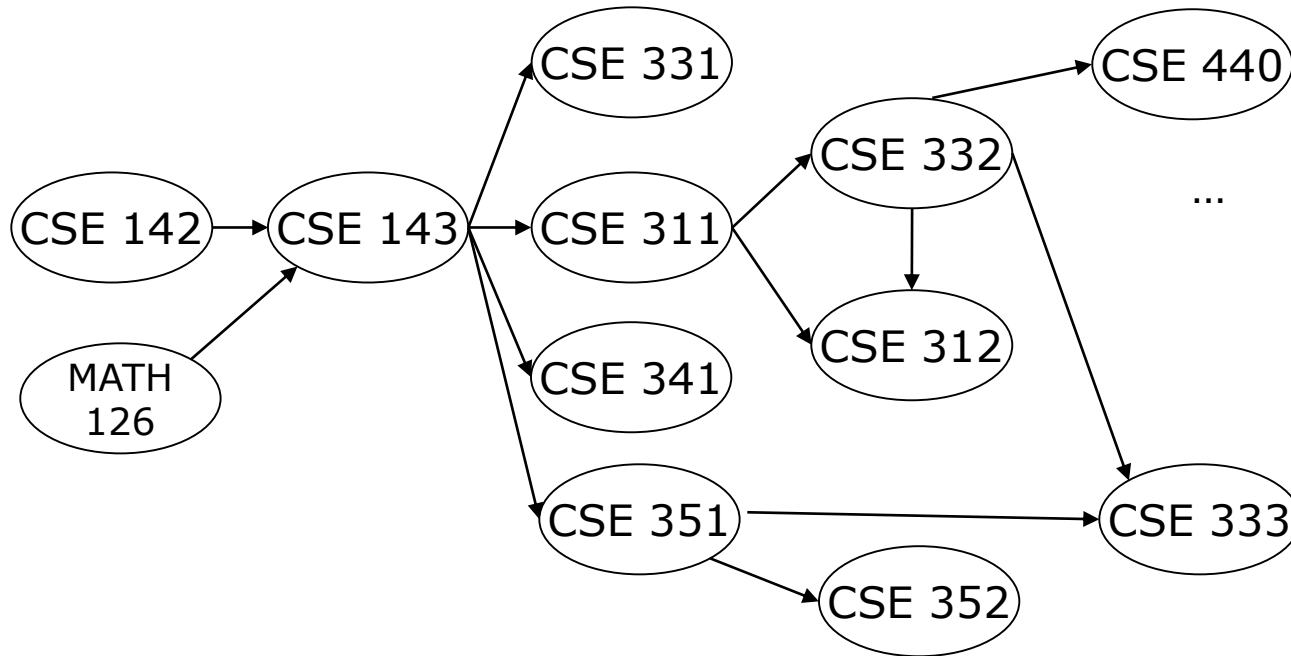
142

143

...

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x									
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0		0			0	0		
			0									

Example



Output:

126

142

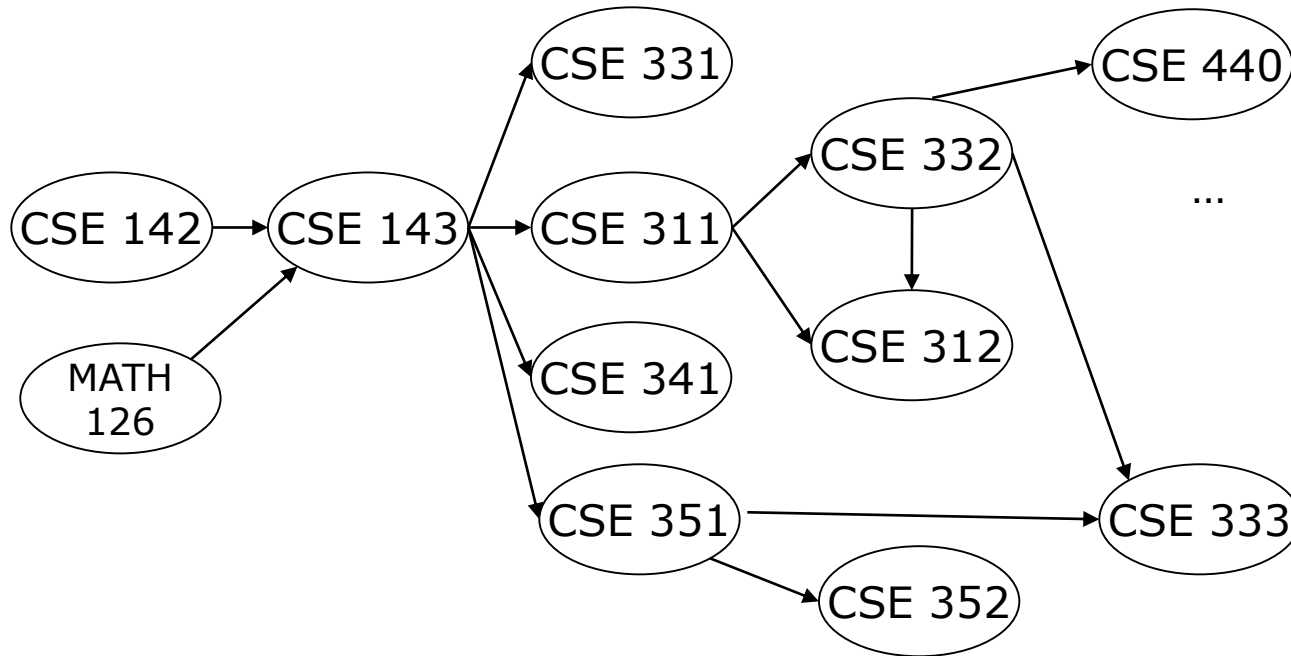
143

311

...

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x								
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0		0	0		
			0									

Example



Output:

126

142

143

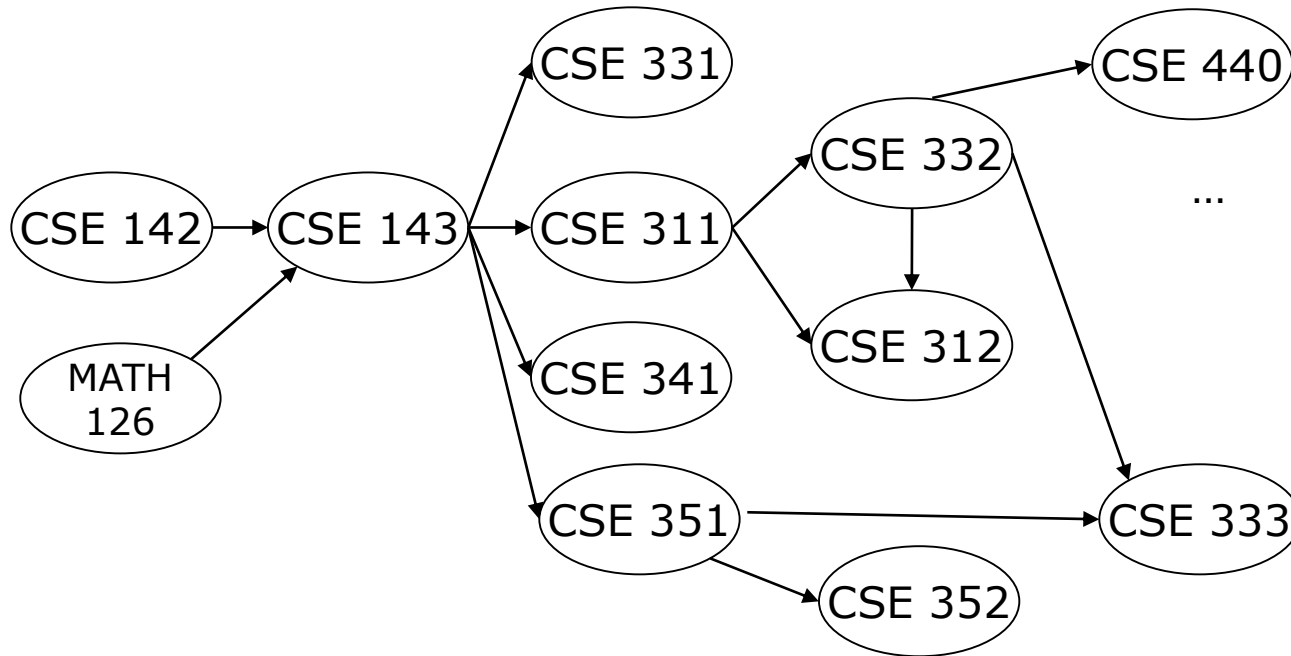
311

331

...

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x		x						
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0		0	0		
			0									

Example



Output:

126

142

143

311

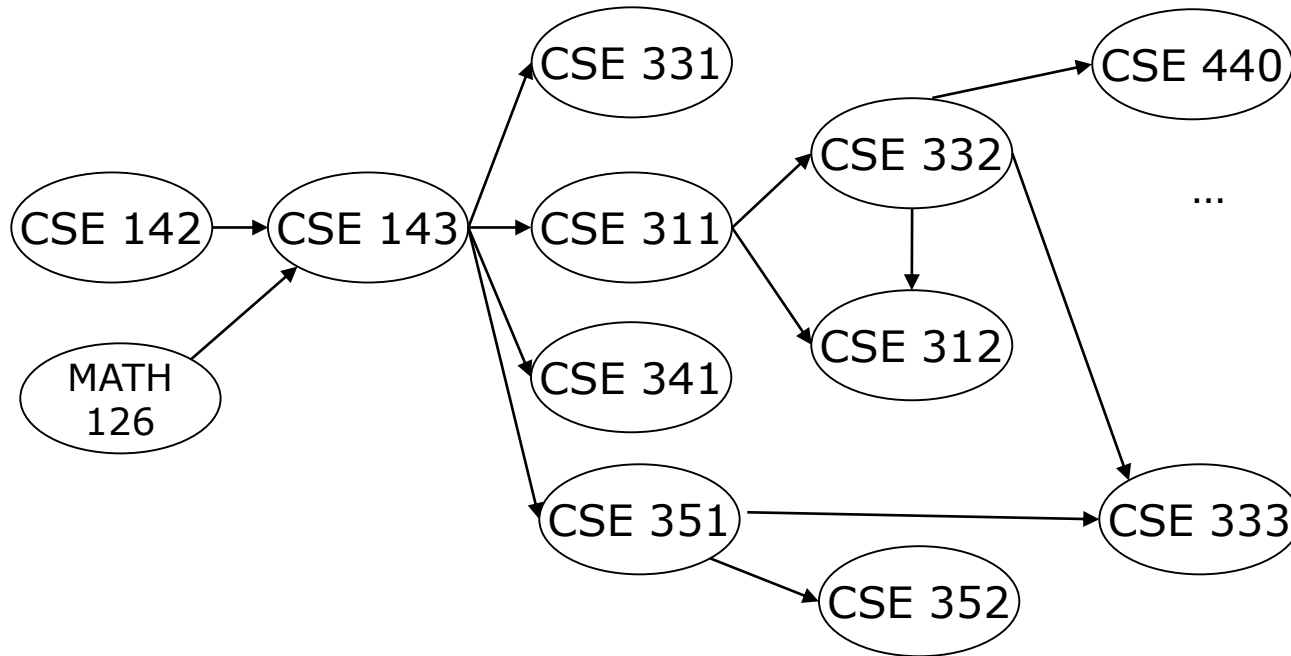
331

332

...

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x		x	x					
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0		0
			0		0							

Example



Output:

126

142

143

311

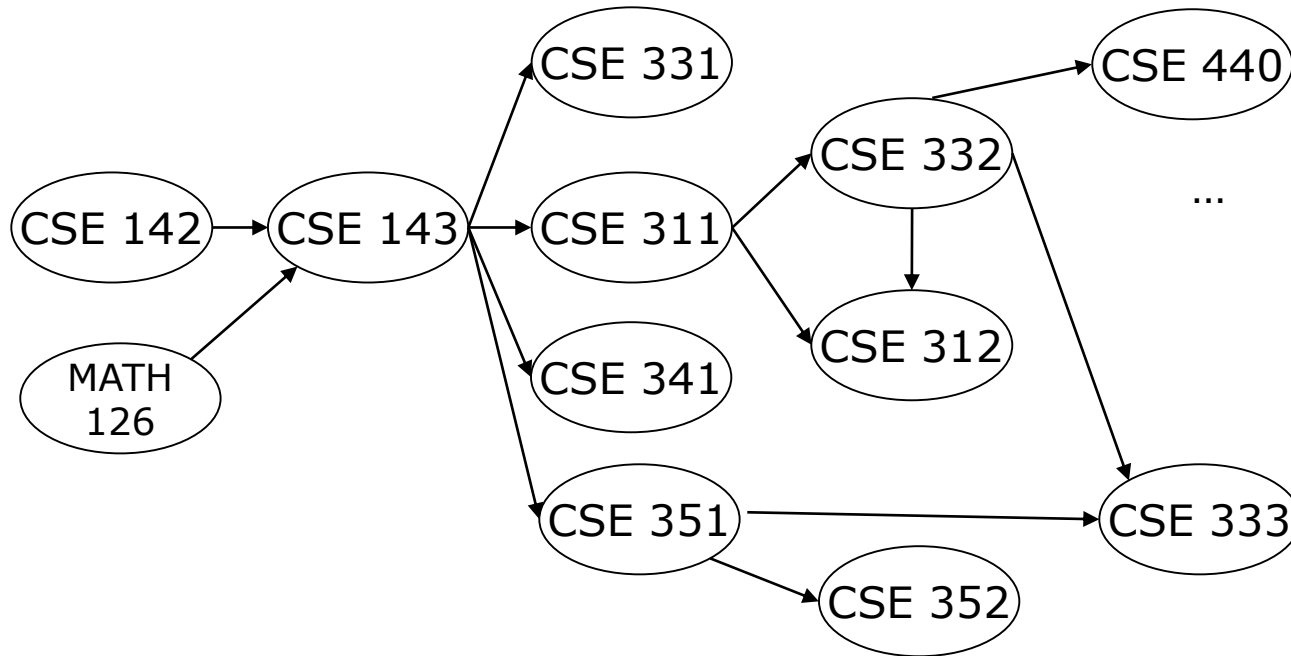
331

332

312

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x	x	x	x					
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0		0
			0		0							

Example

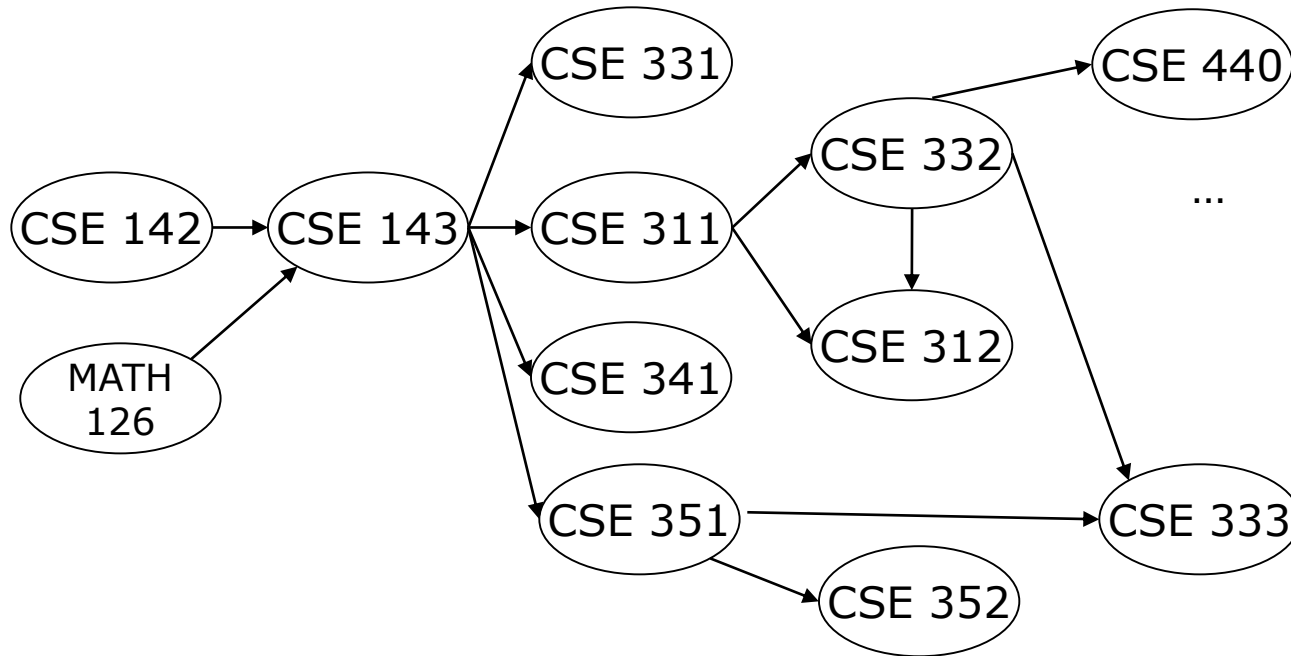


Output:

126
142
143
311
331
332
312
341

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x	x	x	x		x			
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0		0
			0		0							

Example



Output:

126

142

143

311

331

332

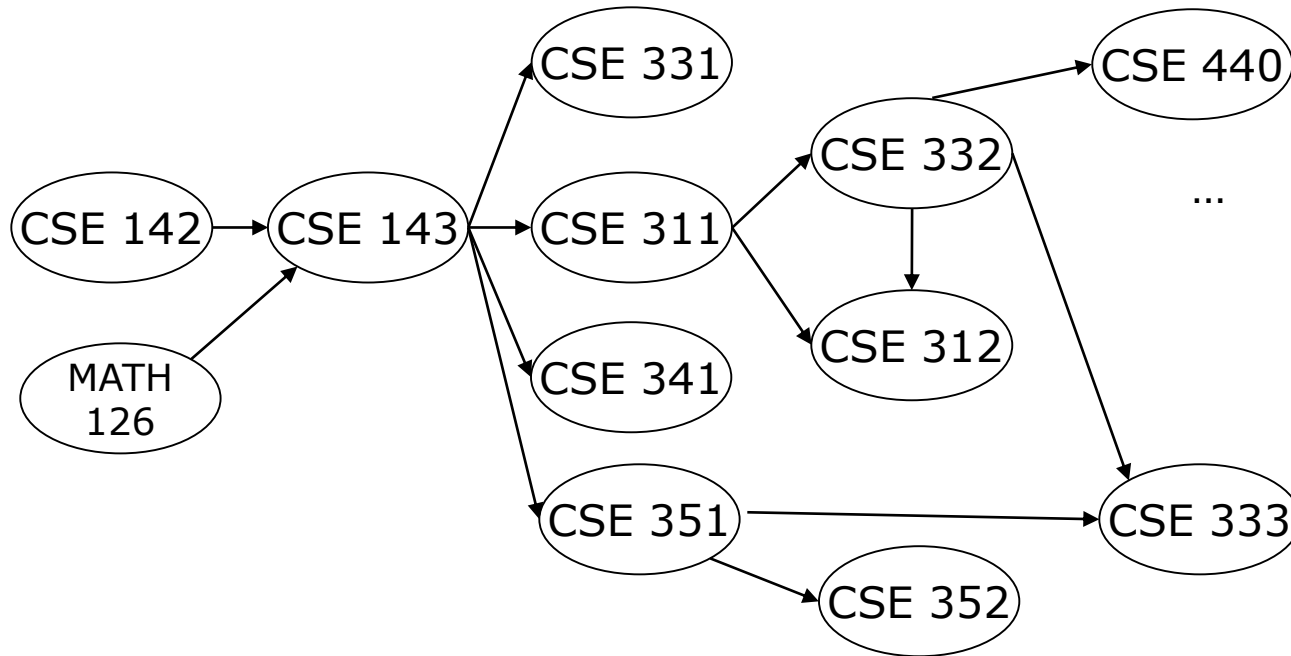
312

341

351

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x	x	x	x		x	x		
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0	0	0
			0		0			0				

Example



Output:

126

142

143

311

331

332

312

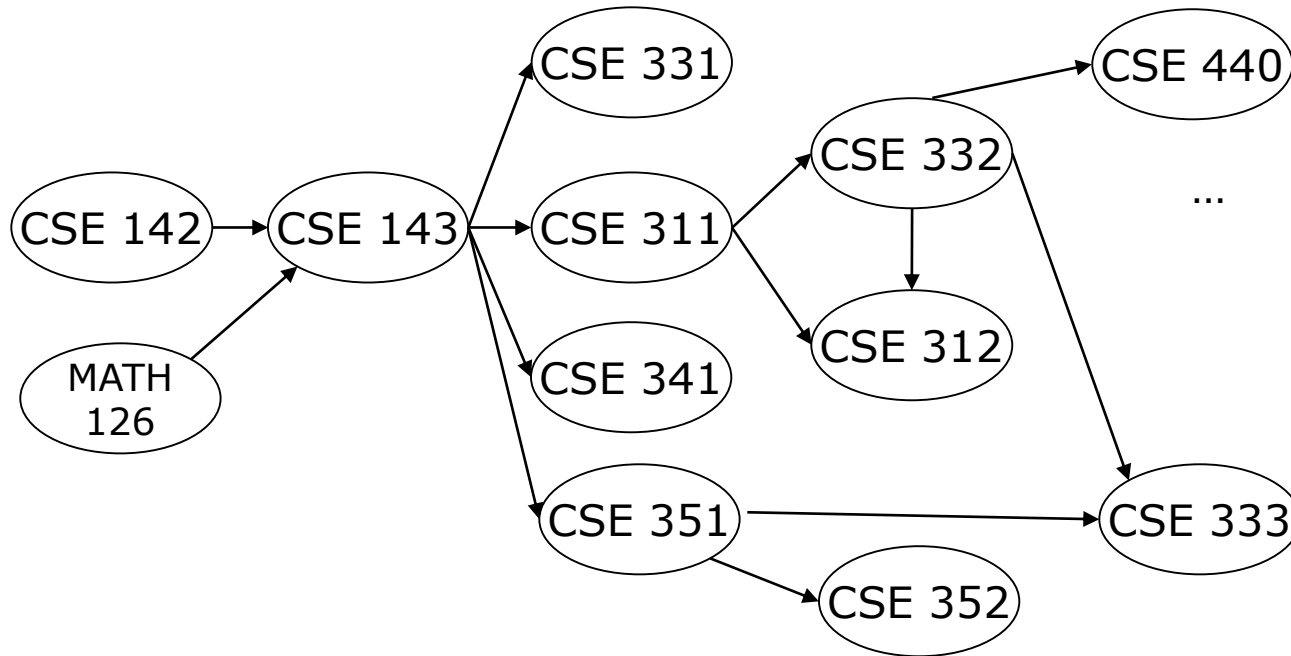
341

351

333

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x	x	x	x	x	x	x		
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0	0	0
			0		0			0				

Example

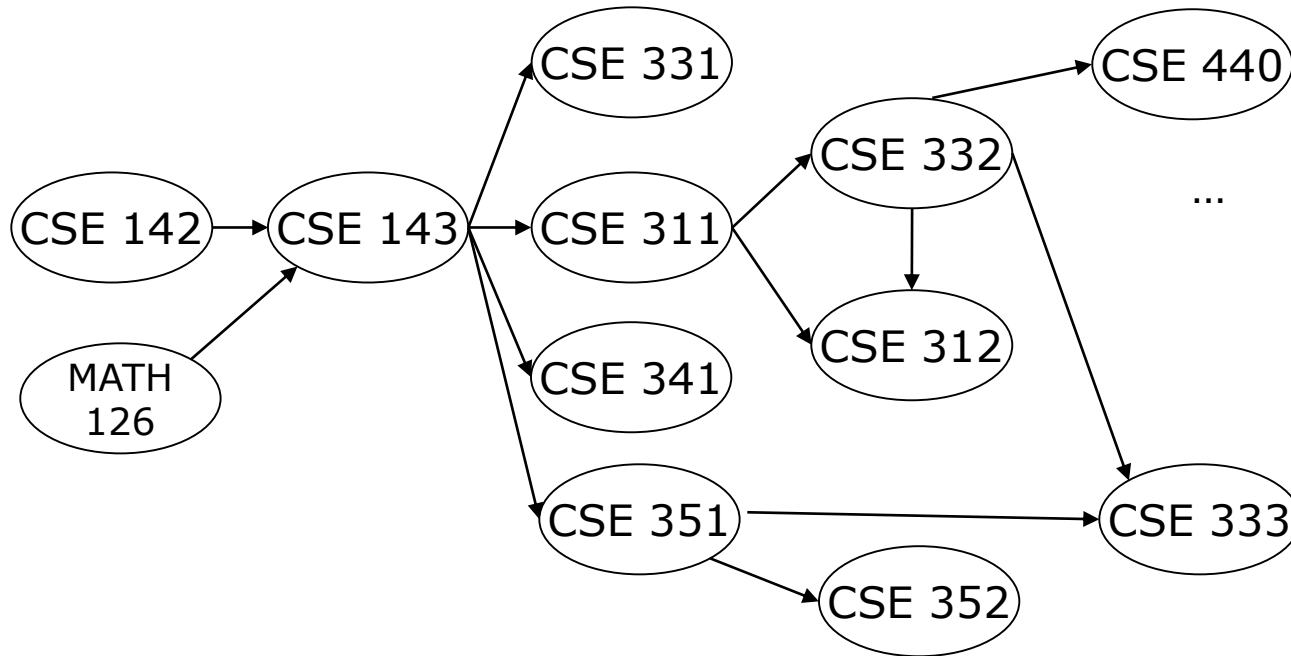


Output:

126 352
 142
 143
 311
 331
 332
 312
 341
 351
 333

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x	x	x	x	x	x	x	x	
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0	0	0
			0		0			0				

Example



Output:

126 352
 142 440
 143
 311
 331
 332
 312
 341
 351
 333

Node:	126	142	143	311	312	331	332	333	341	351	352	440
Removed?	x	x	x	x	x	x	x	x	x	x	x	x
In-deg:	0	0	2	1	2	1	1	2	1	1	1	1
			1	0	1	0	0	1	0	0	0	0
			0		0			0				

Running Time?

```
labelAllWithIndegreesAndEnqueueZeros();  
  
for(i=0; i < numVertices; i++) {  
    v = dequeue();  
    put v next in output  
    for each w adjacent to v {  
        w.indegree--;  
        if(w.indegree==0)  
            enqueue(w);  
    }  
}
```

- Initialization: $O(|V| + |E|)$ (assuming adjacency list)
- Sum of all enqueues and dequeues: $O(|V|)$
- Sum of all decrements: $O(|E|)$ (assuming adjacency list)
- So total is $O(|E| + |V|)$ – much better for sparse graph!

SHORTEST PATH

Single-Source Shortest Path Defined

- Given
 - $G = (V, E)$
 - Source vertex $s \in V$
- Compute
 - $\{SP\}$ = set of shortest paths from s to all vertices in V

Path Weight Defined

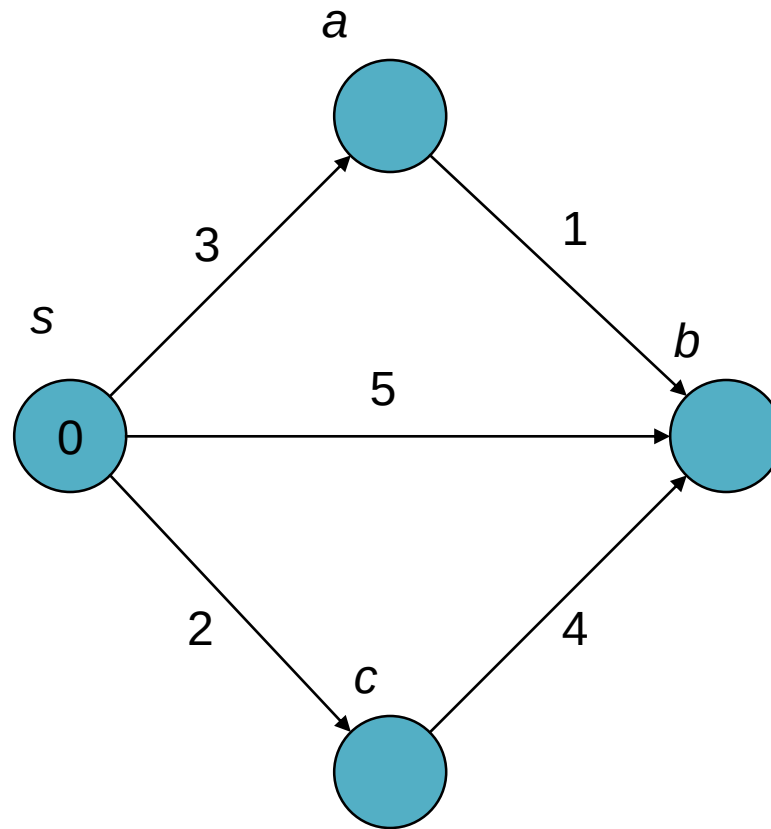
Weight of path $p = \{v_1, v_2, \dots, v_k\}$

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

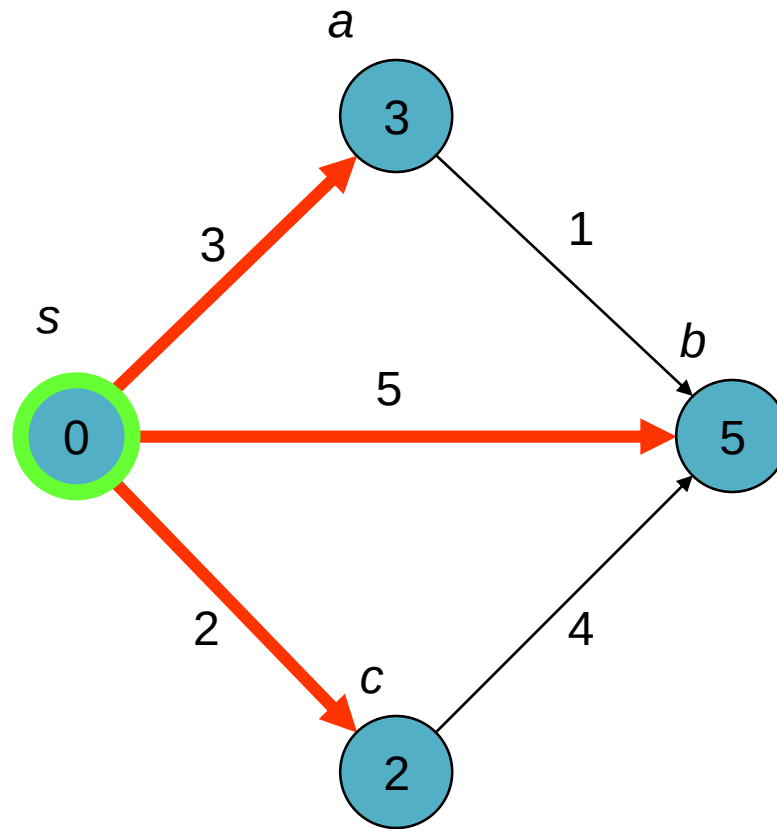
Shortest-path weight from u to v

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v \text{ if path}(u, v) \text{ exists} \\ \infty \text{ if path}(u, v) \text{ does not exist} \end{cases}$$

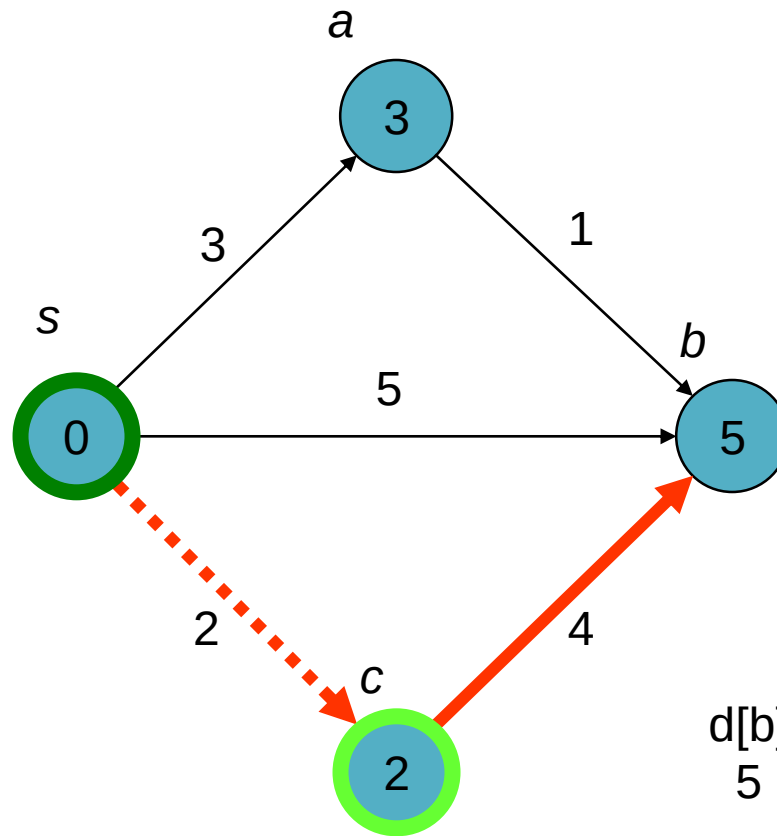
Determining Path Lengths



Determining Path Lengths

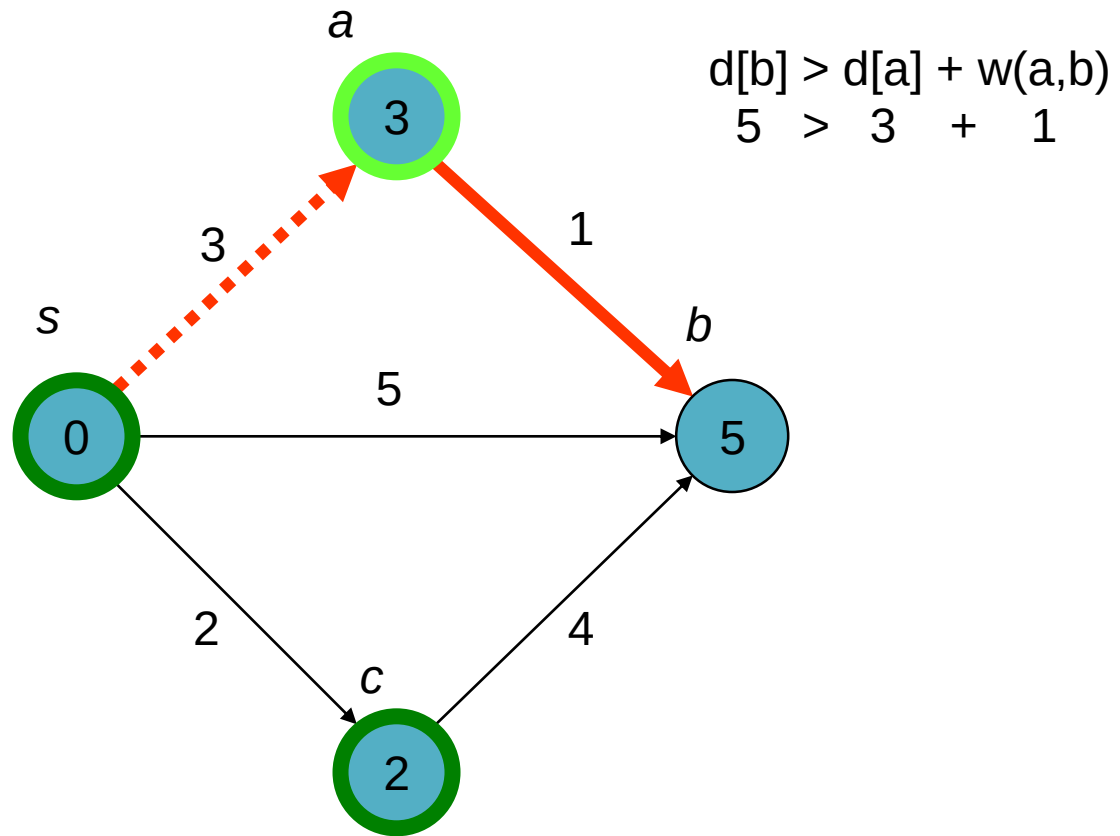


Determining Path Lengths

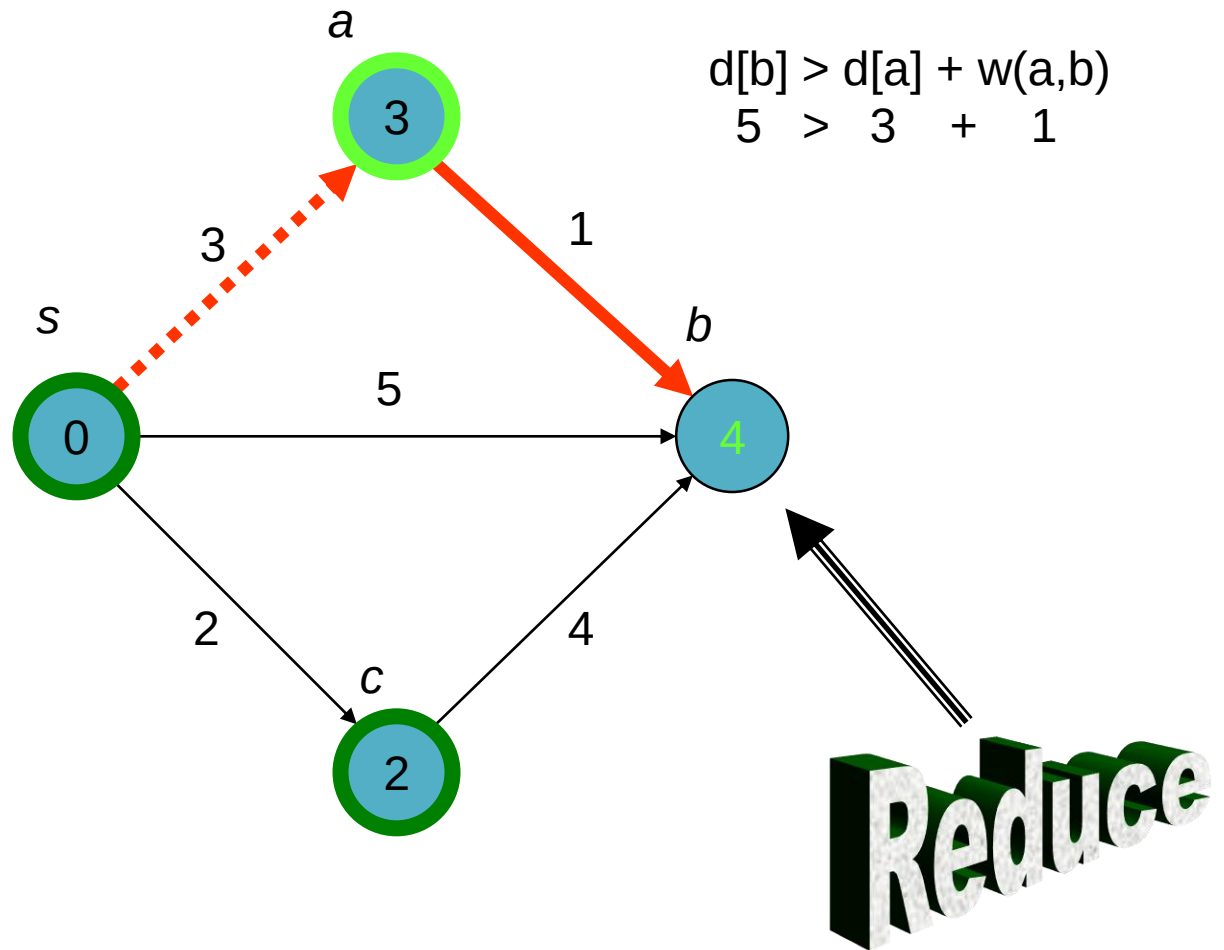


$$d[b] \leq d[c] + w(c,b)$$
$$5 \leq 2 + 4$$

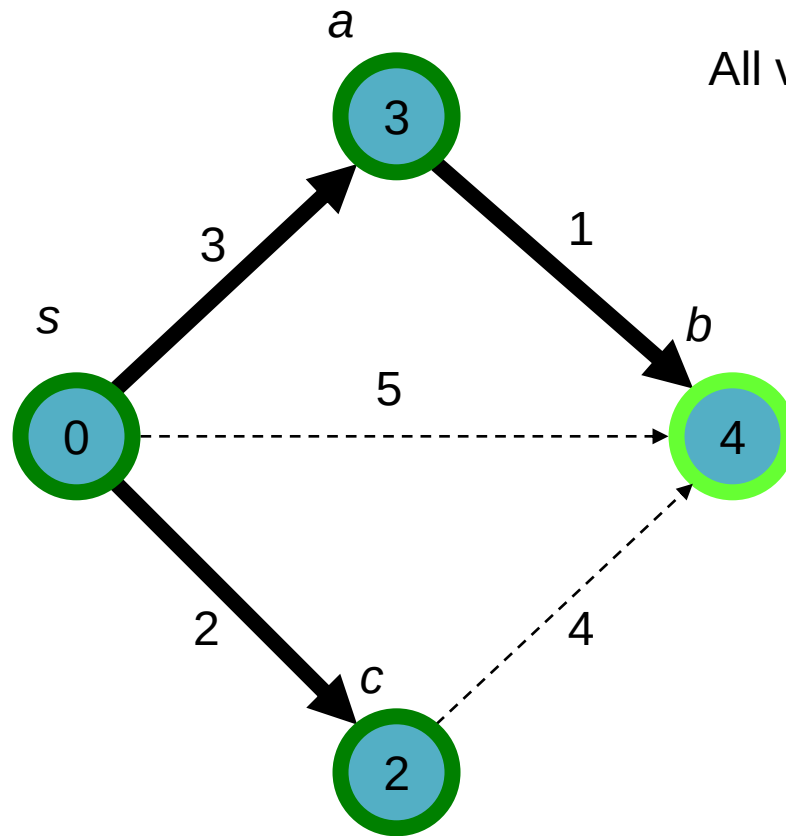
Determining Path Lengths



Determining Path Lengths



Path Lengths - FIN

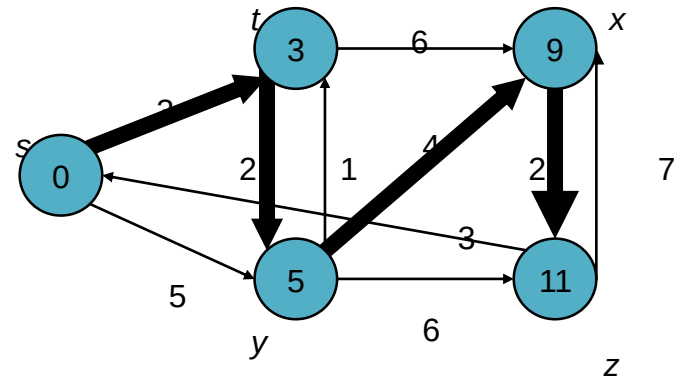
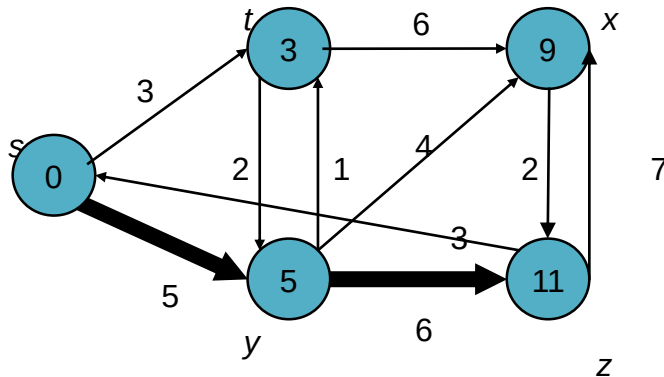
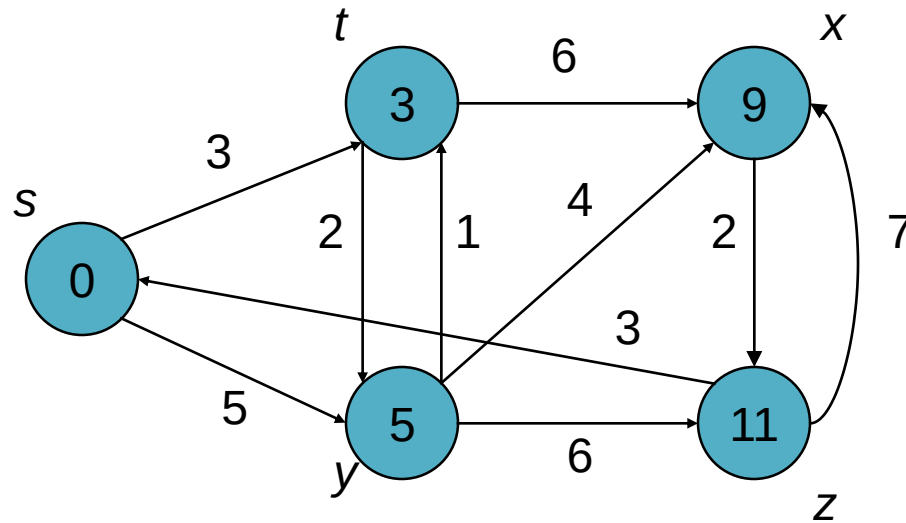


All vertices are in S

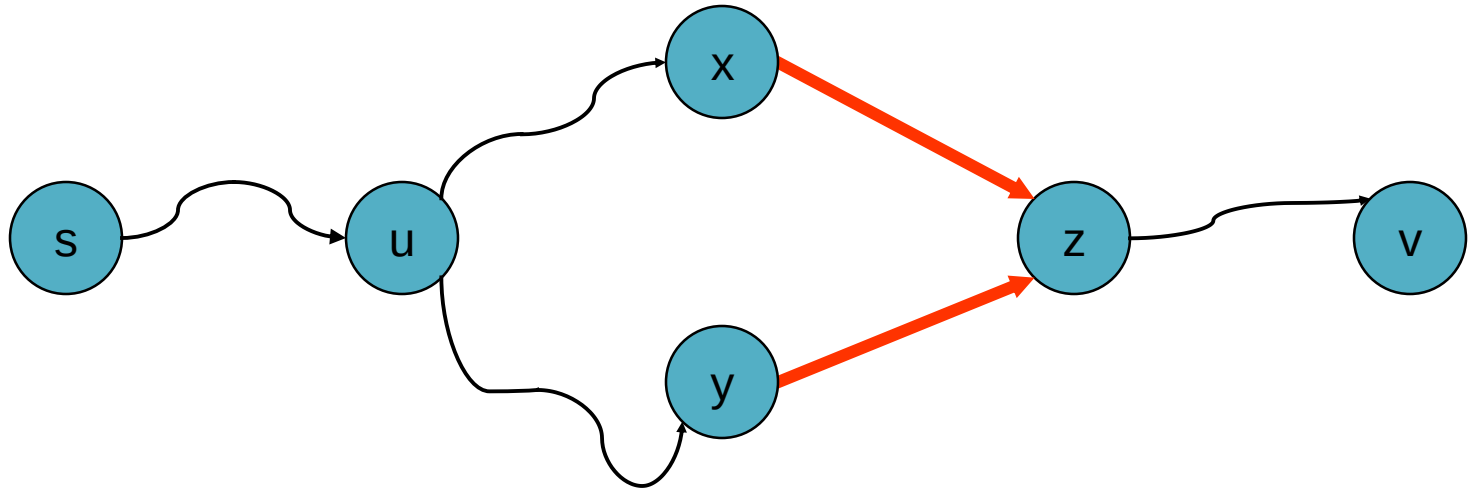
Properties of Shortest Paths

- Cannot contain a negative-weight cycle
 - No need to contain 0-weight cycle
 - No need to contain positive-weight cycle
 - Thus contains no cycles
- Will contain at most $V-1$ edges
 - Assumes no cycles (as stated above)
- Maintain predecessor information
 - E_π & V_π

Shortest Paths Not Unique



Shortest Path Distance Is Unique



$\pi[z]$ must be either x or y , but not both

This is true even if $d[x] + w(x,z) = d[y] + w(y,z)$

establishing that if s is source, $d[z]$ is well-defined and unique

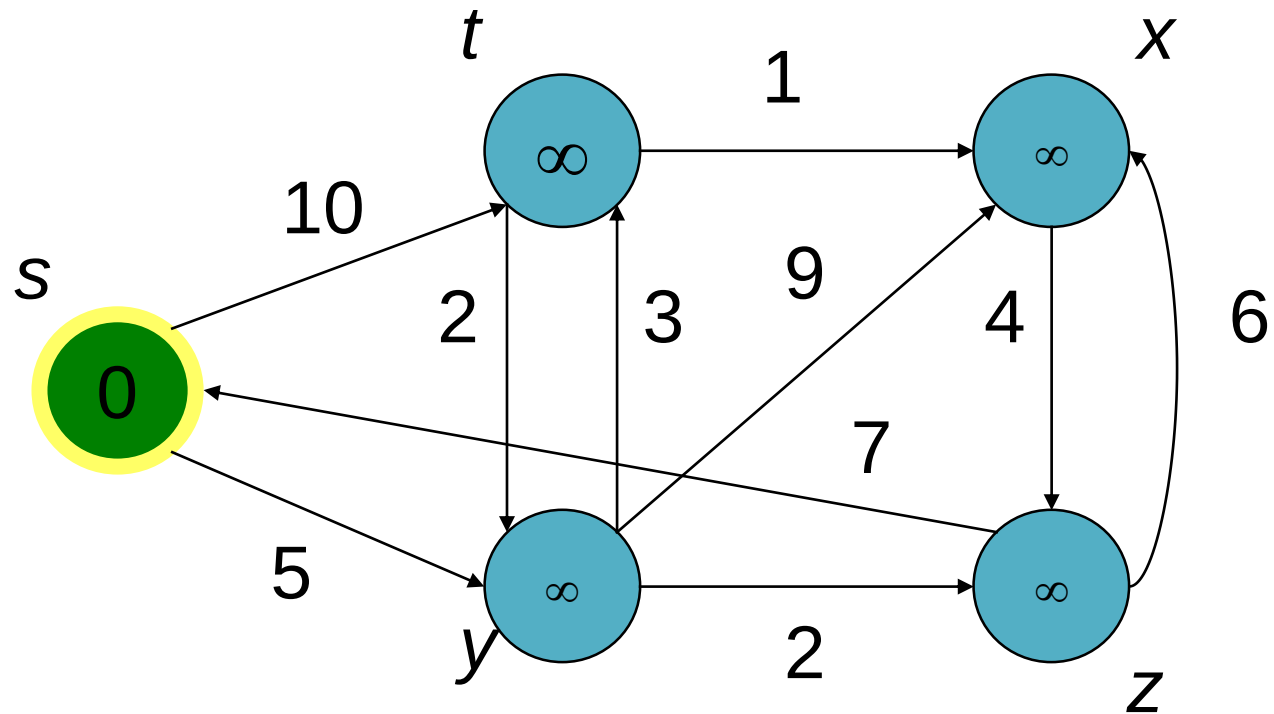
Dijkstra's Algorithm (1959)

Assume $G = (V, E)$, source s , & weight function w
Also assume all edges are non-negative

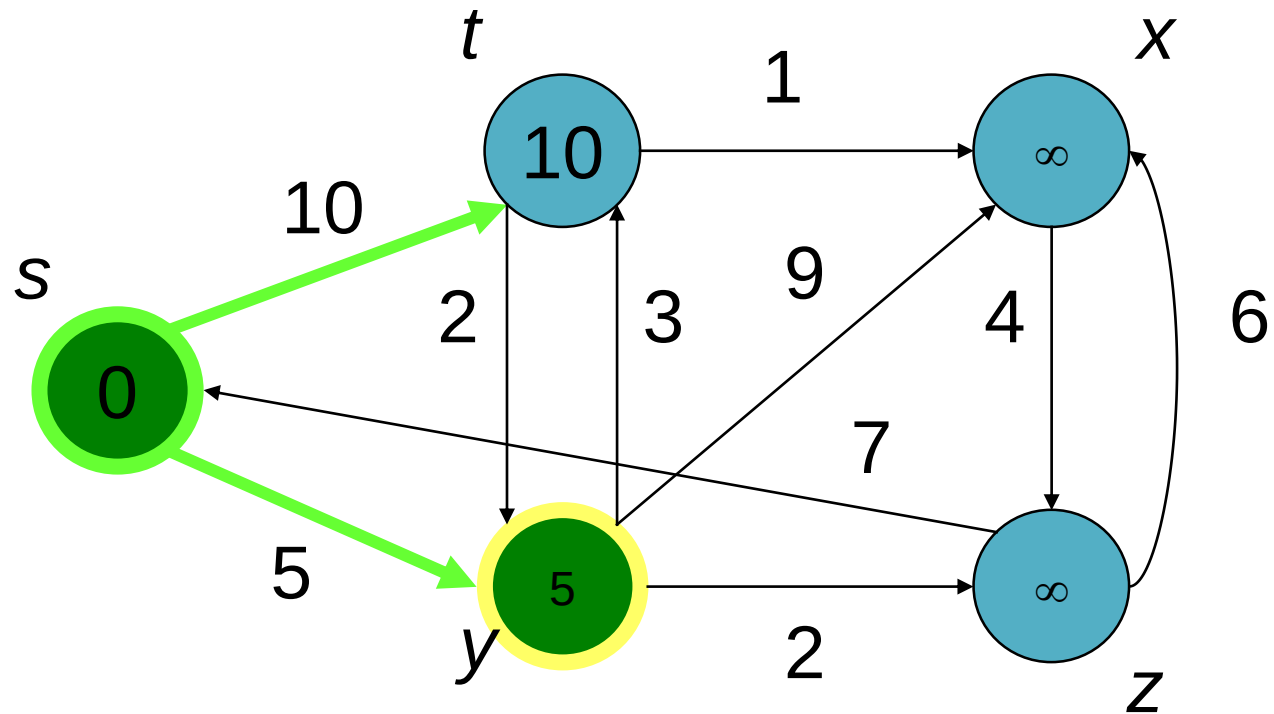
```
Dijkstra( $G, w, s$ )
    Initialize( $G, s$ )

     $S = \{\}$ 
     $Q = V[G]$ 
    while  $Q \neq \{\}$ 
         $u = \text{Extract\_Min}(Q)$ 
         $S.\text{Add}(u)$ 
        for each vertex  $v$  in  $u.\text{Adjacency}()$ 
            Relax( $u, v, w$ )
```

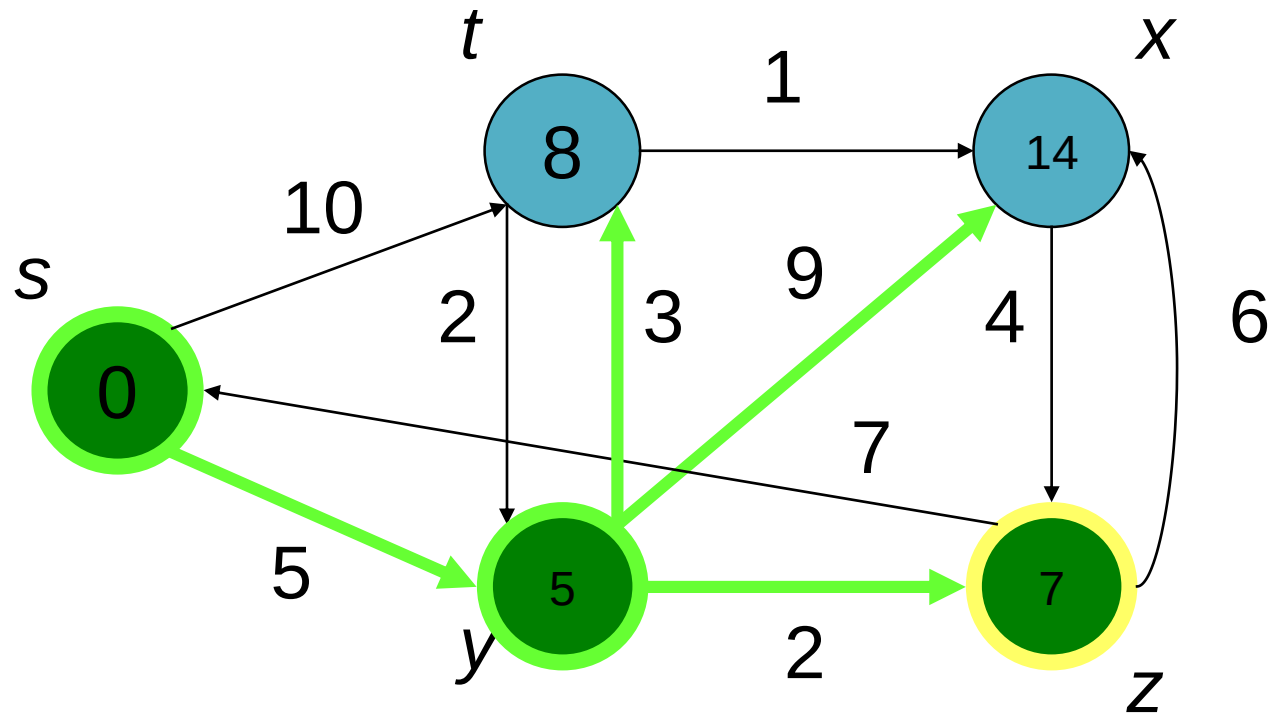
Example: Dijkstra's



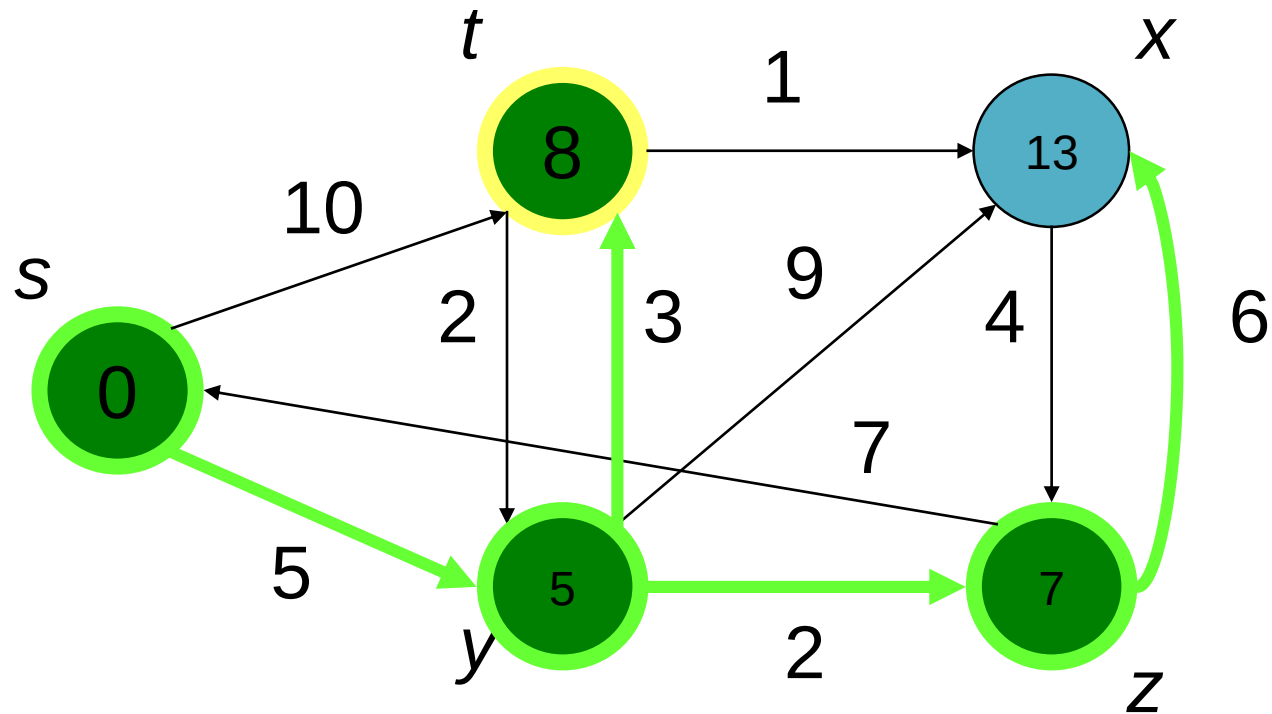
Example: Dijkstra's



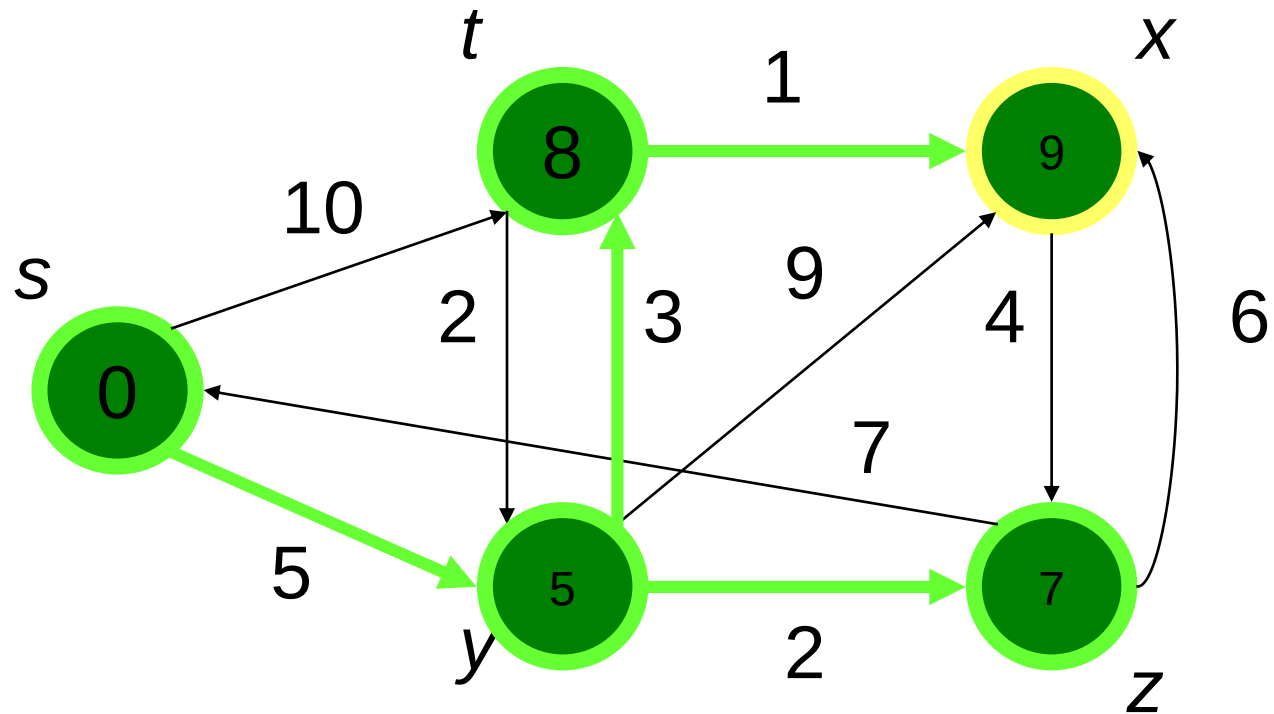
Example: Dijkstra's



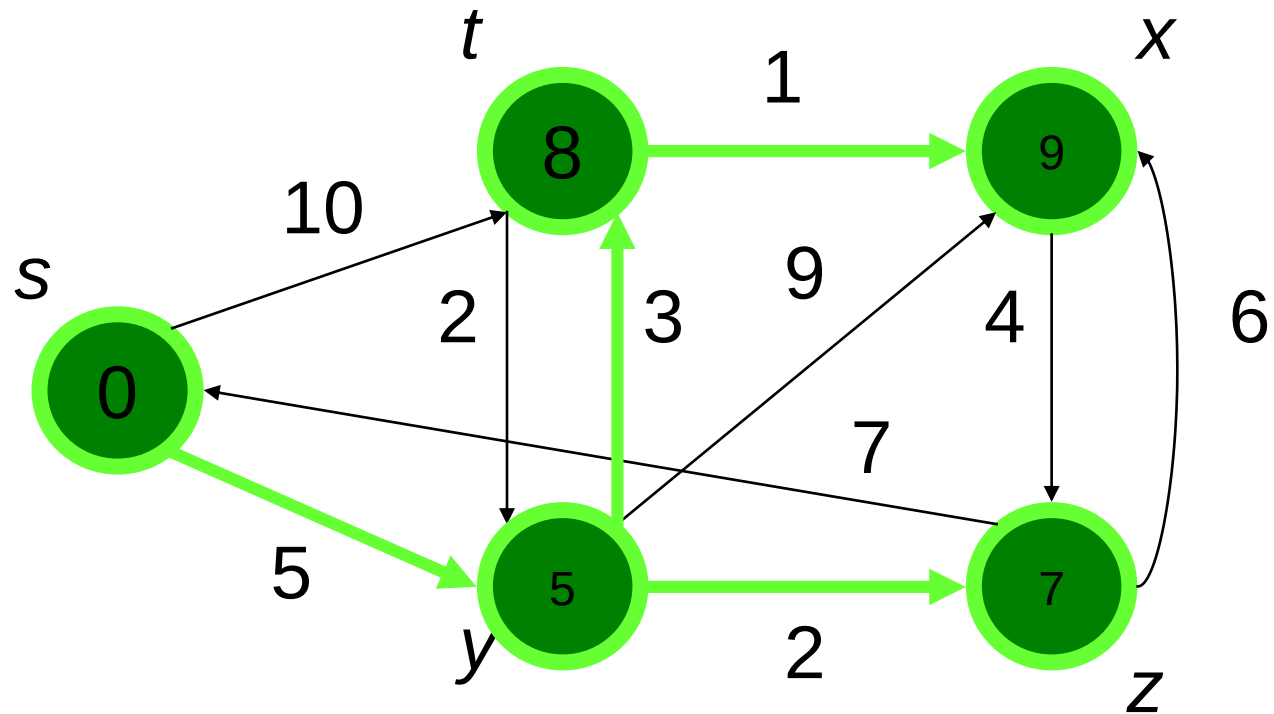
Example: Dijkstra's



Example: Dijkstra's



Dijkstra's - FIN

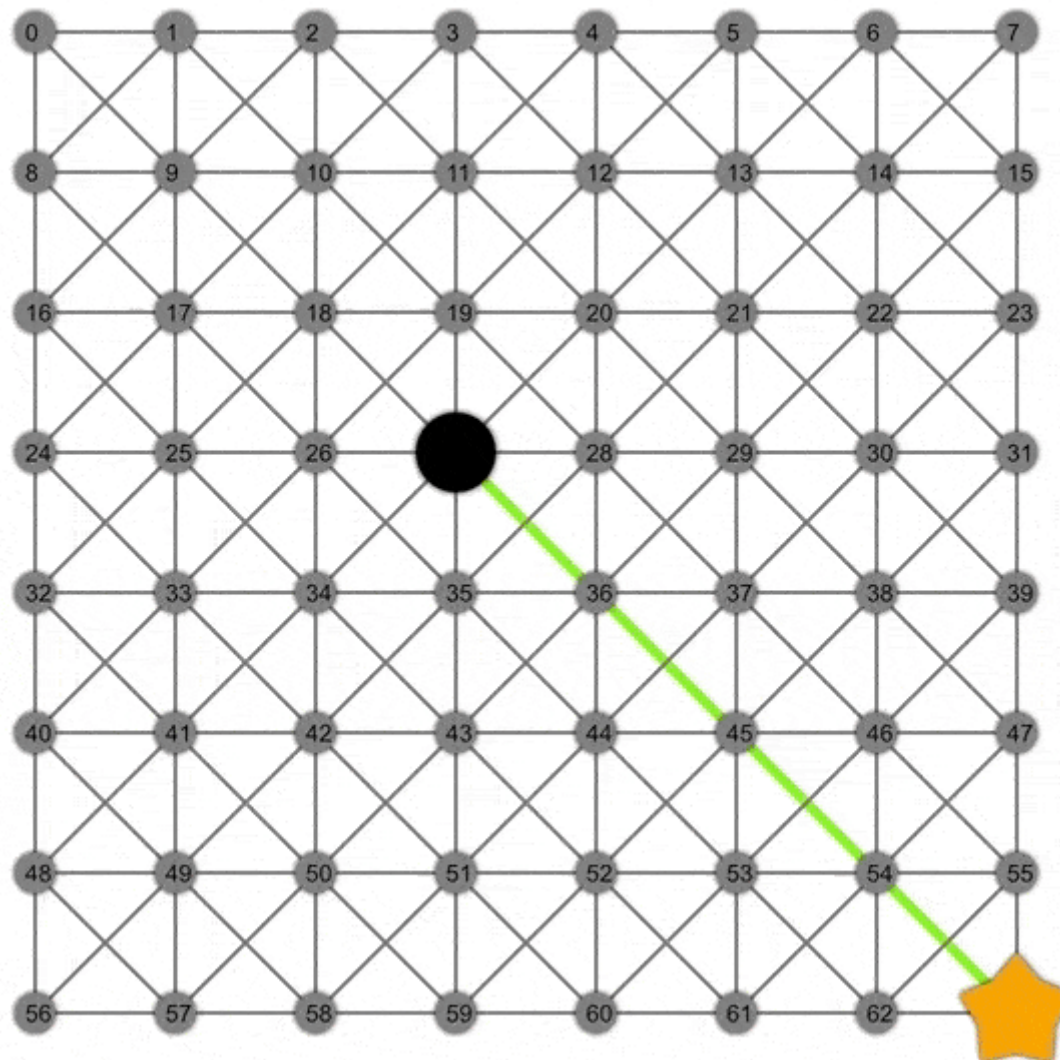


Runtime of Dijkstra's

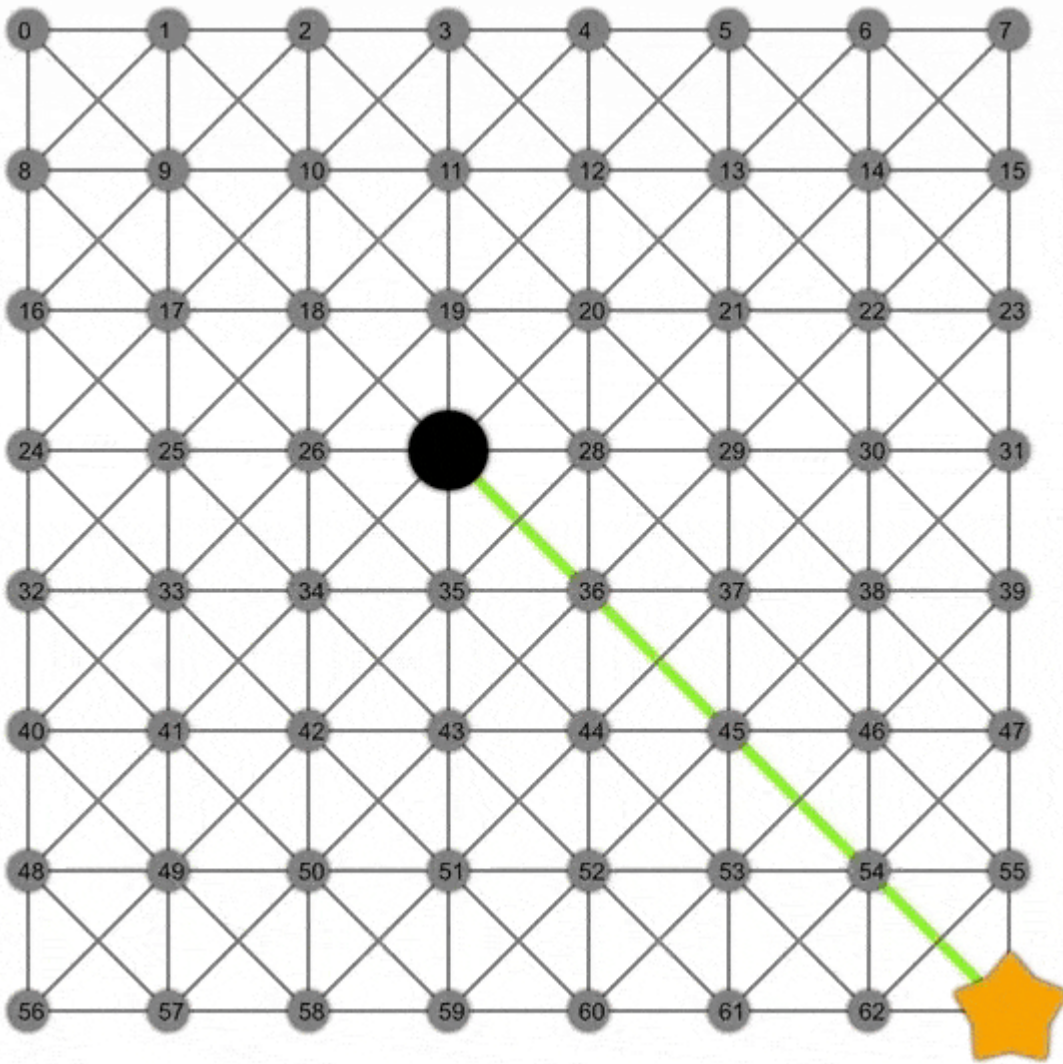
- $O(V \log V + E \log V)$
- $O(E \log V)$
 - If G is connected
- $O(V \log V + E)$
 - If we use a Fibonacci heap



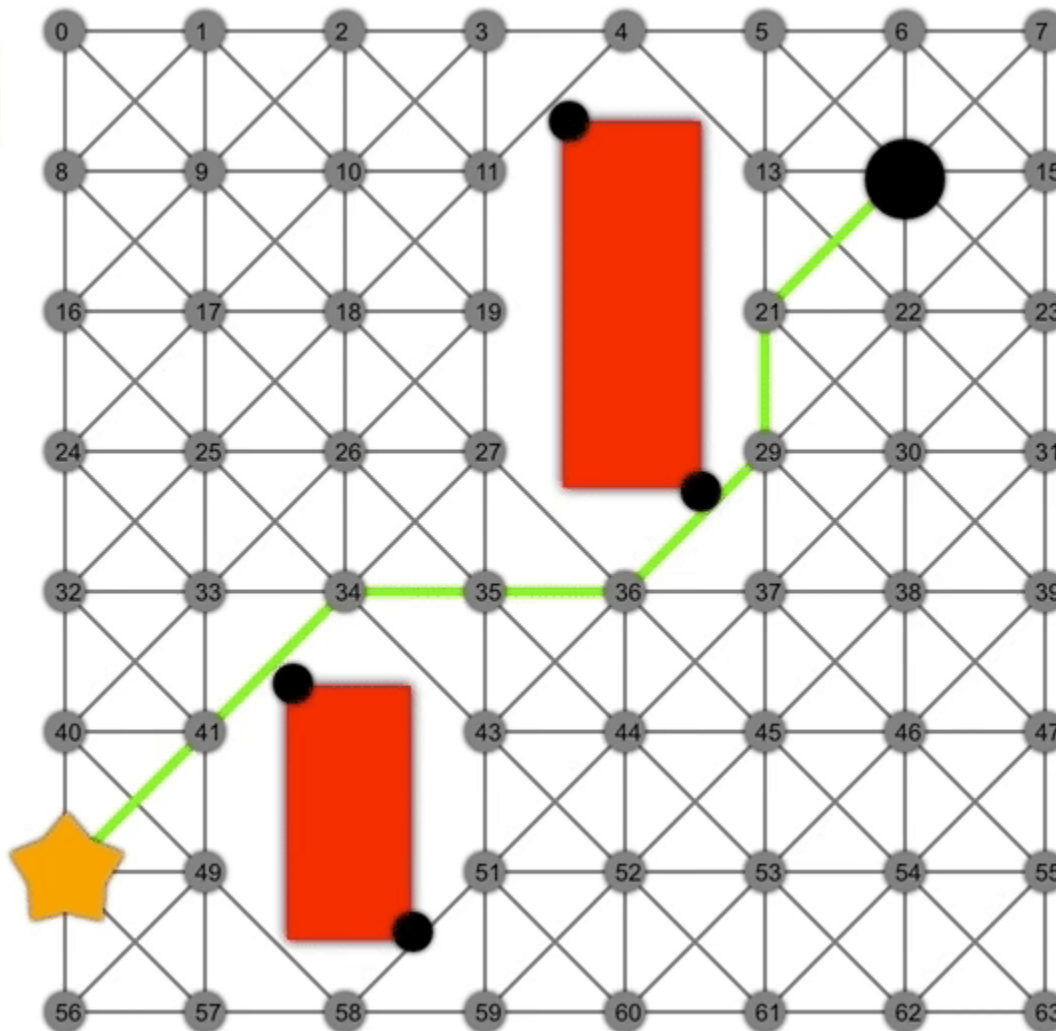
Better
if $V \ll E$
(i.e. sparsely
connected)



BFS!



The main difference from the BFS is that Dijkstra takes into account the costs, the algorithm can now find the actual shortest path considering costs of traveling from node to node.

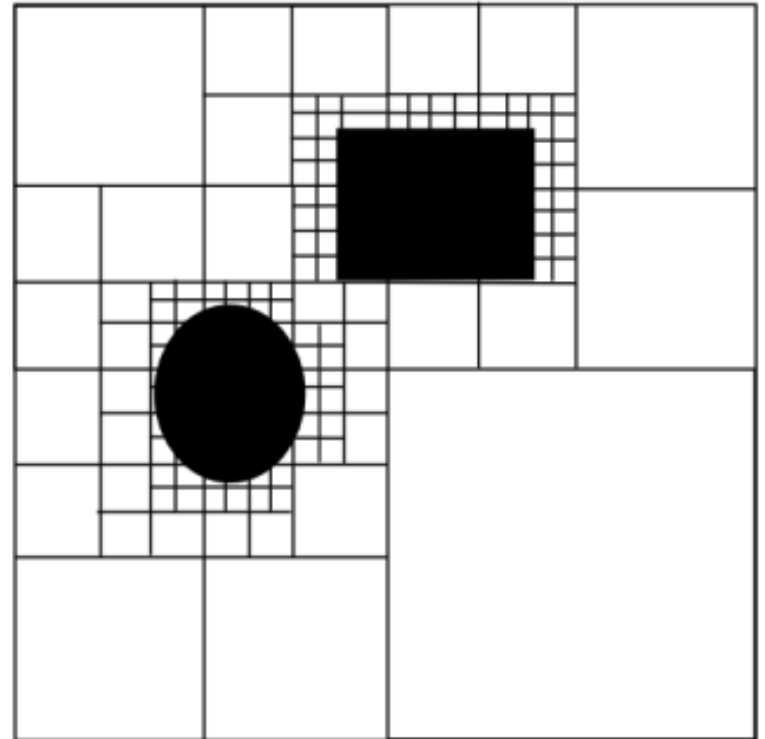
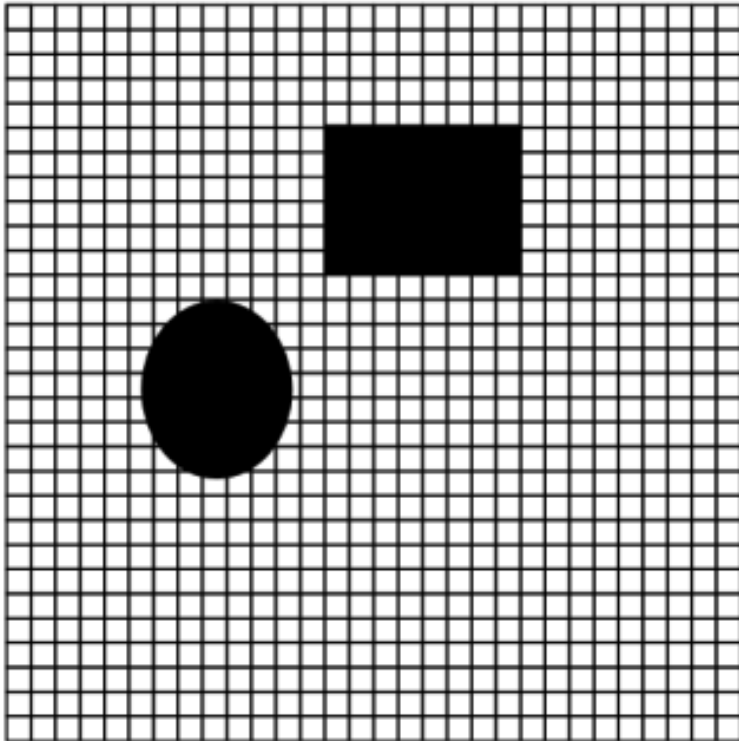


To obtain the A* algorithm from our generalised graph search we just need to pass an actual heuristic function, let's use the euclidean distance between the two nodes as example. By weighting the nodes based on "cost to the node" + "estimation of cost from node to goal" we can speed up the search by going first into nodes that look promising.

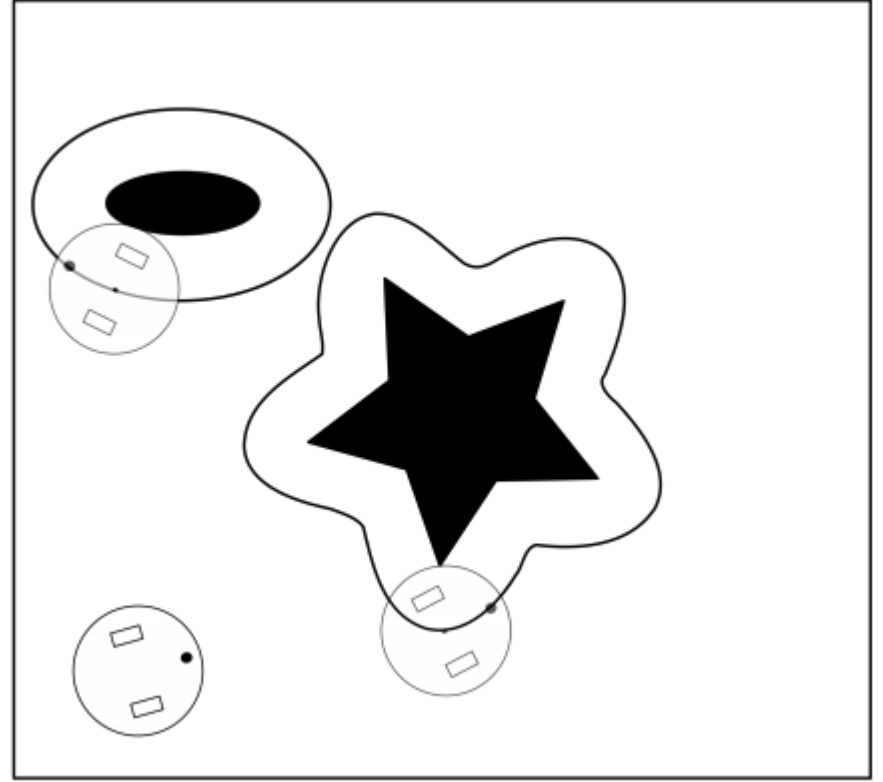
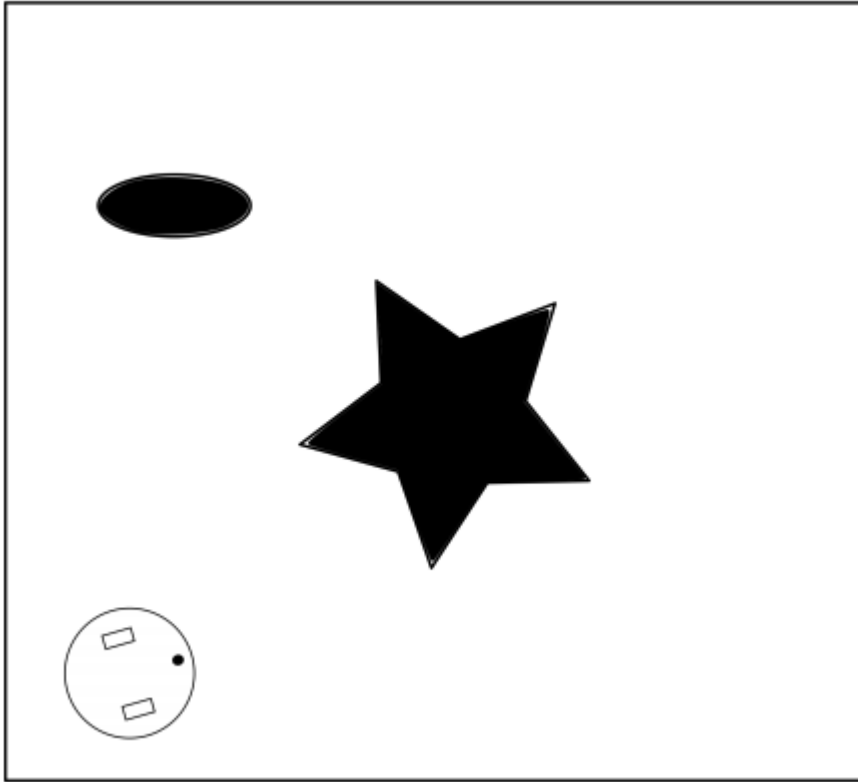
A*

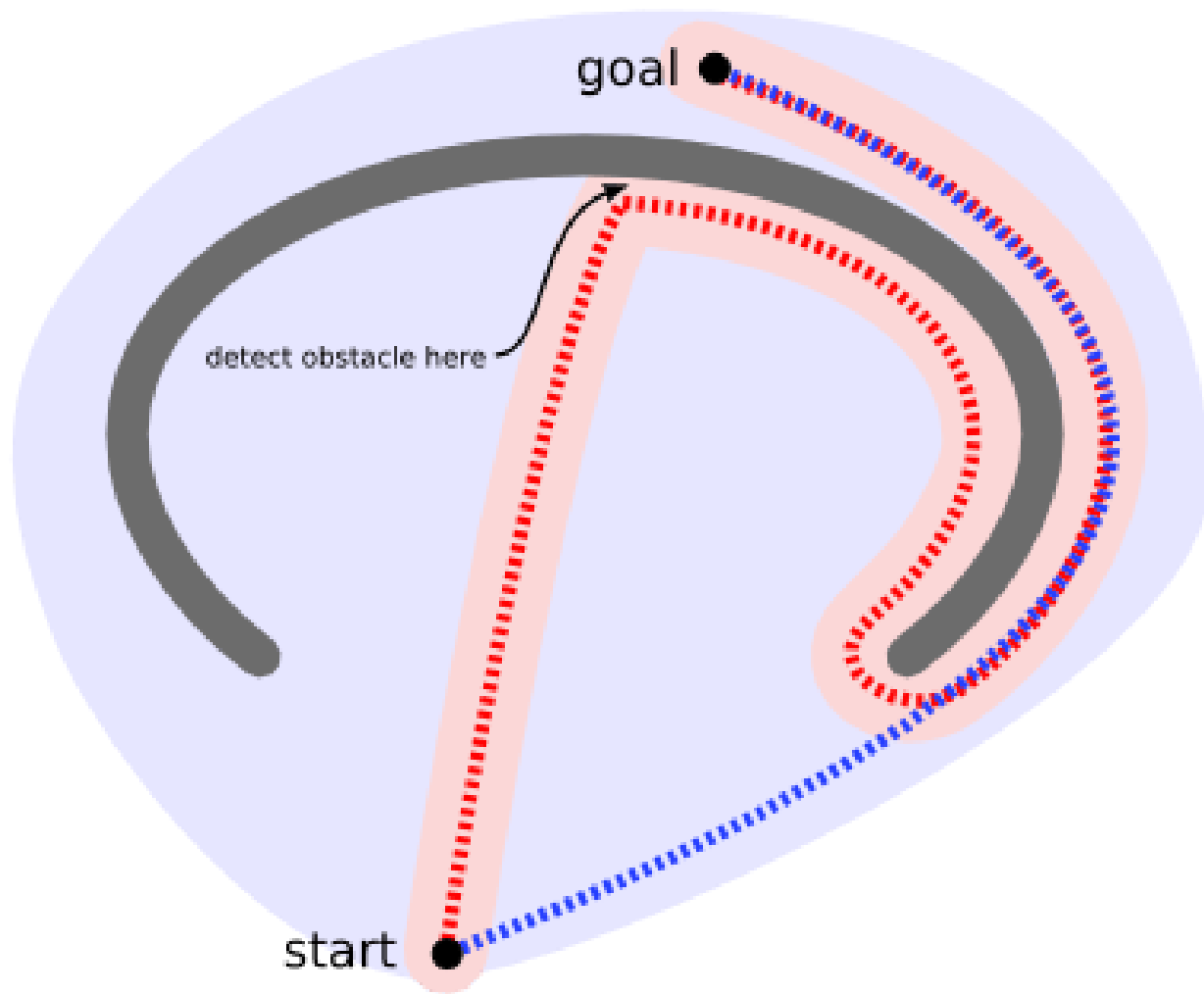


iRobot



iRobot





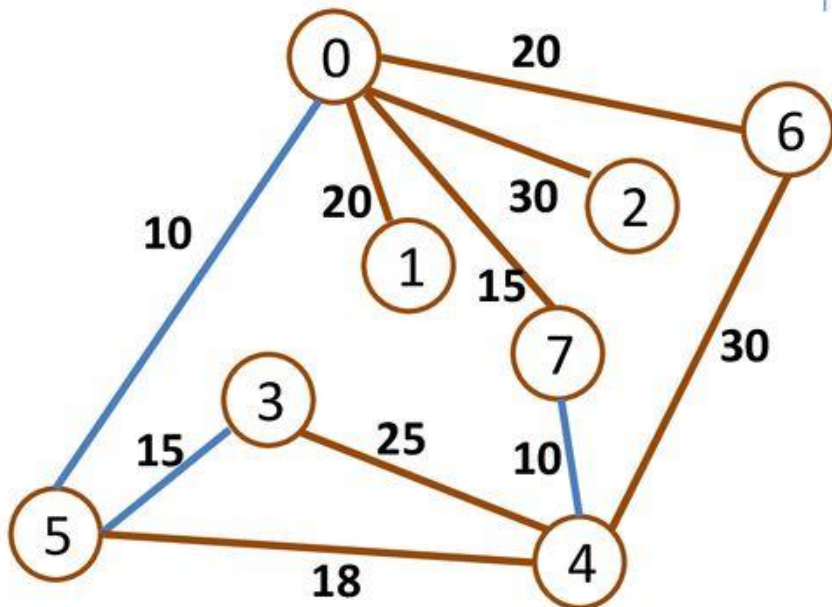


Spanning Tree

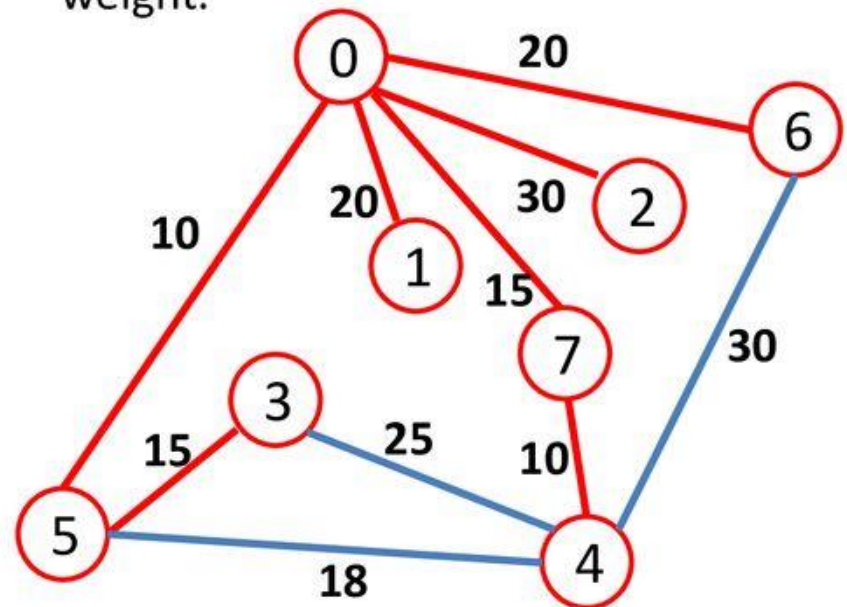
- A **spanning tree** is a tree that connects all vertices of the graph.
- The **weight/cost of a spanning tree** is the sum of weights of its edges.

- Minimum spanning tree (MST)

- Is a **Spanning Tree**: connects all vertices of the graph.
- Has the **smallest total weight** of edges.
- It is **not unique**: Two different spanning trees may have the (same) minimum weight.



Weight: $20+15+30+20+30+25+18 = 158$



Weight: $10+20+15+30+20+10+15 = 120$ 120

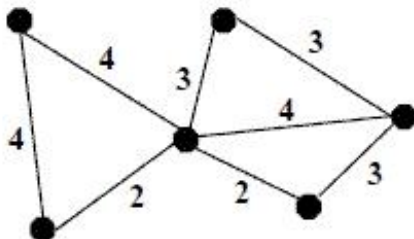
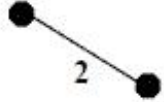
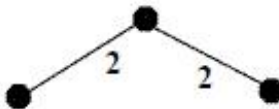
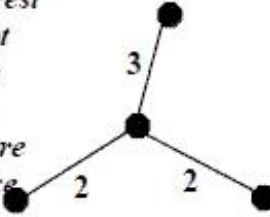
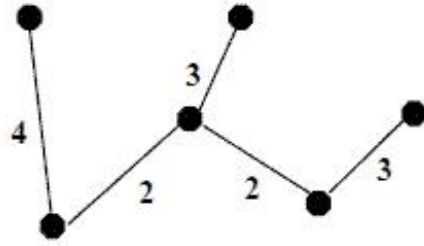
MST Defined

$$w(T) = \sum_{(u,v) \in T} w(u,v)$$

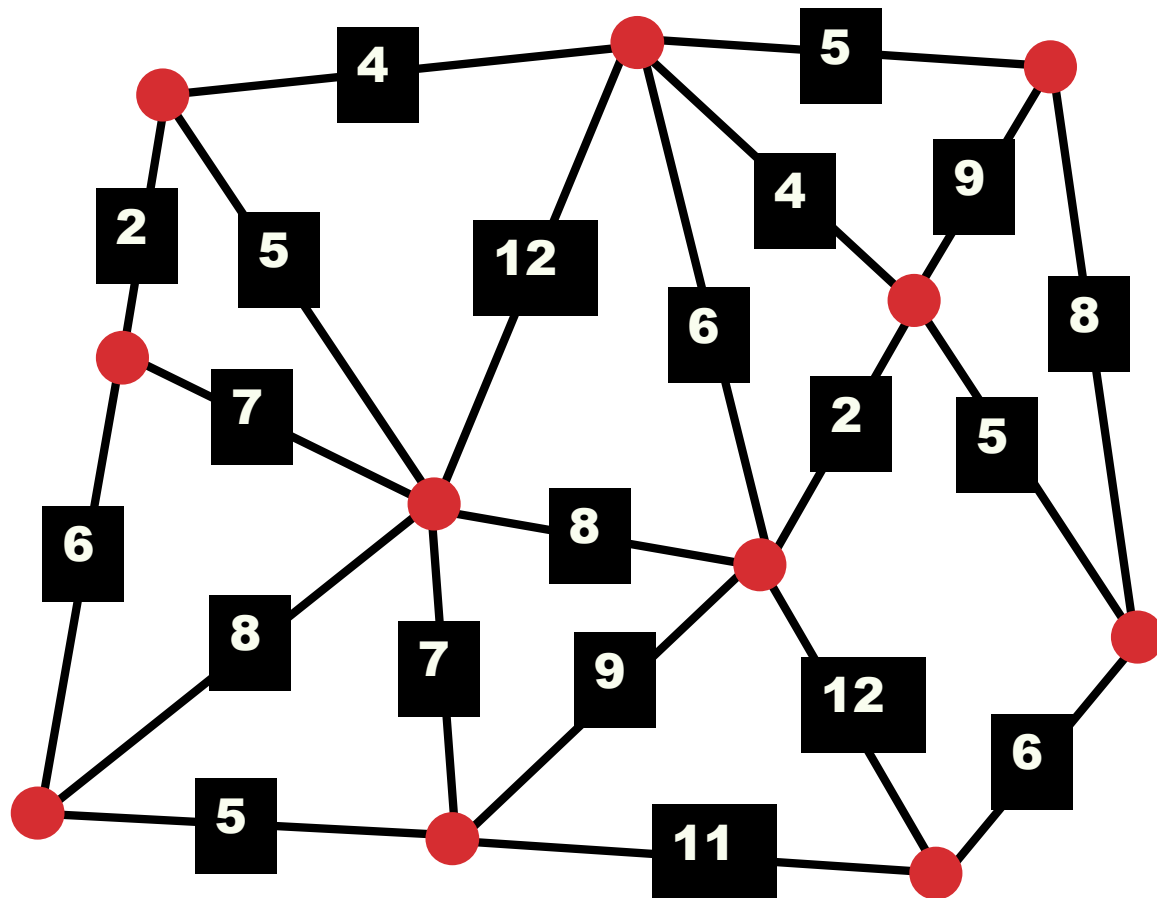
Minimum Spanning Tree

- Connect all nodes at minimum cost.
- One solution.
 - i.e. There is just one minimum
- Two algorithms
 - Prim's
 - Kruskal's

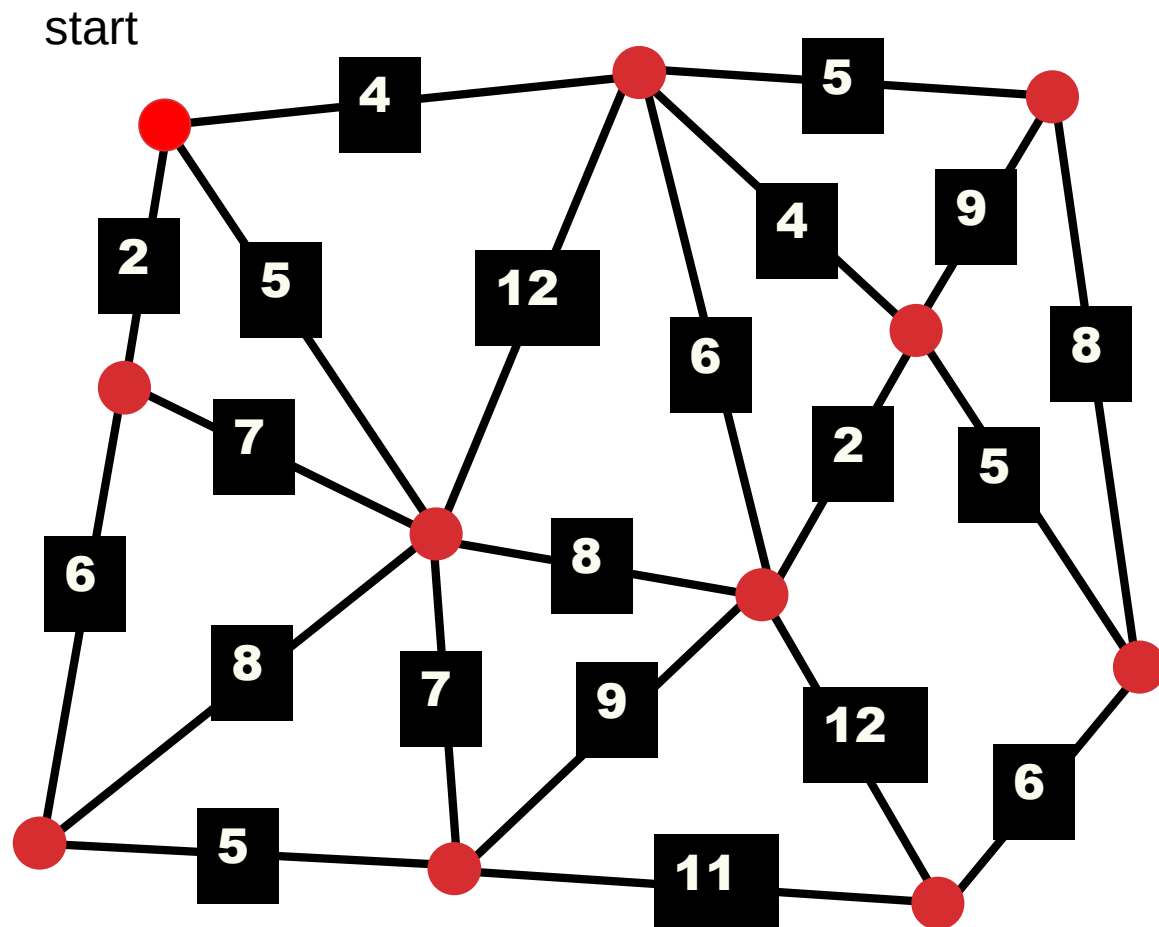
Prim's Algorithm

1 <i>Given a network.....</i> 	2 <i>Choose a vertex</i> 	3 <i>Choose the shortest edge from this vertex.</i> 
4 <i>Choose the nearest vertex not yet in the solution.</i> 	5 <i>Choose the next nearest vertex not yet in the solution, when there is a choice choose either.</i> 	6 <i>Repeat until you have a minimal spanning tree.</i> 

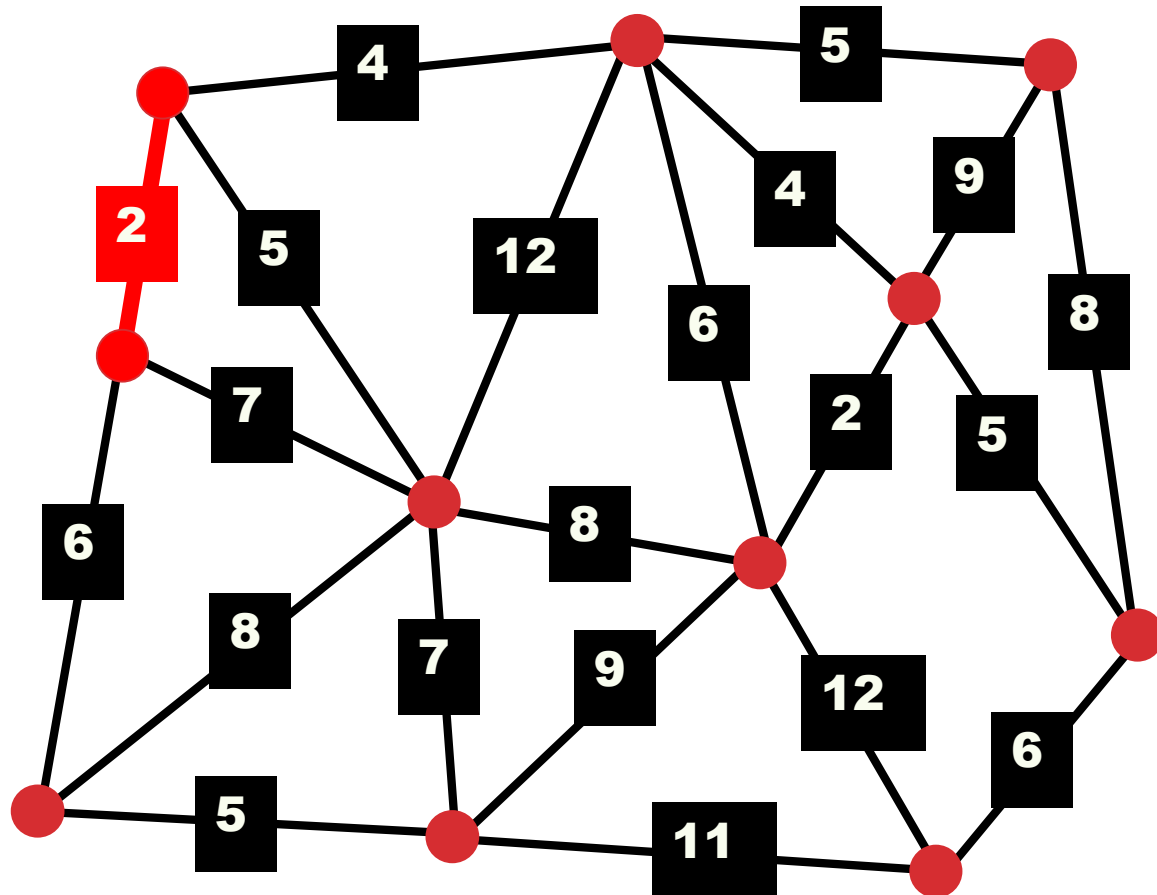
A Graph



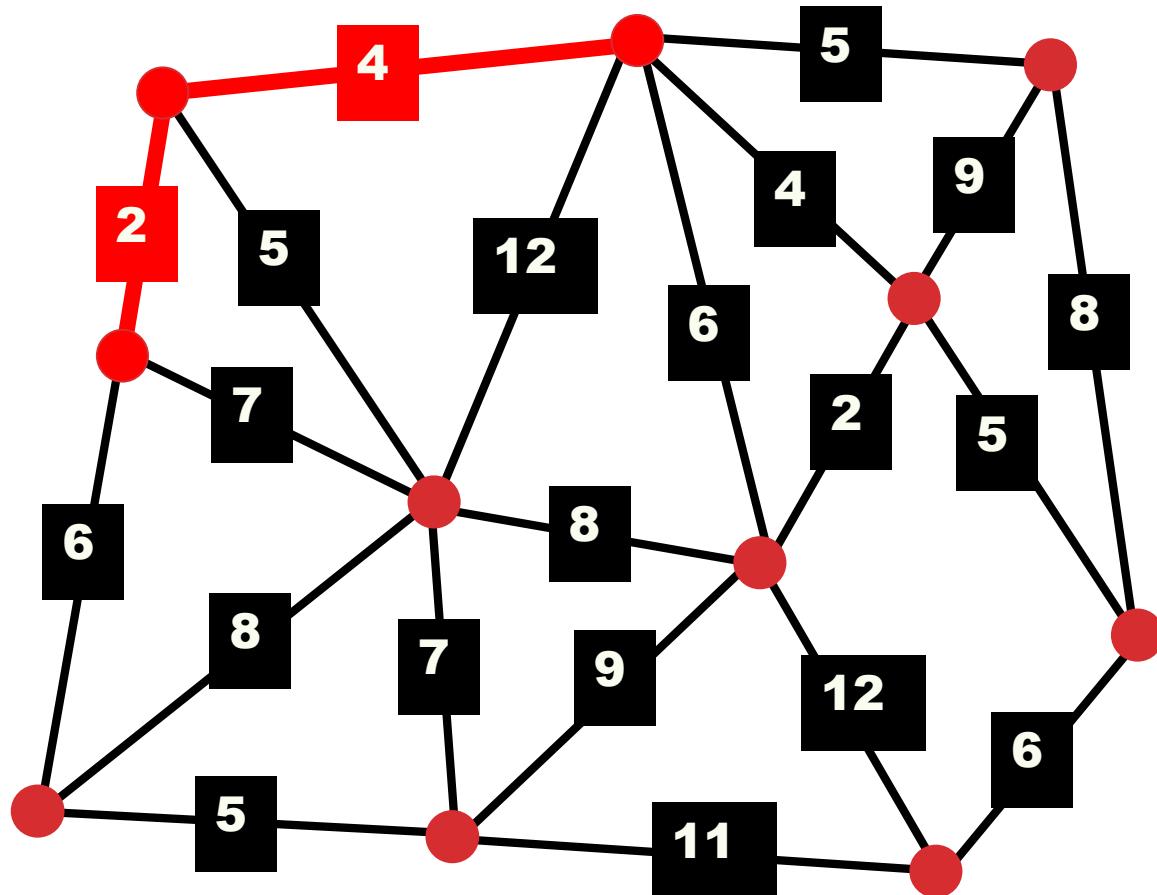
Prim's



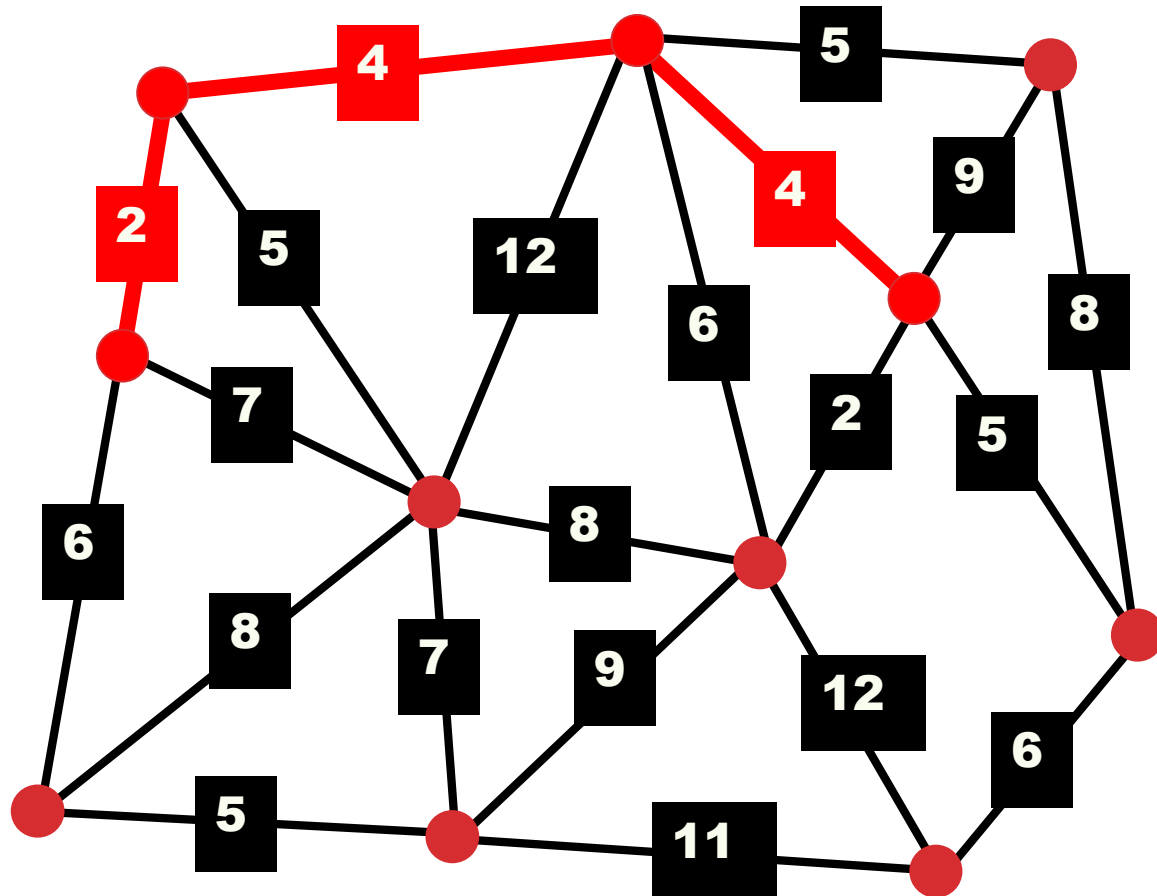
Prim's



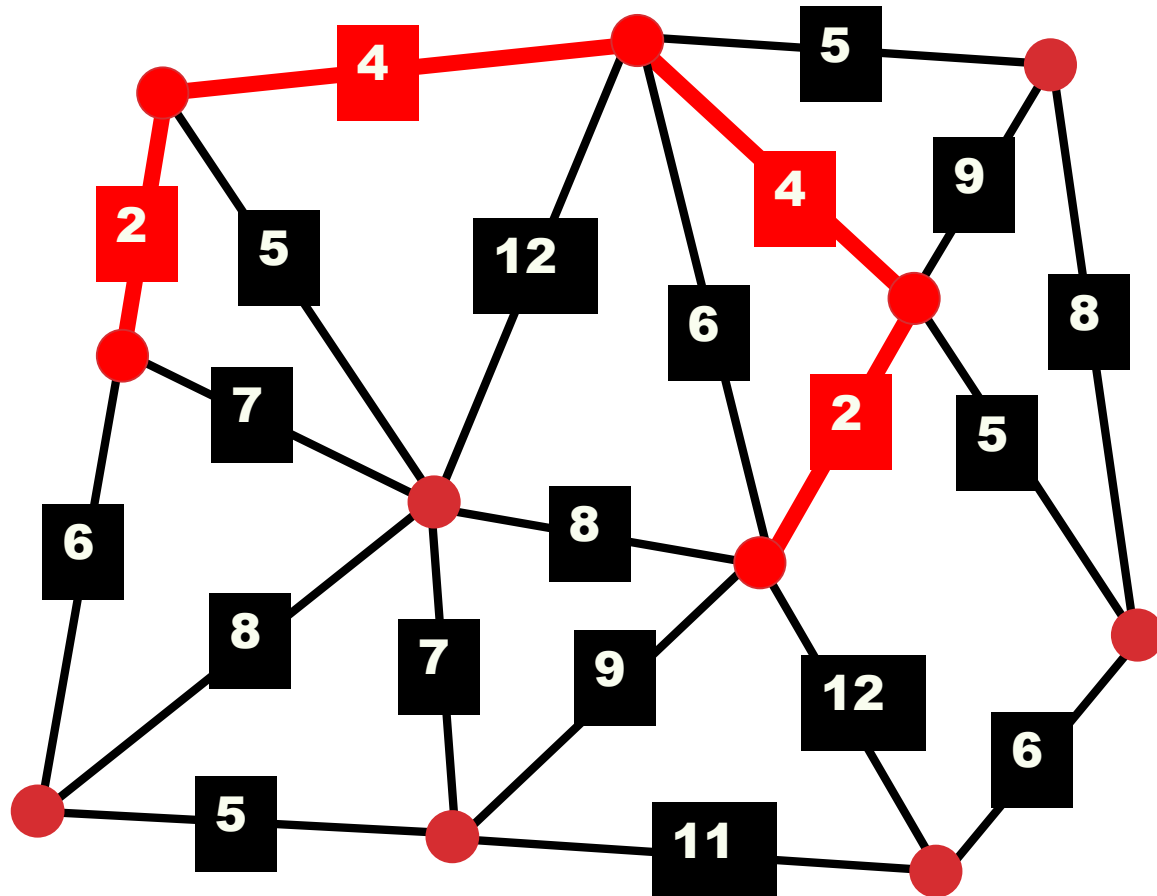
Prim's



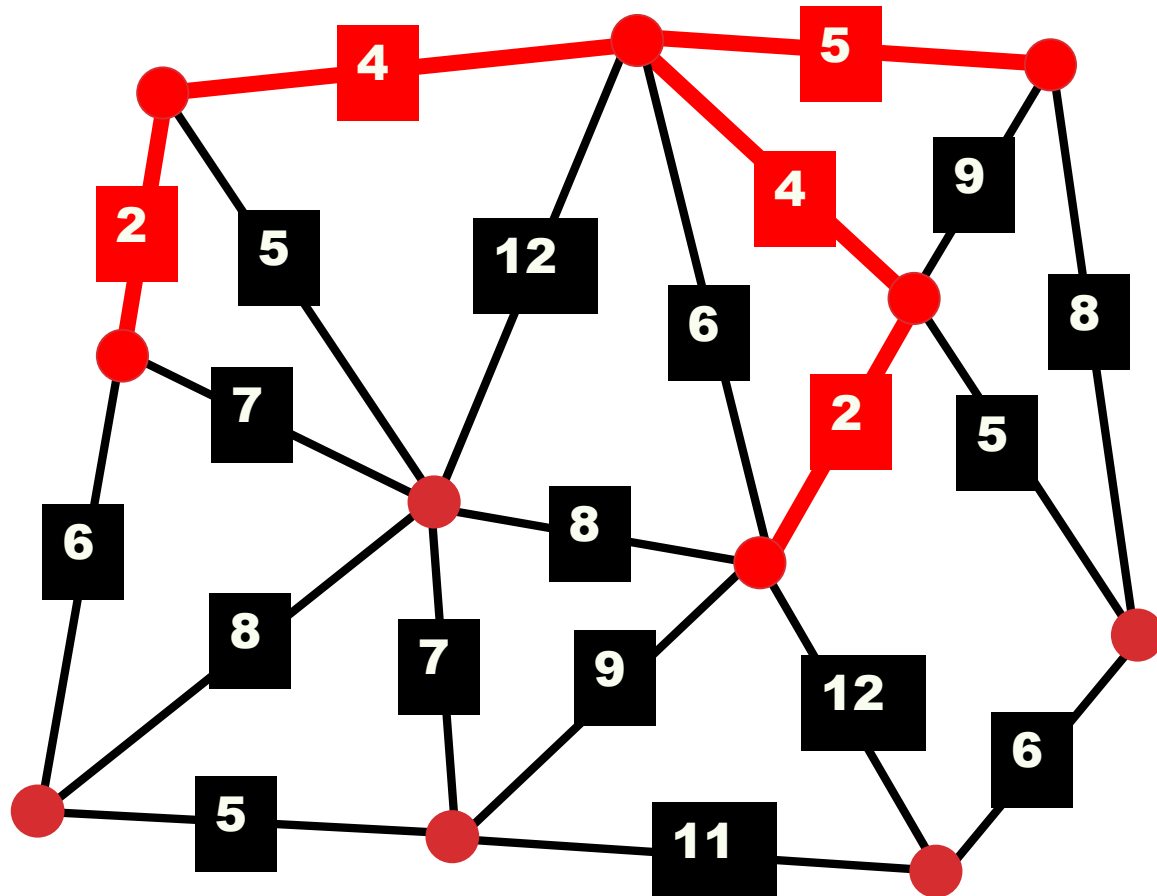
Prim's



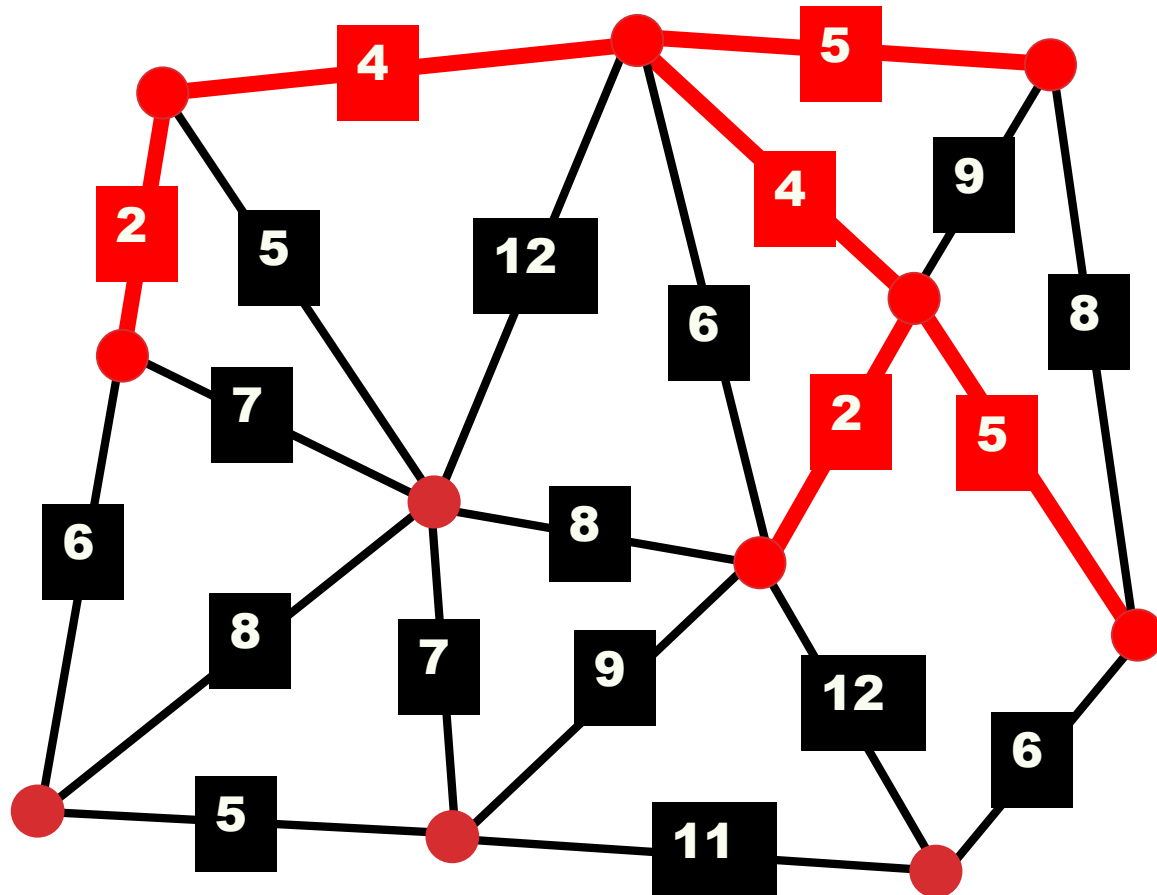
Prim's



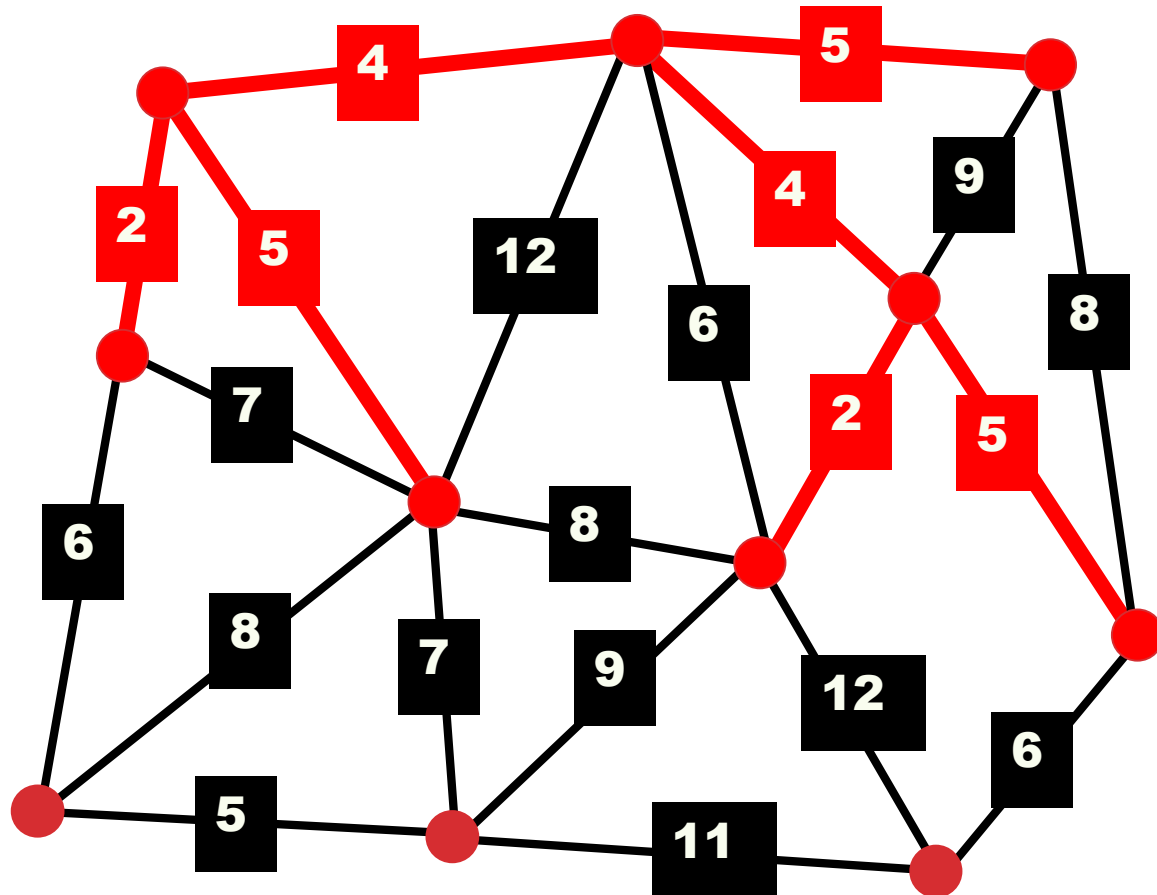
Prim's



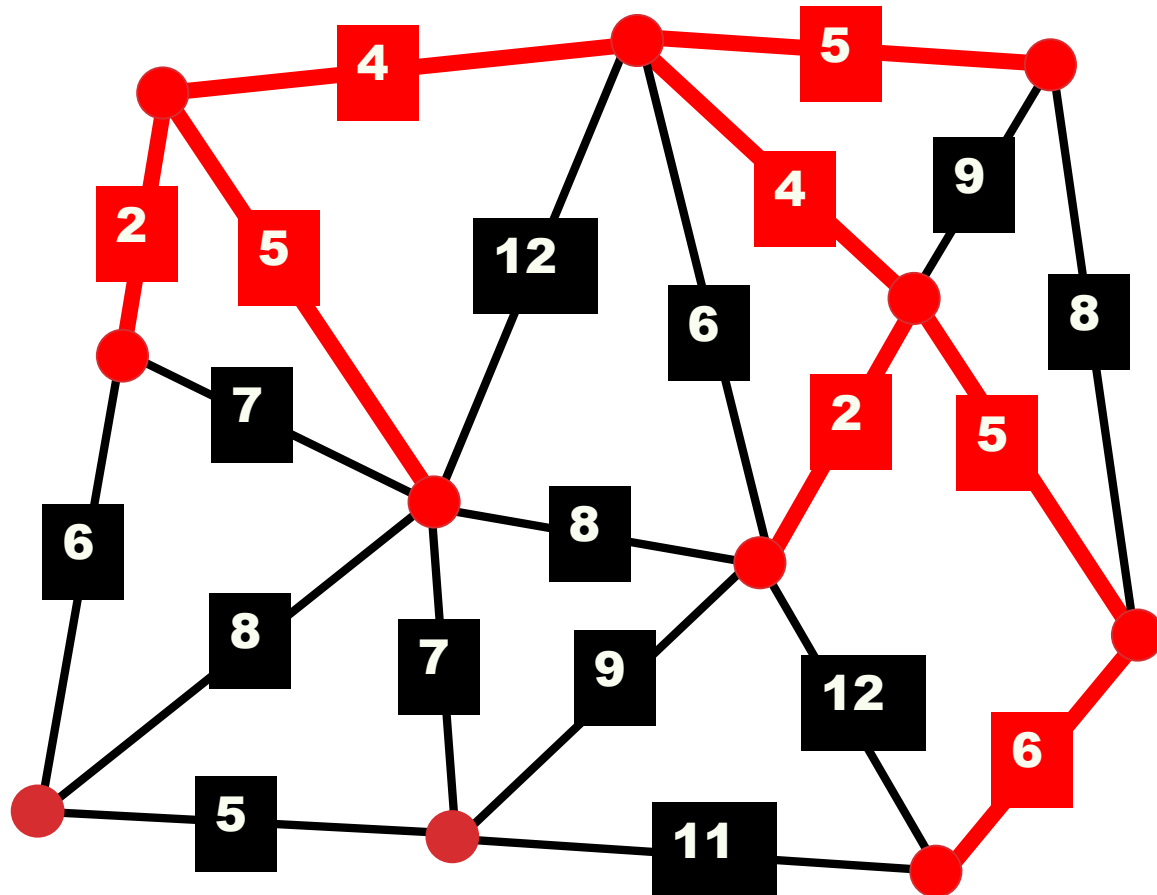
Prim's



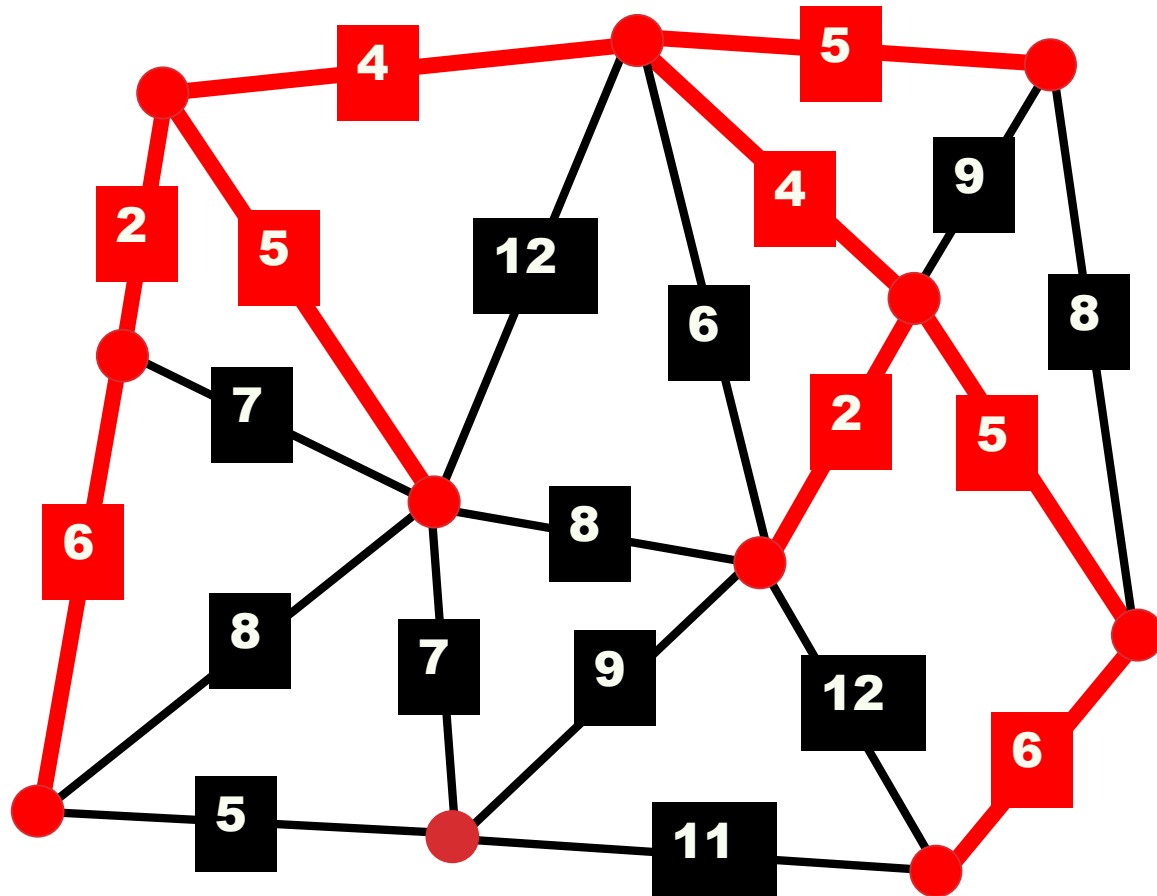
Prim's



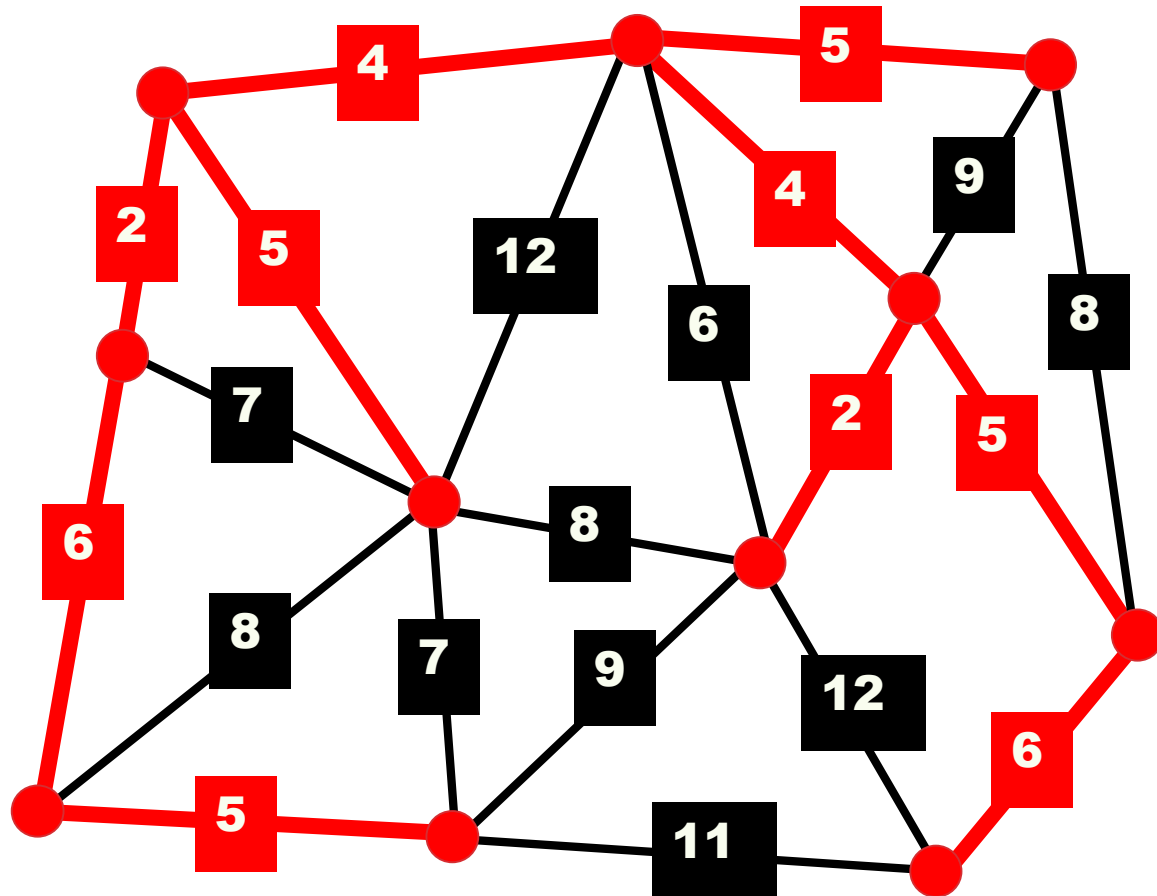
Prim's



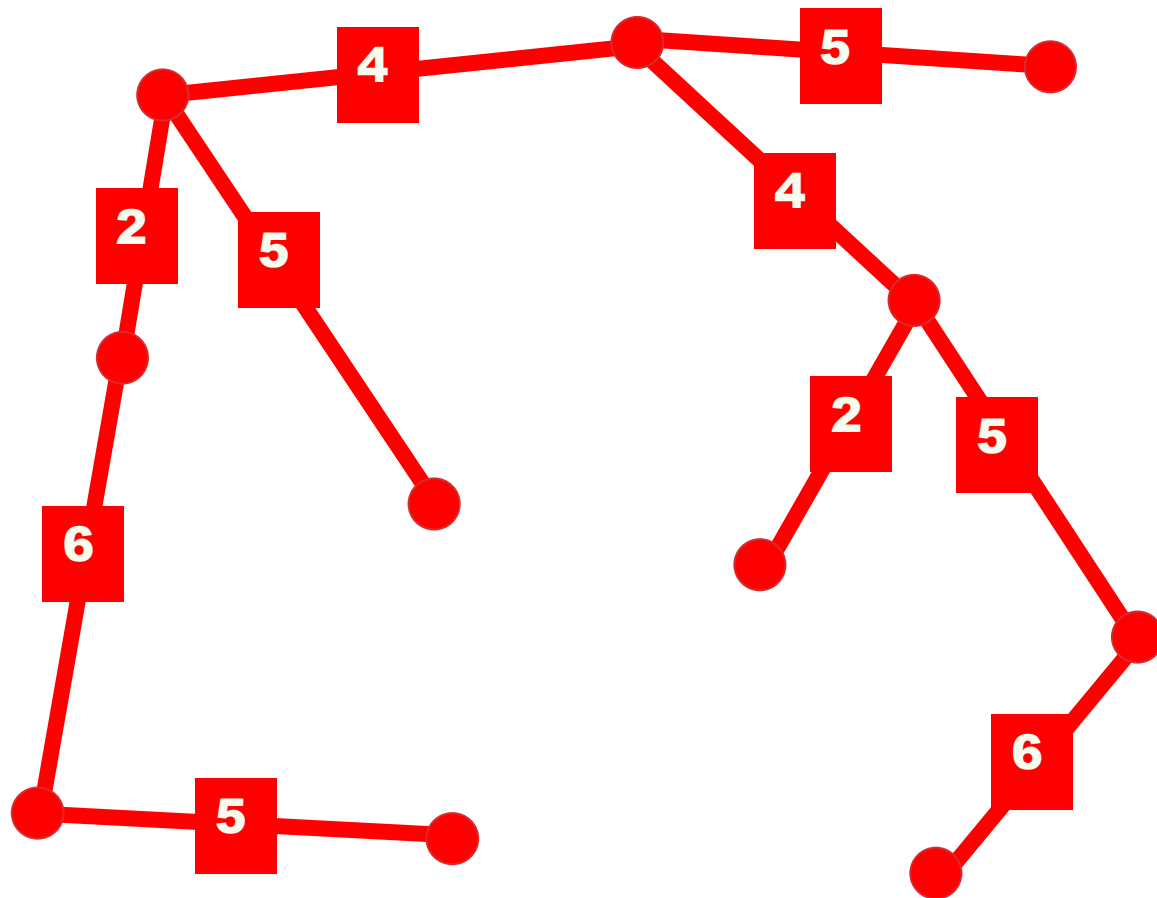
Prim's



Prim's

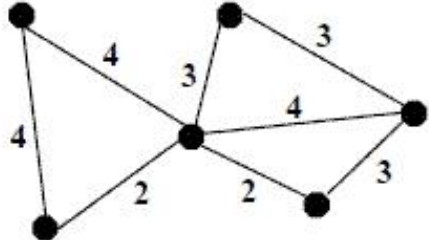
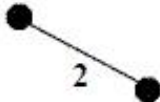
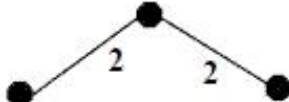
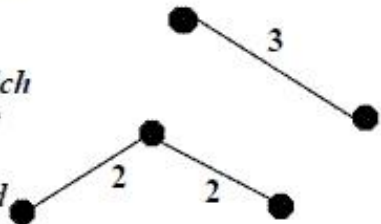
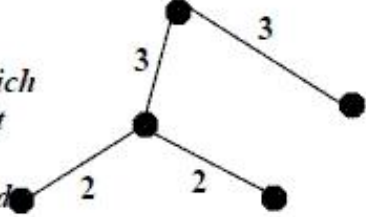
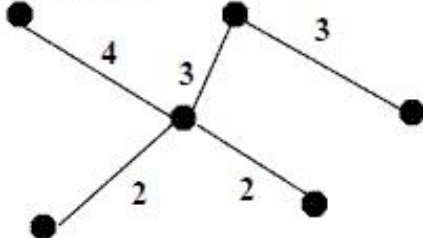


Prim's

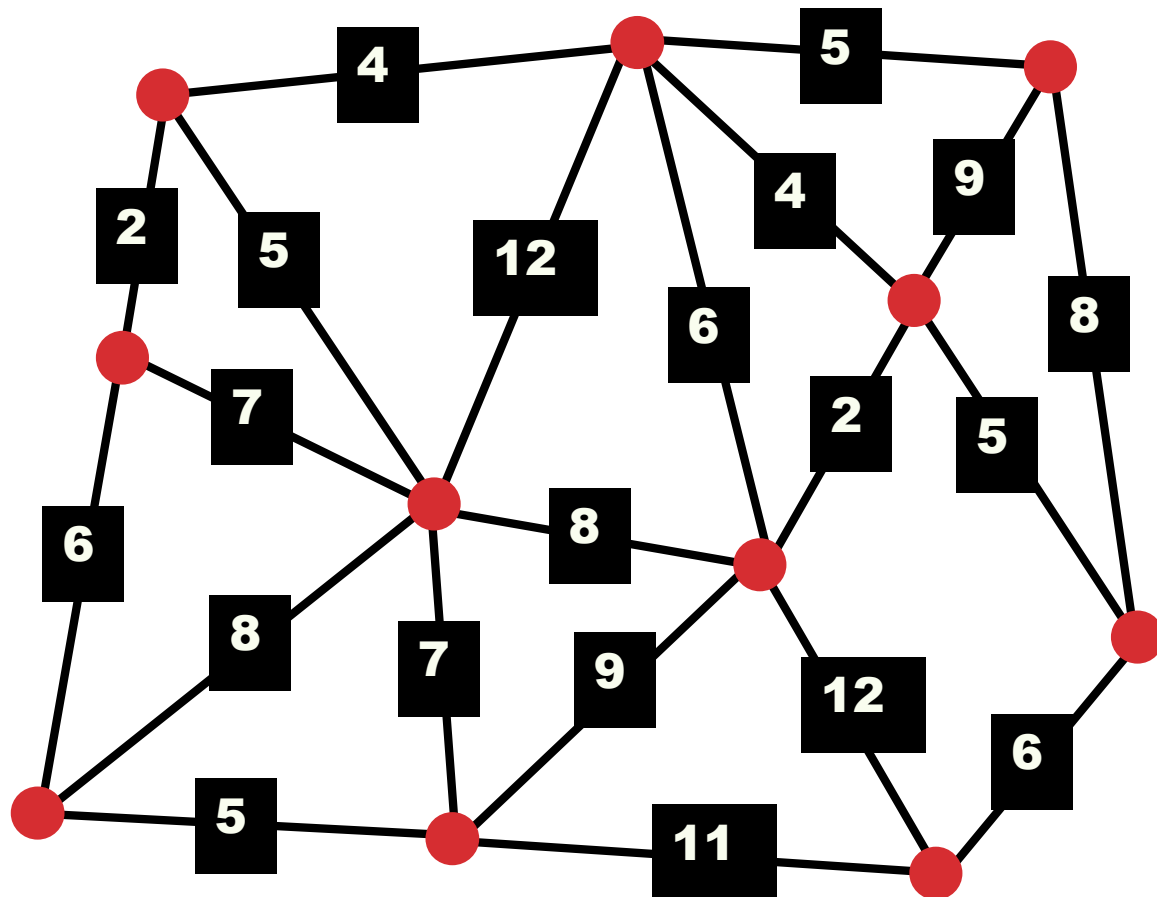


44

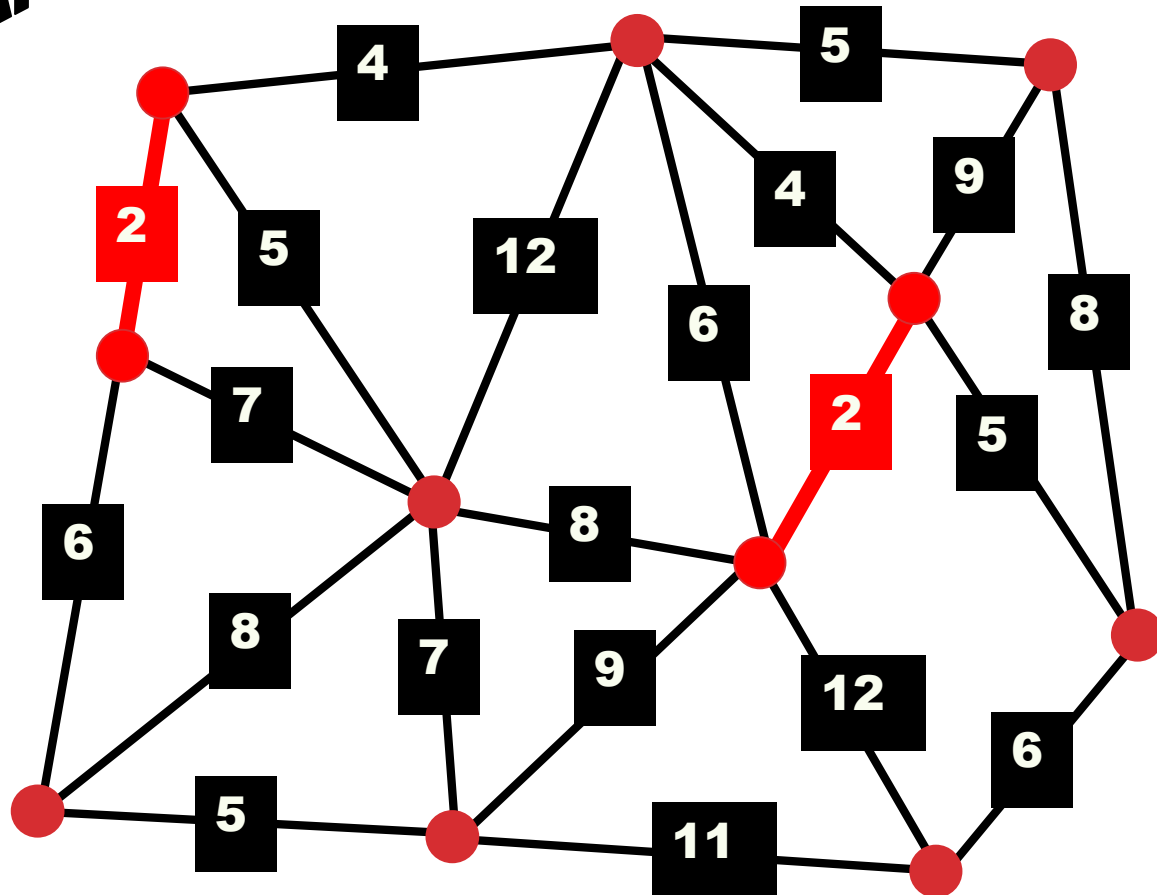
Kruskal's Algorithm

1 Given a network..... 	2 Choose the shortest edge (if there is more than one, choose any of the shortest)..... 	3 Choose the next shortest edge and add it..... 
4 Choose the next shortest edge which wouldn't create a cycle and add it. 	5 Choose the next shortest edge which wouldn't create a cycle and add it. 	6 Repeat until you have a minimal spanning tree. 

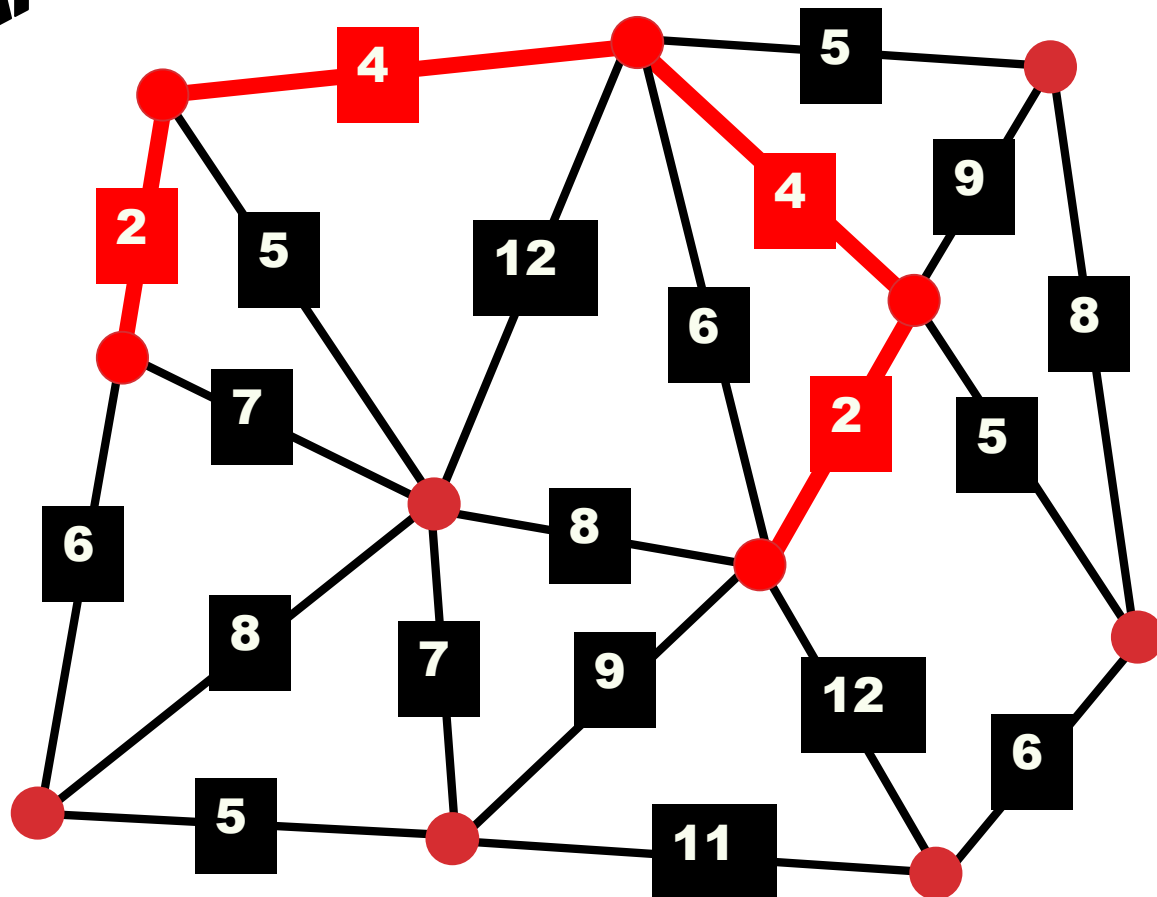
The Same Graph



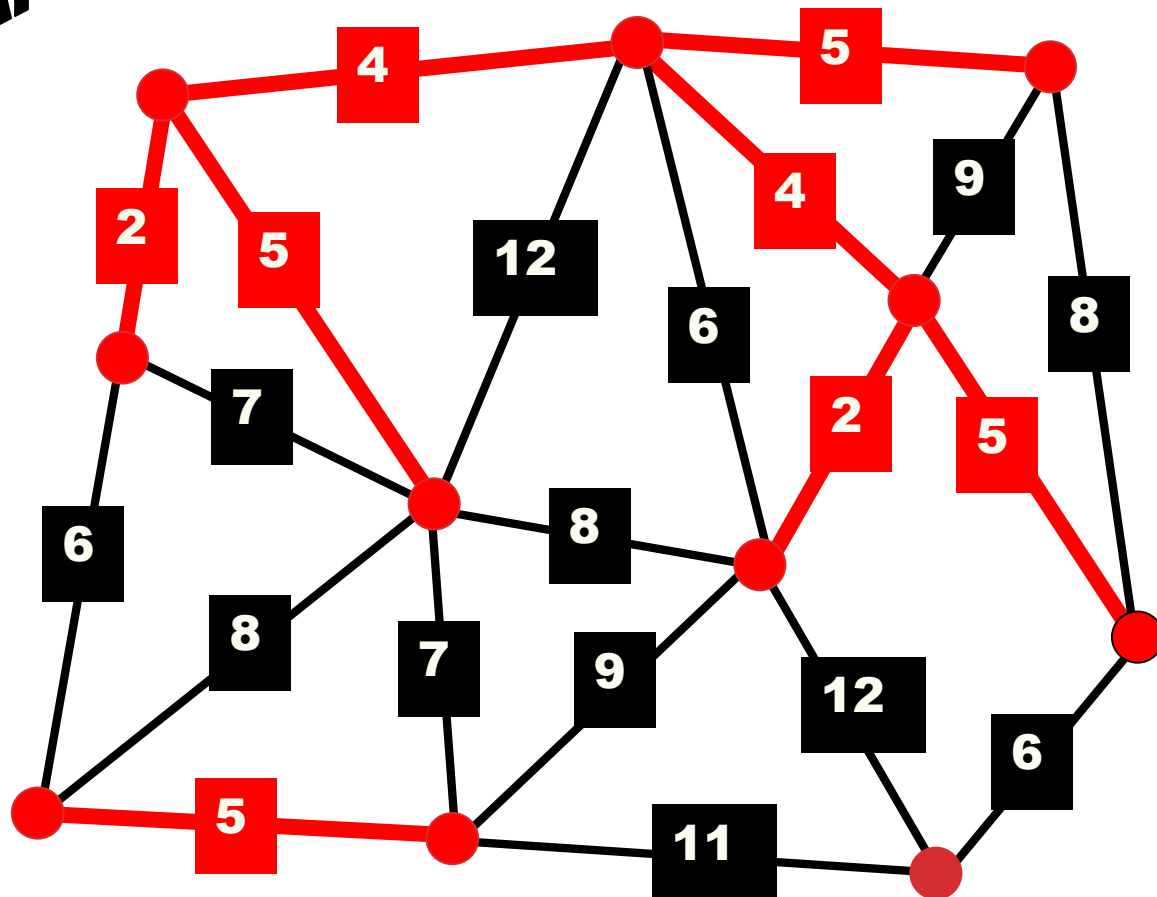
Kruskal's



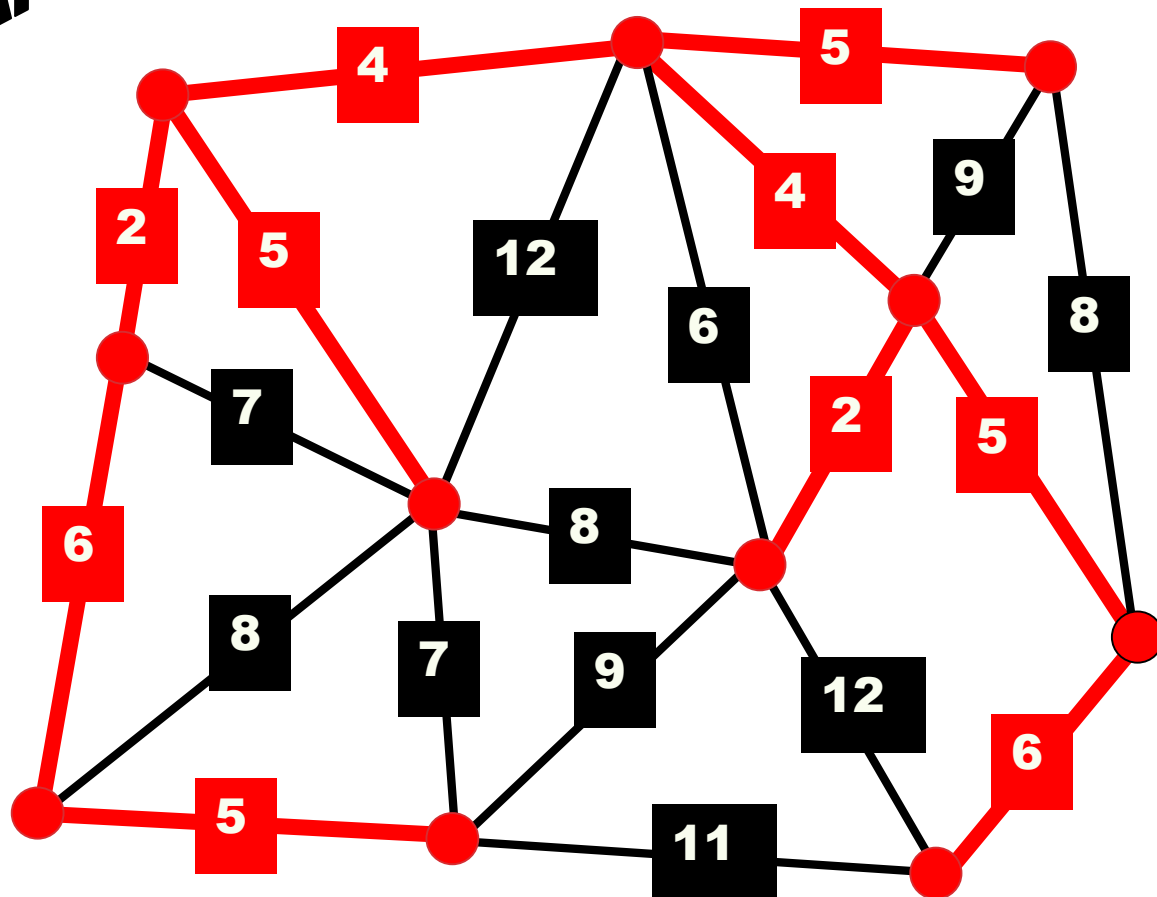
Kruskal's



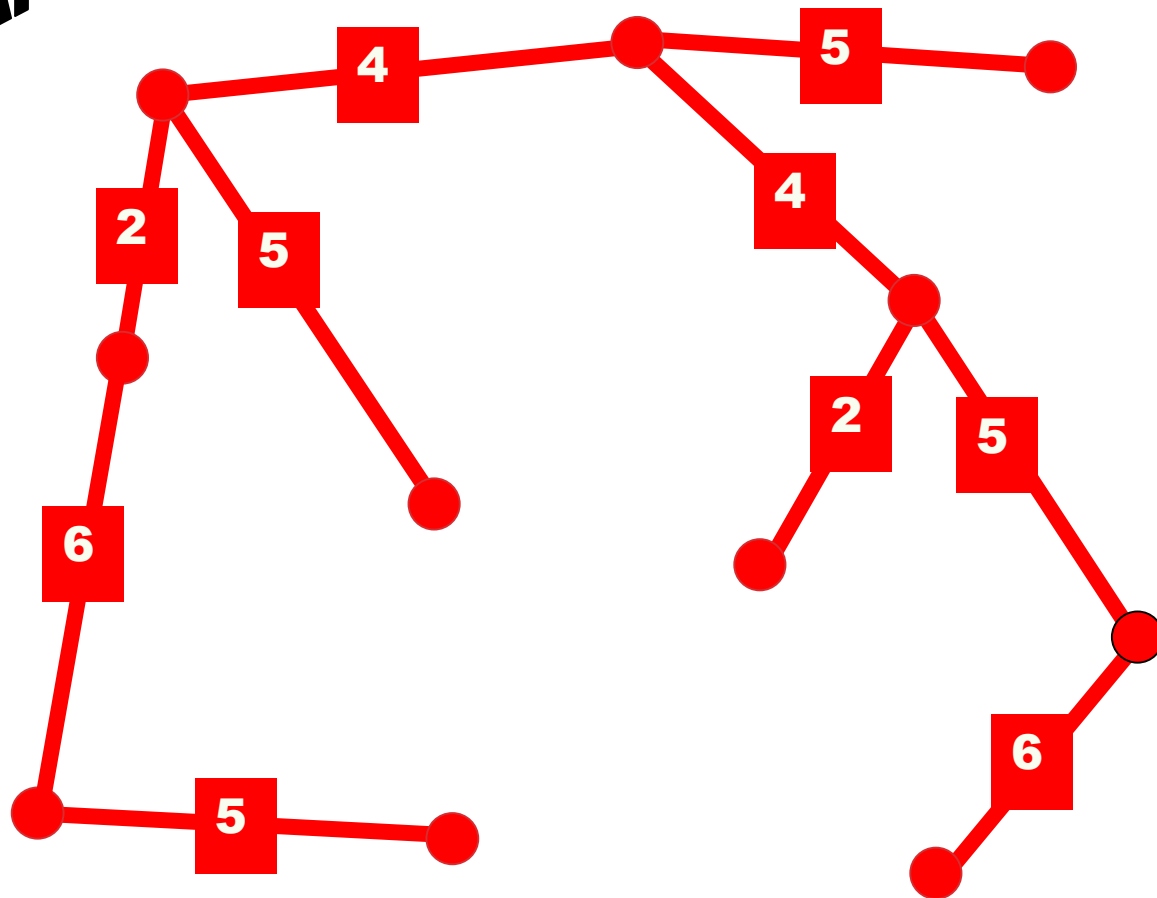
Kruskal's



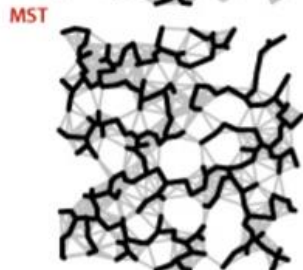
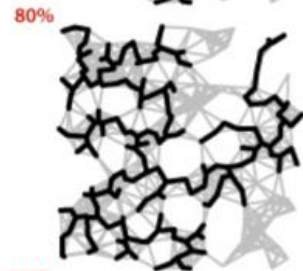
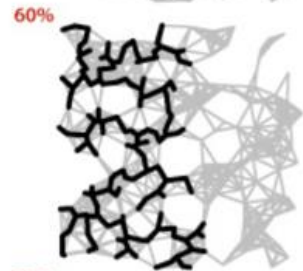
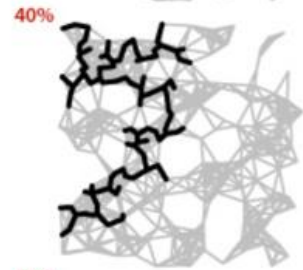
Kruskal's



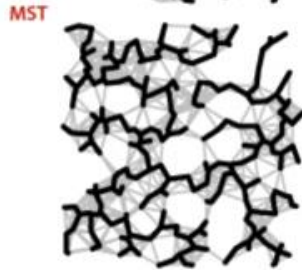
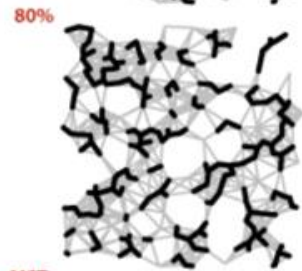
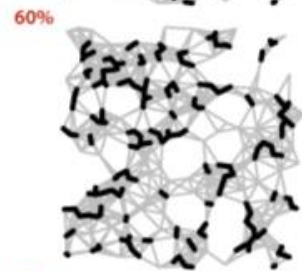
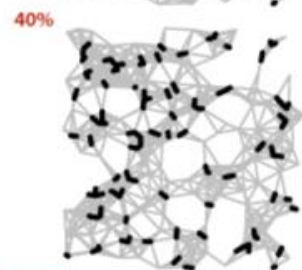
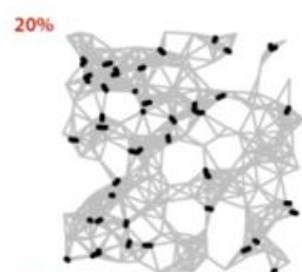
Kruskal's



44



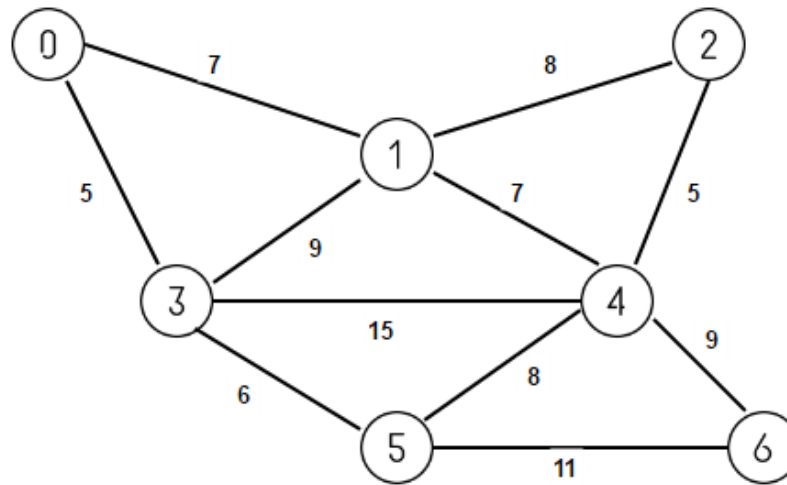
Prim



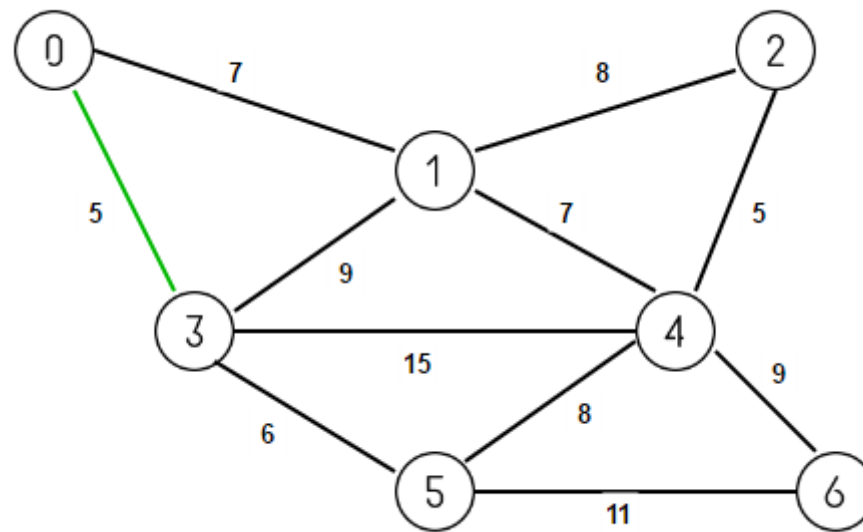
Kruskal

Example

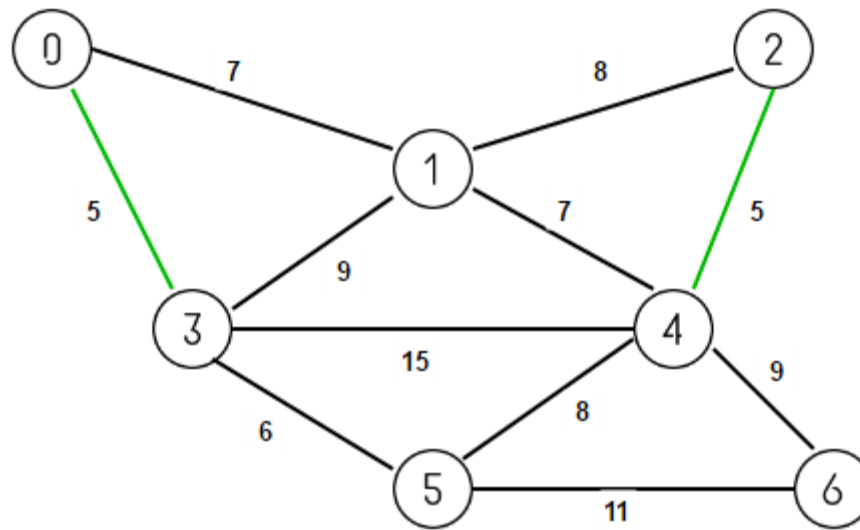
Given a connected and weighted undirected graph, construct a minimum spanning tree?



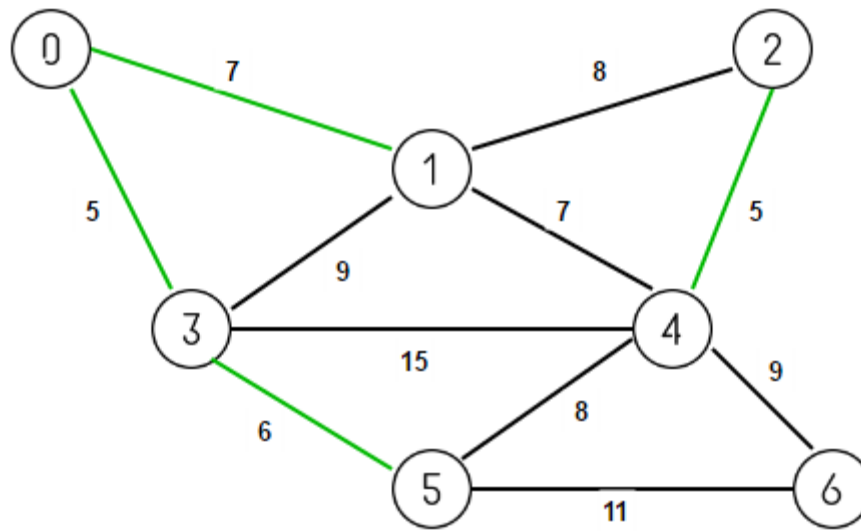
Example



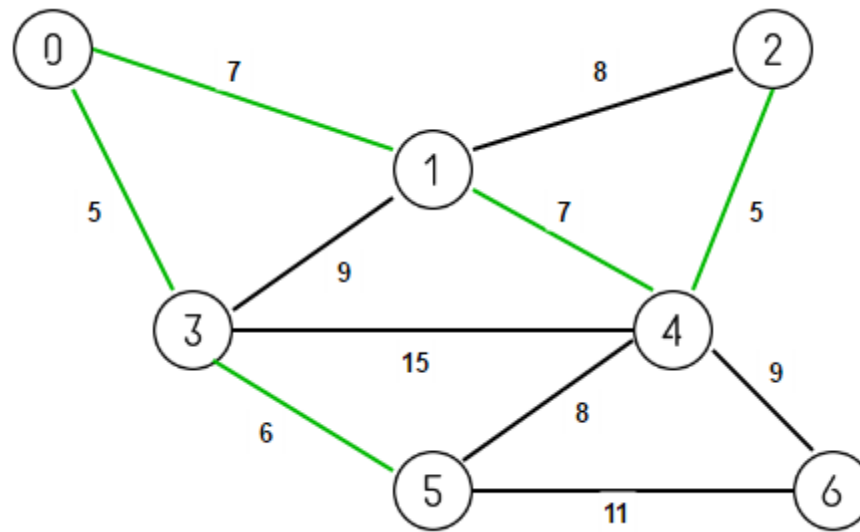
Example



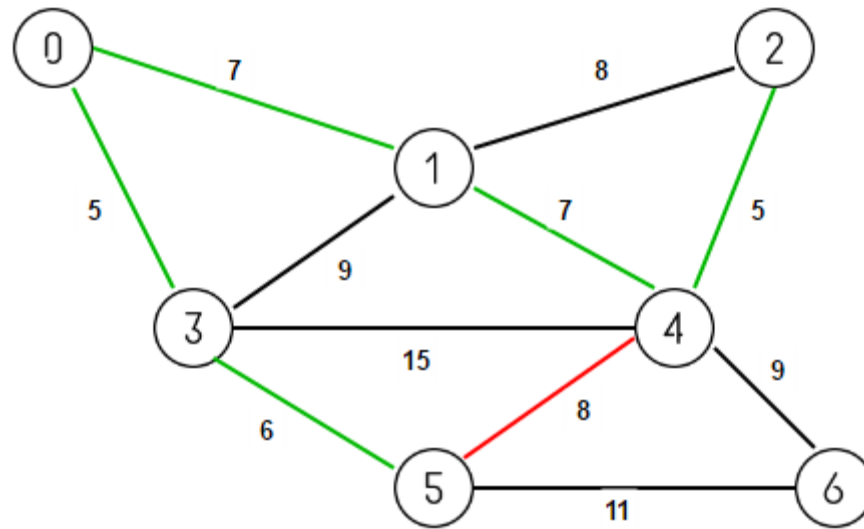
Example



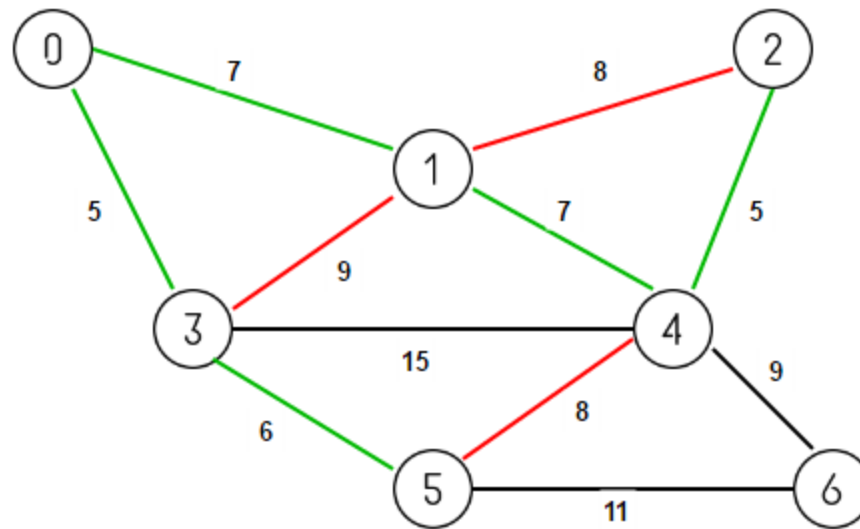
Example



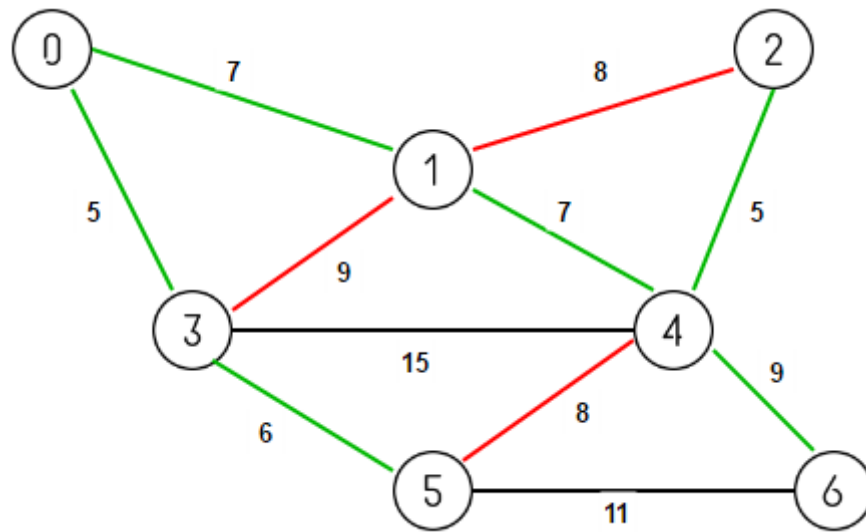
Example



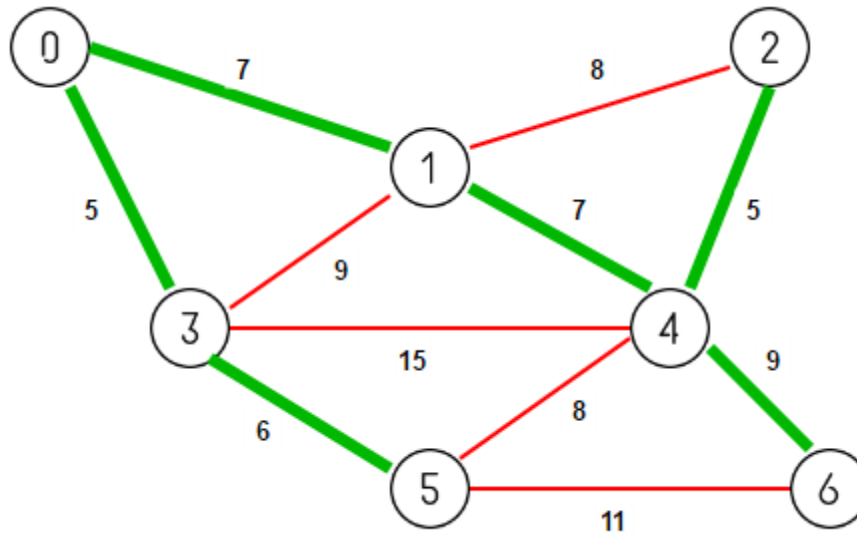
Example



Example

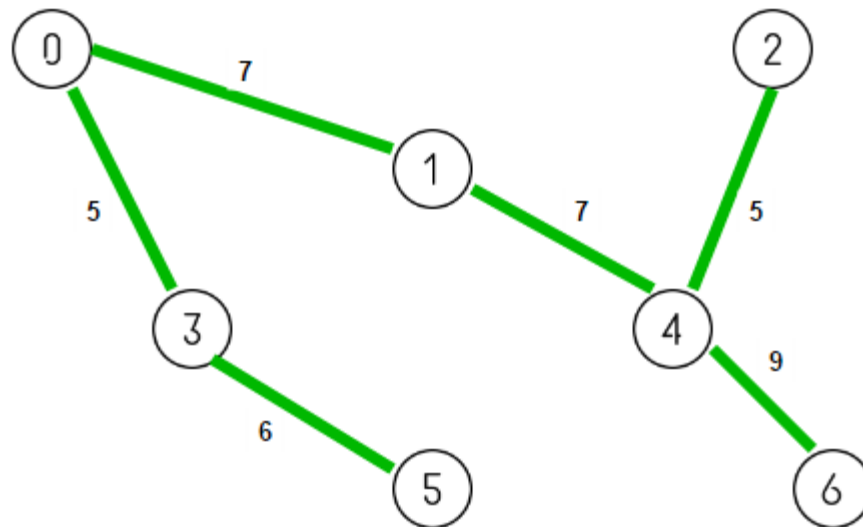


Example



Example

The answer is $6+5+7+7+9+5$



Next...

- Greedy Algorithm uses Kruskal's algorithm

4.5 Network Flow Problems

A network flow is a direct graph without cycle where :

- each edge u has a capacity $c(u) \geq 0$,
- 2 particular nodes **source** and **sink**, the first one has no predecessors while the second one has no successors.

A flow φ of a network is an amount $\varphi(u)$ associated to each edge u such that :

- $\varphi(u) \geq 0, u \in U$.
- $\sum_{u \in U_x^-} \varphi(u) - \sum_{u \in U_x^+} \varphi(u) = 0$ if $x \neq source, x \neq sink$.
- $\varphi(u) \leq c(u)$.

U_x^- : the set of edges (i, x) .

U_x^+ : the set of edges (x, j) .

4.5 Network Flow Problems : Ford-Fulkerson's Algorithm

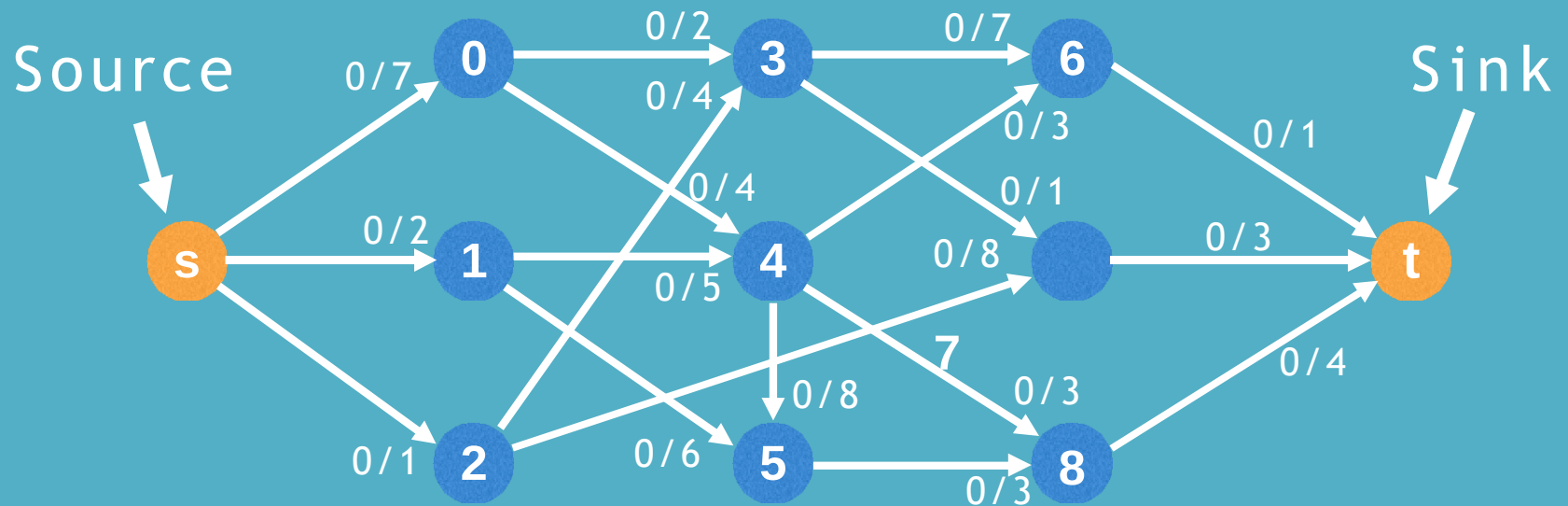
The goal of the algorithm is to look for a maximum flow from **source** to **sink**.

1. Look for an initial complete flow (flow such that each path from **source** to **sink** includes at least one saturated (filled) edge).
2. Improve the initial flow :
 - (a) Look for an improving chain by using Ford-Fulkerson's labeling.
 - i. Mark (by $+i$) the node j of any non saturated edge (i, j) such that i is marked.
 - ii. Mark (by $-l$) the node k of any edge (k, l) having a non empty flow and such that l is marked.
 - (b) If we cannot mark the **sink** then we have obtained a maximum flow.
 - (c) Otherwise, complete the marked chain and goto (a).

If the capacities are all integer and the max flow is C , then the total running time is $O(C|E|)$. This running time can be improved when using a variant of Dijkstra algorithm to select the augmented paths.

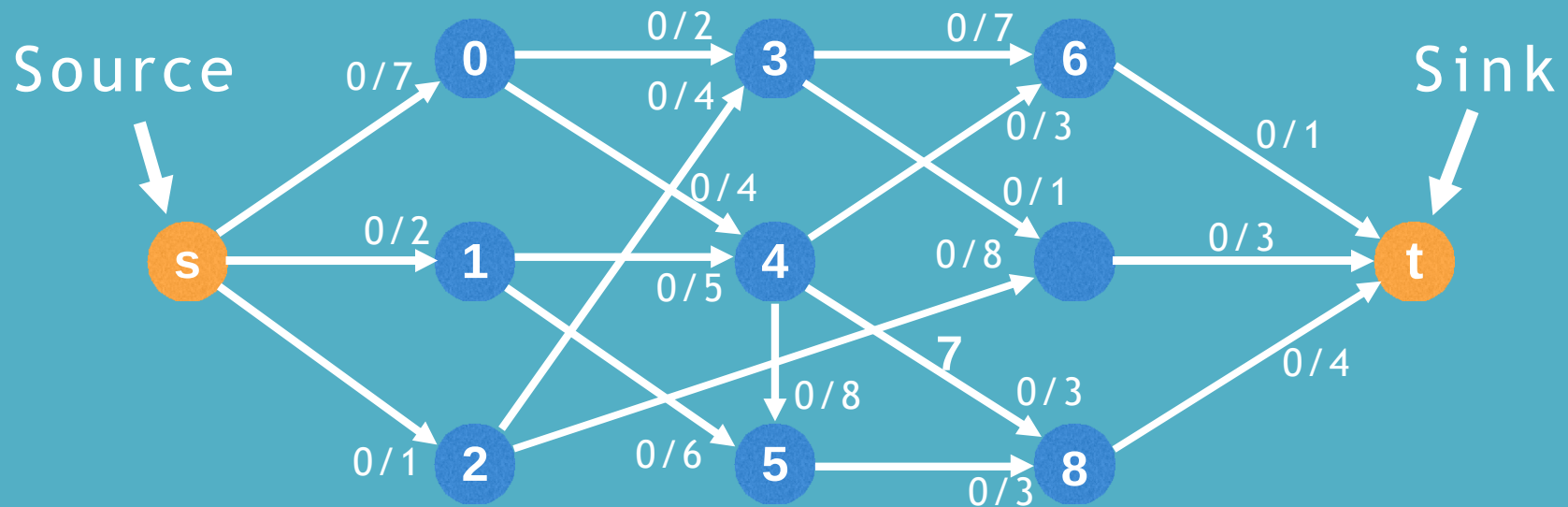
Max flow

Q: With an infinite input source, how much “flow” can we push through the network given that each edge has a certain capacity?



Max flow

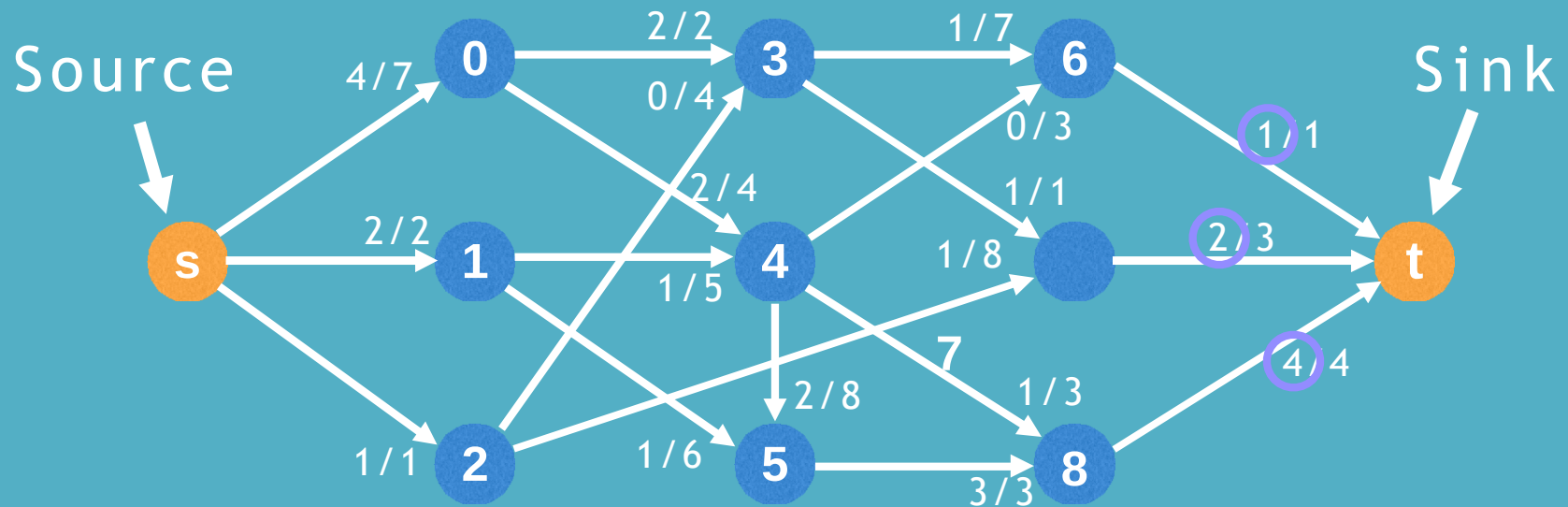
Q: With an infinite input source, how much “flow” can we push through the network given that each edge has a certain capacity?



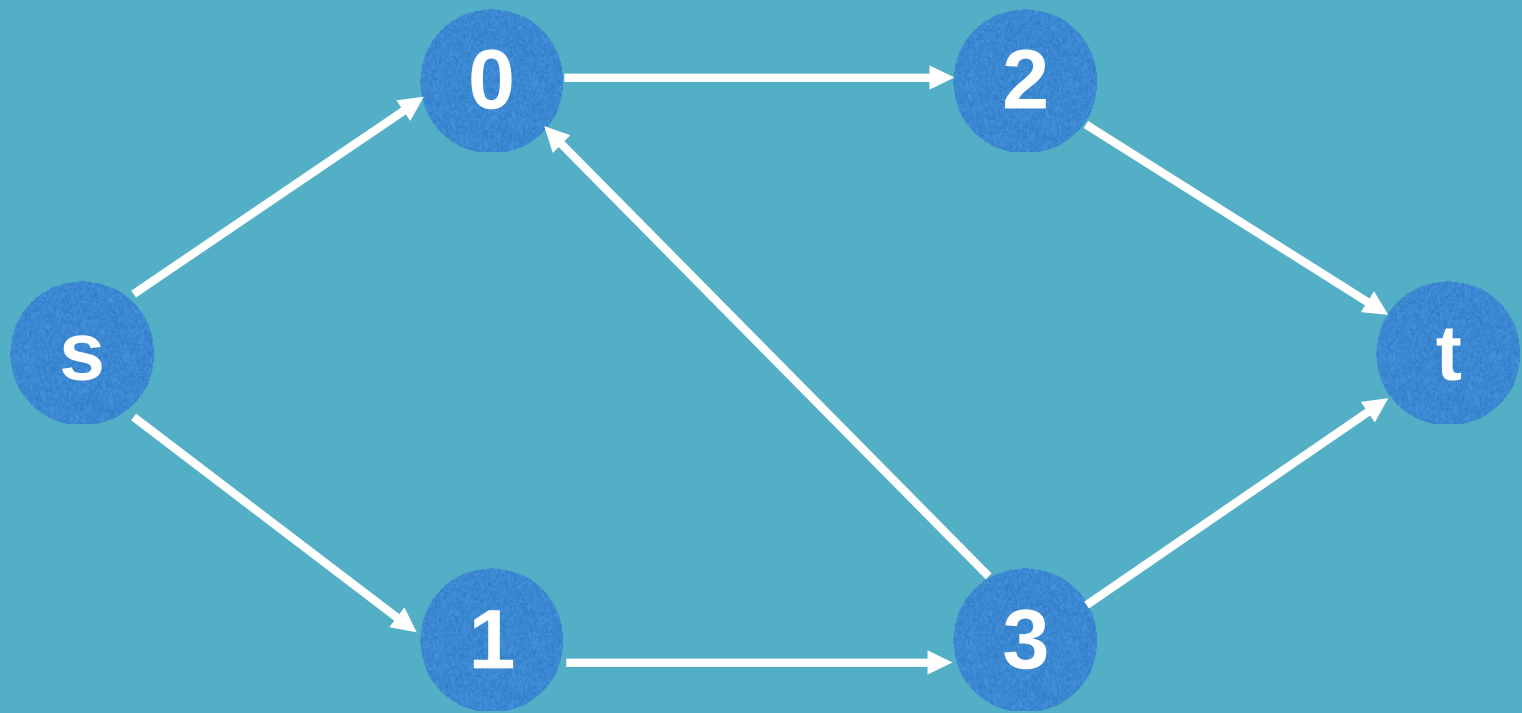
Suppose the edges are roads with cars, pipes with water or wires with electric current. Flow represents the volume of water allowed to flow through the pipes, the number of cars the roads can sustain in traffic and net electric current. Effectively, it's the “bottleneck” value for the amount of flow that can pass through the network from source to sink under all the constraints.

Max flow

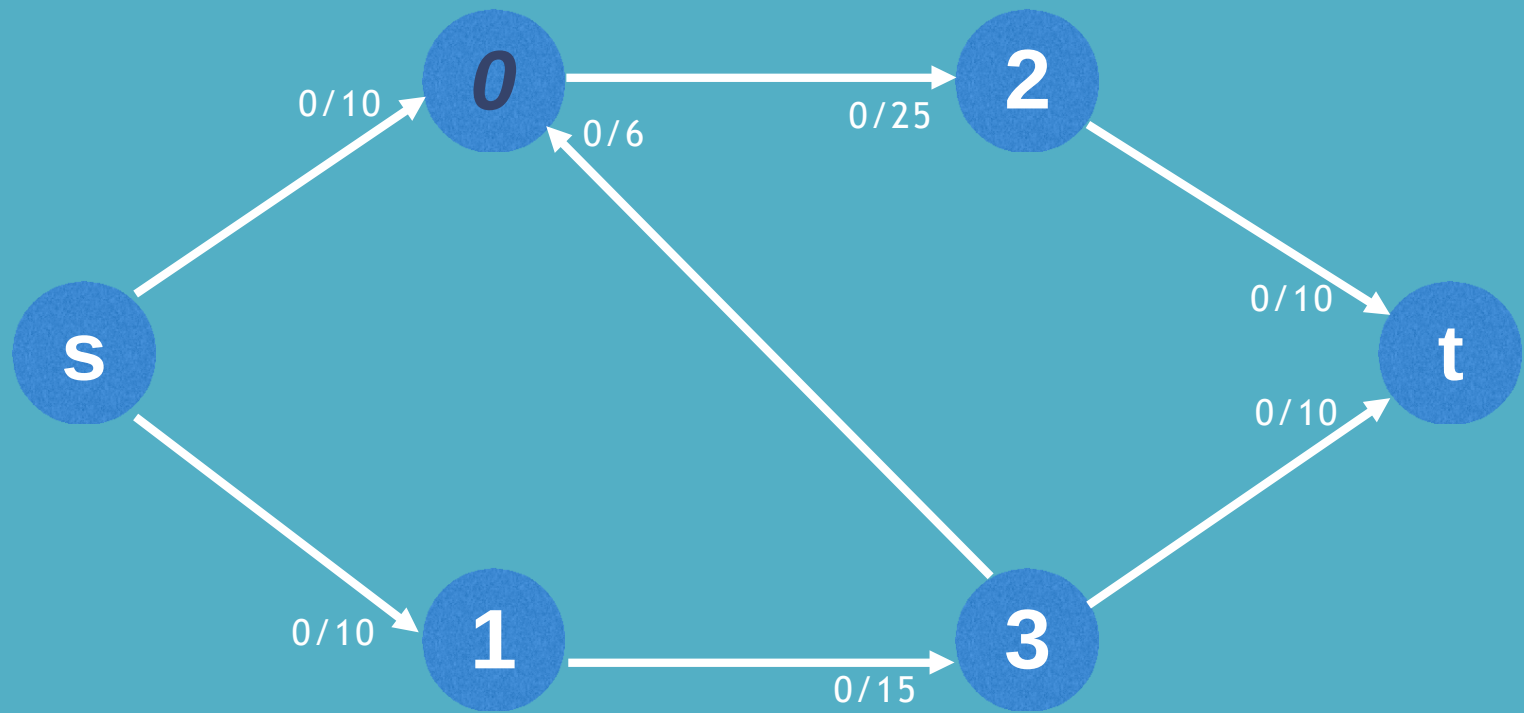
Q: With an infinite input source, how much “flow” can we push through the network given that each edge has a certain capacity?



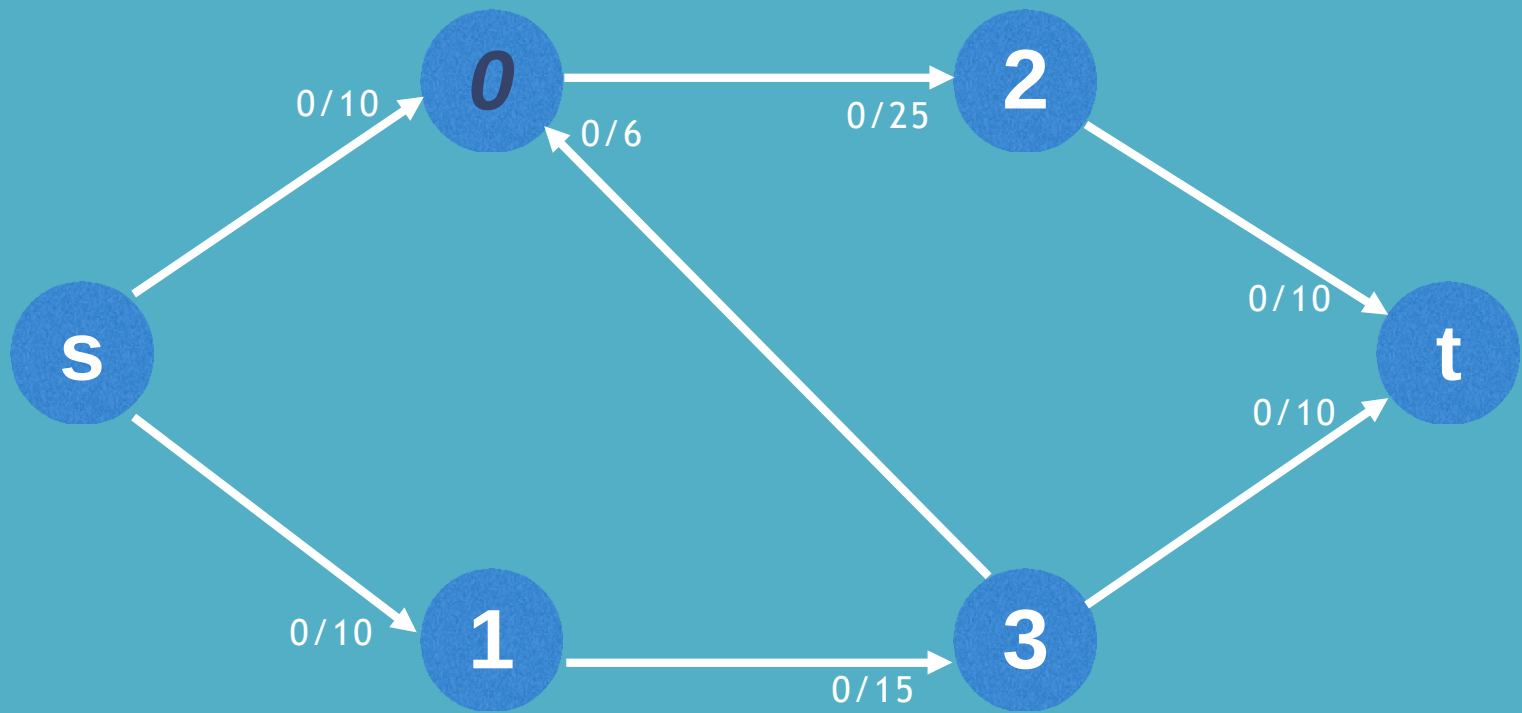
In this flow graph, the max flow is 7 since $1+2+4=7$ is the sum of the flows entering the sink node which is the most we can push through the network.



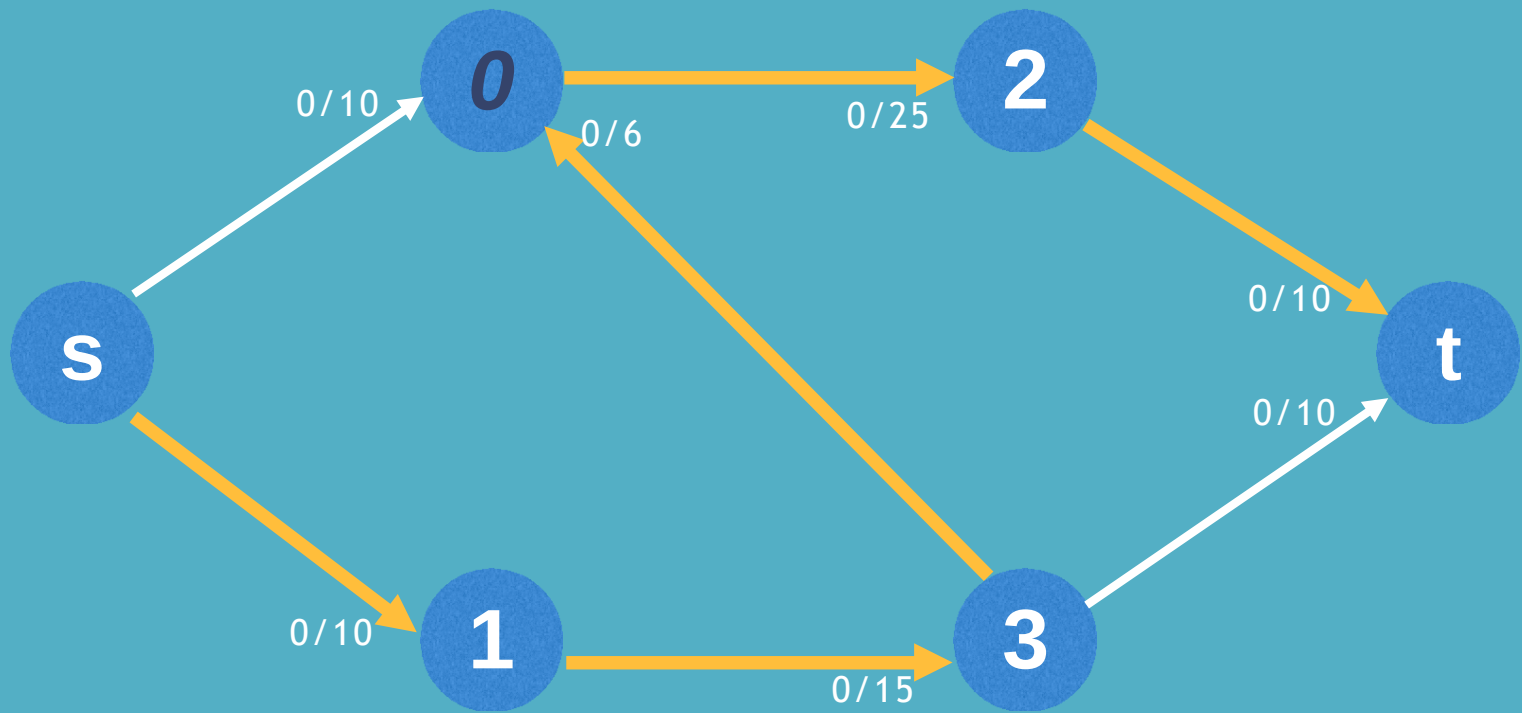
A **flow graph** (flow network) is a directed graph where each edge (also called an arc) has a certain **capacity** which can receive a certain amount of **flow**. The flow running through an edge must be less than or equal to the capacity.



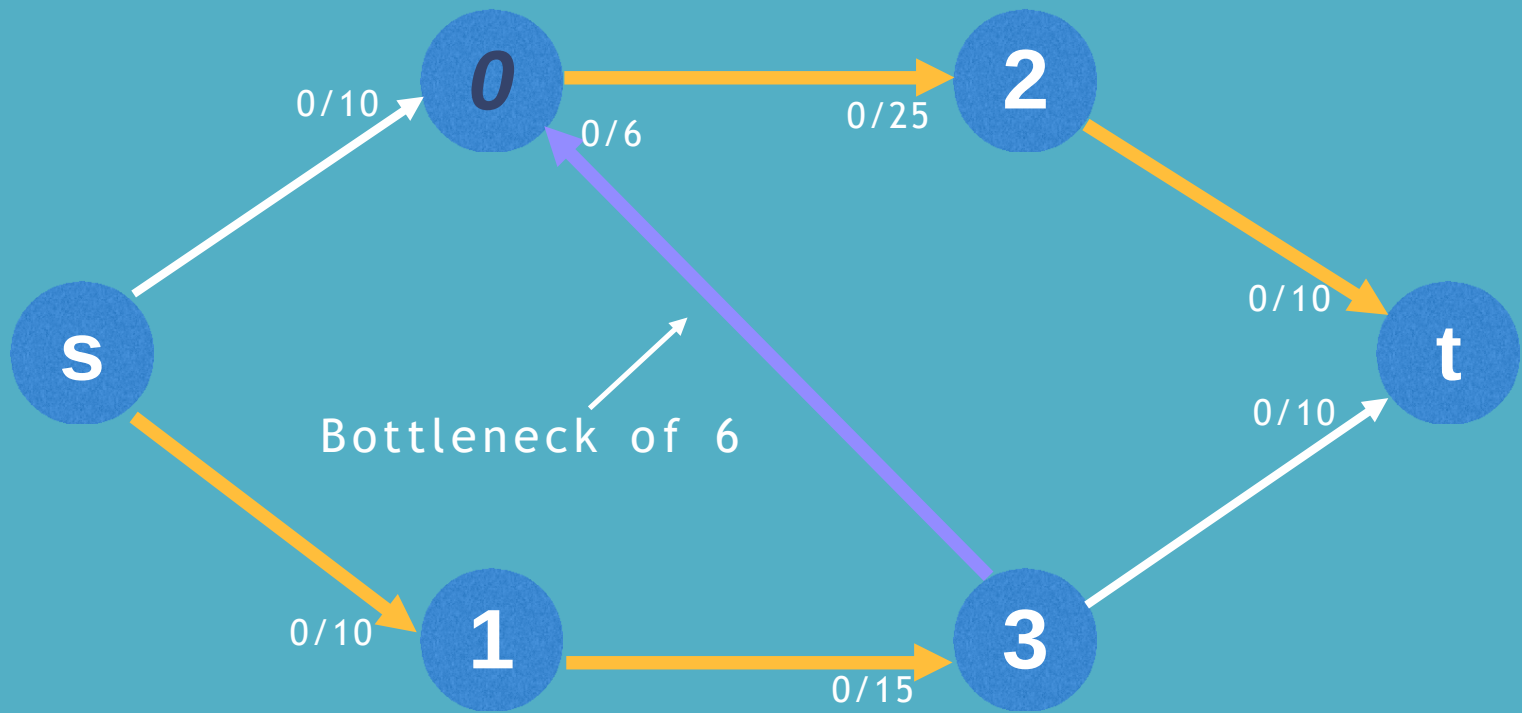
Each edge in the flow graph has a certain flow and capacity specified by the fraction adjacent to each edge. Initially, the flow through each edge is 0 and the capacity is a non negative value.



To find the maximum flow (and min-cut as a by product), the Ford-Fulkerson method repeatedly finds **augmenting paths** through the **residual graph** and **augments the flow** until no more augmenting paths can be found.



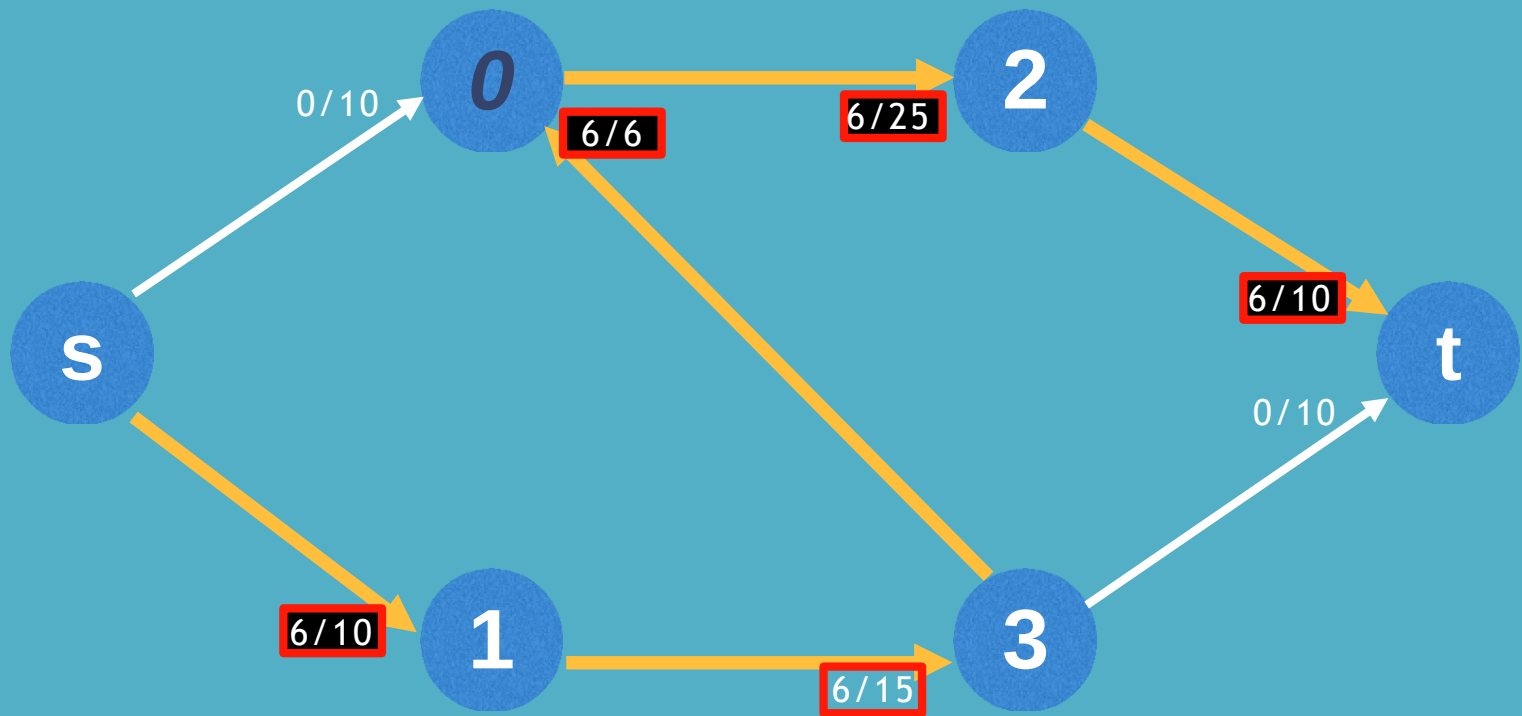
An **augmenting path** is a path of edges in the residual graph with unused capacity greater than zero from the source s to the sink t .



In the augmenting path above, the bottleneck is the “smallest” edge on the path. We can use the bottleneck value to augment the flow along the path.

The smallest remaining capacity of any edge along the augmenting path is:

$$\min(10-0, \quad 15-0, \quad 6-0, \quad 25-0, \quad 10-0) = \min(10, 15, 6, 25, 10) = 6$$



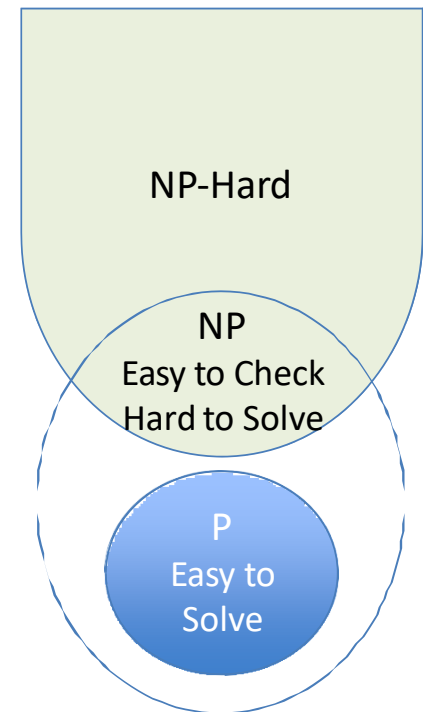
Augmenting the flow means *updating* the flow values of the edges along the augmenting path.

For forward edges, this means
increasing the flow by the
bottleneck value.

4.8 Introduction to NP-Completeness

- Easy versus Hard
- The Class NP
- NP-Complete Problems

	P	NP	NP-complete	NP-hard
Solvable in P	X			
Sol. Verifiable in P	X	X	X	
Reduces any NP prob. In P time			X	X



The class P

- The class P consists of those problems that are solvable in polynomial time.
- More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k , where n is the size of the input to the problem
- The key is that n is the **size of input**

NP

- NP is not the same as non-polynomial complexity/running time. NP does not stand for not polynomial.
- NP = Non-Deterministic polynomial time
- NP means verifiable in polynomial time
- Verifiable?
 - If we are somehow given a 'certificate' of a solution we can verify the legitimacy in polynomial time

Complexity Class P

- Deterministic in nature
- Solved by conventional computers in polynomial time
 - $O(1)$ Constant
 - $O(\log n)$ Sub-linear
 - $O(n)$ Linear
 - $O(n \log n)$ Nearly Linear
 - $O(n^2)$ Quadratic
- Polynomial upper and lower bounds

Complexity Class NP

- Non-deterministic part as well
- `choose(b)`: choose a bit in a non-deterministic way and assign to `b`
- If someone tells us the solution to a problem, we can verify it in polynomial time
- Two Properties: non-deterministic method to generate possible solutions, deterministic method to verify in polynomial time that the solution is correct.

What happened to automata?

- Problem is in NP if it is decidable by some non deterministic Turing machine in polynomial time.
- Remember that the model we have used so far is a deterministic Turing machine
- It is provable that a Non Deterministic Turing Machine is equivalent to a Deterministic Turing Machine
- The deterministic version of a poly time non deterministic Turing machine will run in exponential time (worst case)

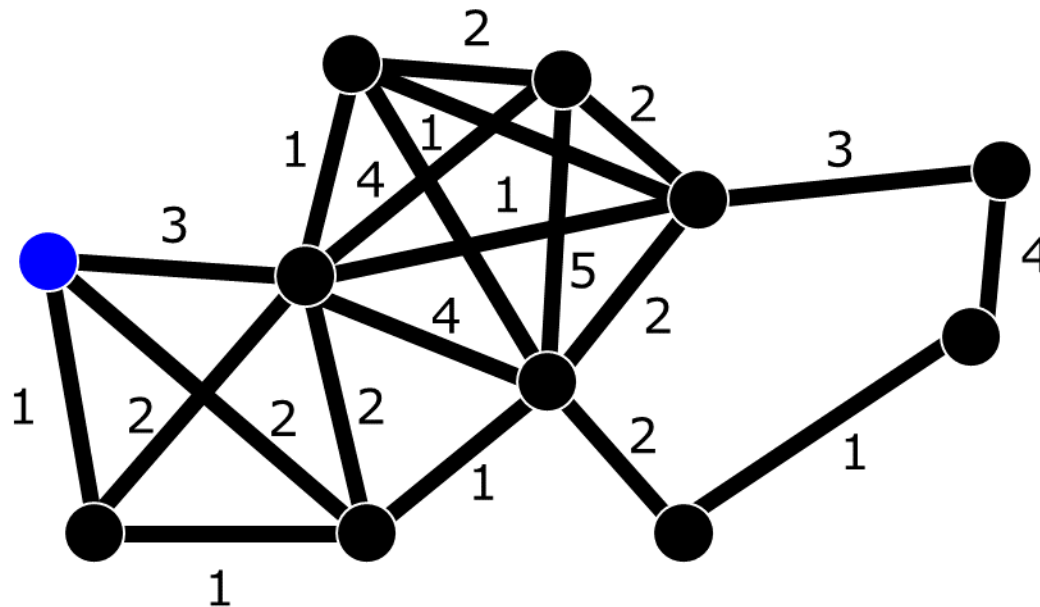
NP problems

- Graph theory has these fascinating(annoying?) pairs of problems
 - Shortest path algorithms?
 - Longest path is NP complete (we'll define NP complete later)
 - Eulerian tours (visit every vertex but cover every edge only once, even degree etc). Solvable in polynomial time!
 - Hamiltonian tours (visit every vertex, no vertices can be repeated). NP complete

Hamiltonian cycles

- Determining whether a directed graph has a Hamiltonian cycle does not have a polynomial time algorithm (yet!)
- However if someone was to give you a sequence of vertices, determining whether or not that sequence forms a Hamiltonian cycle can be done in polynomial time
- Therefore Hamiltonian cycles are in NP

TSP



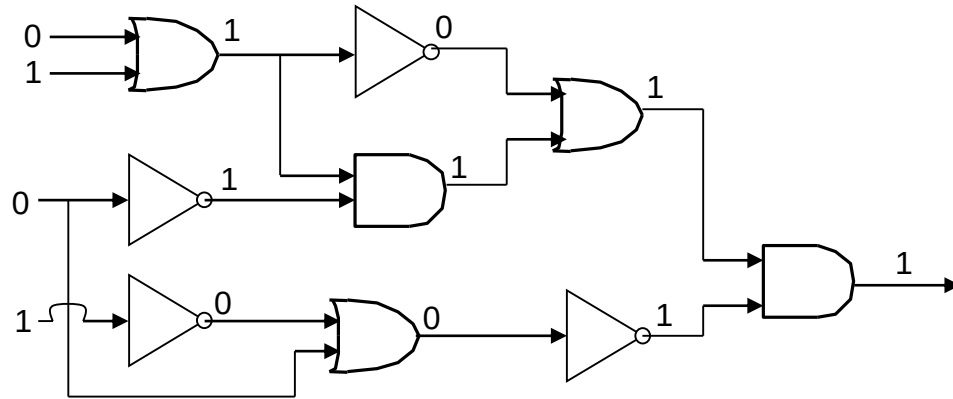
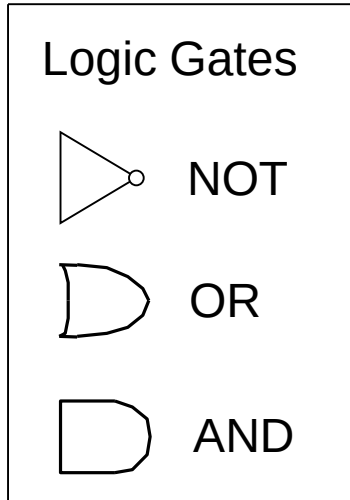
$i = 23$

- For each two cities, an integer cost is given to travel from one of the two cities to the other. The salesperson wants to make a minimum cost circuit visiting each city exactly once.

P is a subset of NP

- Since it takes polynomial time to run the program, just run the program and get a solution
- But is NP a subset of P?
- No one knows if $P = NP$ or not
- Solve for a million dollars!
 - <http://www.claymath.org/millennium-problems>
 - The Poincare conjecture is solved today

Circuit-SAT



- Take a Boolean circuit with a single output node and ask whether there is an assignment of values to the circuit's inputs so that the output is "1"

What is not in NP?

- Undecidable problems
 - Given a polynomial with integer coefficients, does it have integer roots
 - Hilbert's nth problem
 - Impossible to check for all the integers
 - Even a non-deterministic TM has to have a finite number of states!
 - More on decidability later
- Tautology
 - A boolean formula that is true for all possible assignments
 - Here just one 'verifier' will not work. You have to try all possible values

Amusing analogy

- Students believe that every problem assigned to them is NP-complete in difficulty level, as they have to find the solutions.
- Teaching Assistants, on the other hand, find that their job is only as hard as NP, as they only have to verify the student's answers.
- When some students confound the TAs, even verification becomes hard

Reducibility

- a problem Q can be reduced to another problem Q' if any instance of Q can be “easily rephrased” as an instance of Q' , the solution to which provides a solution to the instance of Q
- Is a linear equation reducible to a quadratic equation?
 - Sure! Let coefficient of the square term be 0

NP - hard

- What are the hardest problems in NP?

$$L_1 \leq_p L_2$$

- That notation means that L_1 is reducible in polynomial time to L_2 .
- The less than symbol basically means that the time taken to solve L_1 is no worse than a polynomial factor away from the time taken to solve L_2 .

NP-hard

- A problem (a language) is said to NP-hard if every problem in NP can be poly time reduced to it.

$$L' \leq_p L \text{ for every } L' \in NP$$

NP Complete problems/languages

- Need to be in NP
- Need to be in NP-Hard

If both are satisfied then it is an NP complete problem

Reducibility is a transitive relation.

If we know a single problem in NP-Complete that helps when we are asked to prove some other problem is NP-Complete

Assume problem P is NP Complete

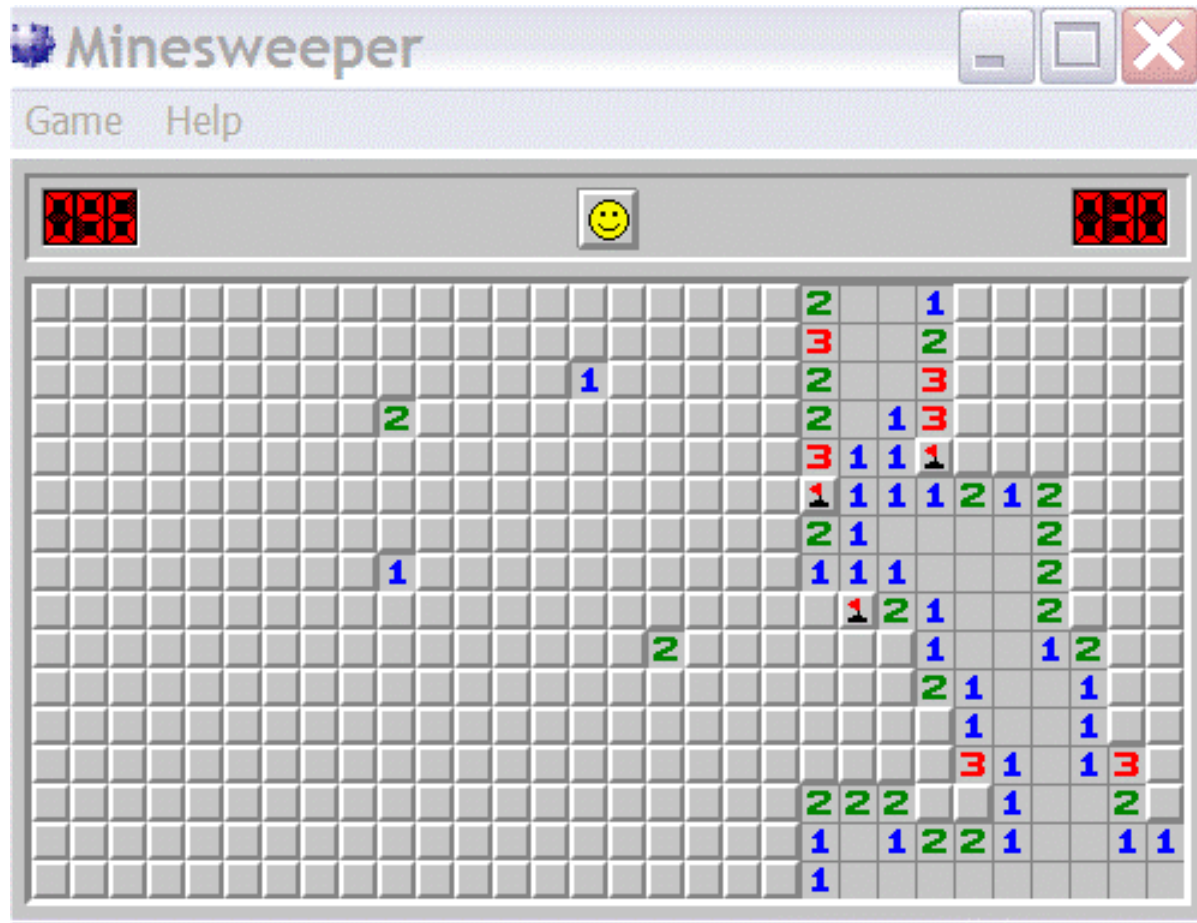
All NP problems are reducible to this problem

Now given a different problem P'

If we show P reducible to P'

Then by transitivity all NP problems are reducible to P'

Minesweeper



Minesweeper

- The Minesweeper consistency problem:
 - *Given a rectangular grid partially marked with numbers and/or mines – some squares being left blank – determine if there is some pattern of mines in the blank squares that give rise to the numbers seen. That is, determine if the grid is consistent for the rules of the game.*

Minesweeper

- This is certainly an NP problem:
 - We can guess all possible configurations of mines in the blank squares and see if any work.
- To see that it is NP complete is much harder.
 - The trick is to code logical expressions into partially filled minesweeper grids. Then we will have demonstrated that SAT can be coded into Minesweeper, so Minesweeper is also NP-complete.

Tetris

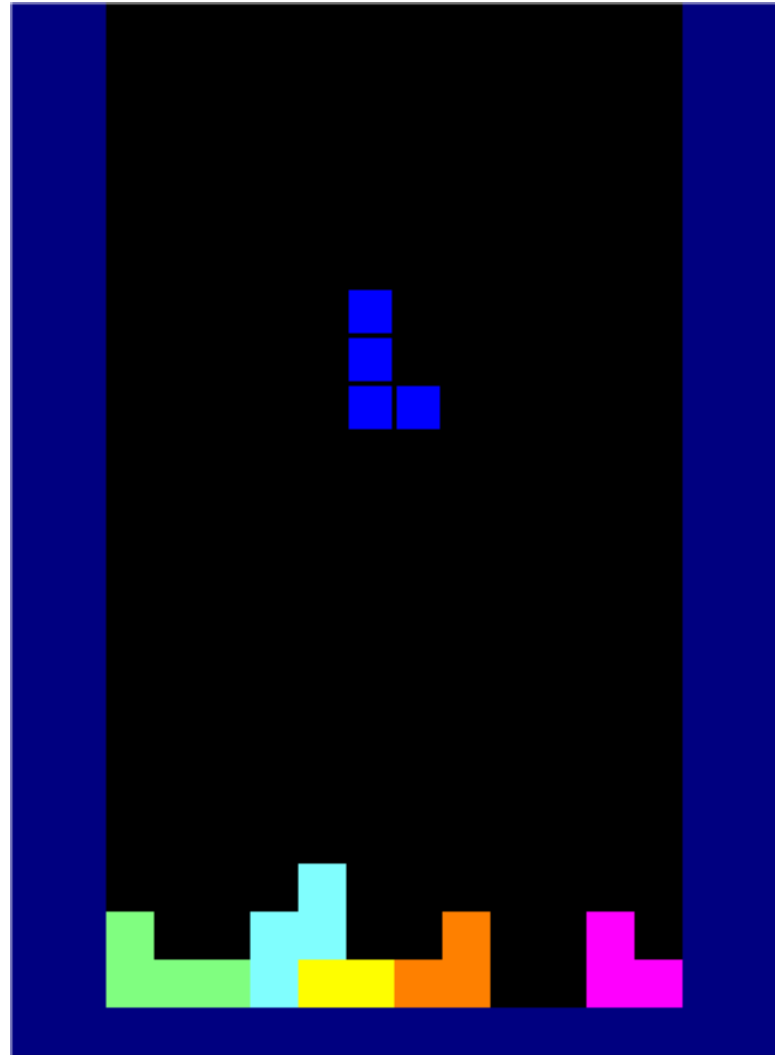
Rules:

The pieces fall, you can rotate them.

When a line gets all filled up, it is “cleared”.

If 4 lines get filled simultaneously, that’s a “Tetris” – extra points!

You lose if the screen fills up so no new pieces can appear.



Tetris, the "offline" version

- You get to know all the pieces (there will only be finitely many in this version), and the order they will appear.
- You start with a partially filled board.

Tetris, offline

- The following problems are NP-complete:
 - Maximizing the number of rows cleared.
 - Maximizing the number of pieces placed before losing.
 - Maximizing the number of Tetrises.
 - Maximizing the height of the highest filled grid-square over the course of the sequence.

Tetris, offline

- It's easy to see each of these is NP – just guess the different ways to rotate and place each piece, and see which is largest.
- It's (as usual) harder to show NP-completeness.

Sudoku

Each row/column/square has a single instance of each number 1-9.

Solving a board is NP-complete.

(To show completeness, have to do Samuri Sudoku on steroids!)

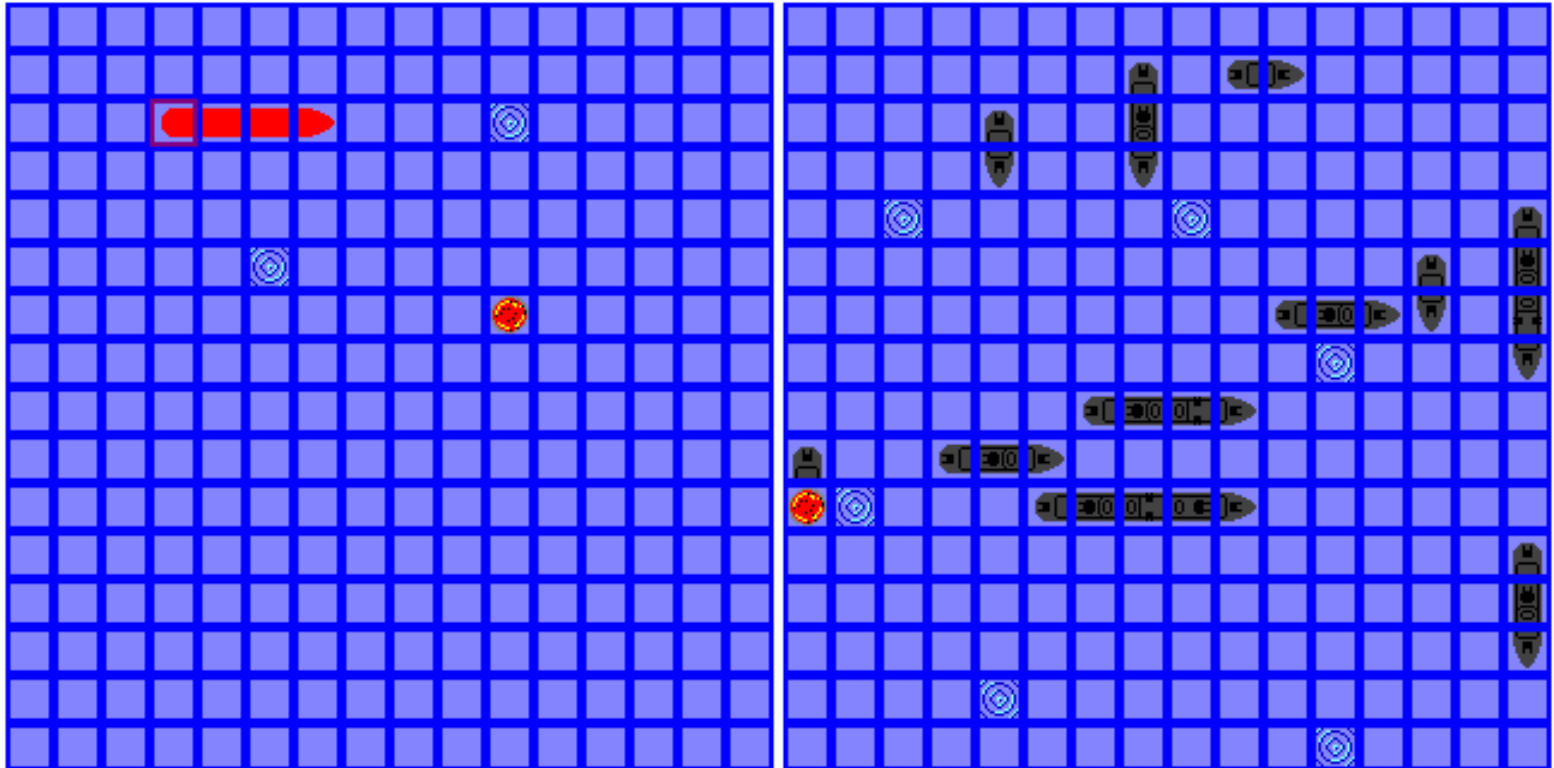
5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

(Due to Takyuki Yato and Takahiro Seta at the University of Tokyo)

2	3			7	5				3	4		7	2	
	7			1							2		8	
	4			3	8				8		5		1	
3			6	8		9			5				3	
1			7	2	6	5			6	2		4		1
		5								7				
		8					1	6					6	5
9					2				1	8	9			
4	6				8		2	3	4		8	1	3	7
				2						9				
					4				6	3				
					5				2	1				
8					4									7
3	5			7		9			8	3	5	7	8	2
	4		2	5	8			5					7	1
						8						2		9
	6	9			4	2					5		9	
					1						6		8	3
6	7	8				1			8	2	4		7	
4		2				5						9	2	
	1	7	3							4	3		5	

Battleship

- Sink the enemy's ships – NP Compete!



NP-Complete

- It is also interesting to look at the relationship between some easy problems and hard ones:

Hard Problems (NP-Complete)	Easy Problems (in P)
SAT, 3SAT	2SAT
Traveling Salesman Problem	Minimum Spanning Tree
3D Matching	Bipartite Matching
Knapsack	Fractional Knapsack